

Xây dựng dự án IoT

Zero Code với Tasmota



Lời mở đầu

Chào mừng các bạn đến với hành trình khám phá thế giới Internet of Things (IoT) đầy thú vị và tiềm năng! Trong kỷ nguyên số hóa, IoT đang ngày càng trở nên phổ biến, len lỏi vào mọi khía cạnh của cuộc sống, từ ngôi nhà thông minh, thành phố thông minh cho đến các ứng dụng công nghiệp tiên tiến.

Có lẽ khi nhắc đến IoT, nhiều bạn sẽ nghĩ ngay đến những dòng code phức tạp, những kiến thức chuyên sâu về lập trình. Tuy nhiên, cuốn sách này sẽ mang đến một góc nhìn hoàn toàn mới: xây dựng dự án IoT một cách dễ dàng và trực quan, hoàn toàn không cần đến một dòng code nào!

Chúng ta sẽ cùng nhau khám phá sức mạnh của firmware Tasmota và vi điều khiển ESP32 – bộ đôi hoàn hảo cho phép bạn hiện thực hóa những ý tưởng IoT sáng tạo một cách nhanh chóng và hiệu quả. Tasmota là một firmware mã nguồn mở, mạnh mẽ, được thiết kế đặc biệt để điều khiển các thiết bị thông minh thông qua giao diện web trực quan. ESP32, với khả năng kết nối Wi-Fi và Bluetooth mạnh mẽ, là trái tim của nhiều dự án IoT hiện đại.

Cuốn sách này được biên soạn dành riêng cho các bạn học sinh, sinh viên, những người mới bắt đầu làm quen với lĩnh vực IoT. Chúng tôi sẽ dẫn dắt các bạn từng bước, từ những khái niệm cơ bản nhất về Tasmota và ESP32, cách cài đặt và cấu hình, cho đến việc tích hợp chúng với các nền tảng IoT phổ biến để xây dựng những dự án thực tế và hữu ích.

Chúng tôi tin rằng, với các kiến thức nền tảng được giới thiệu, hướng dẫn chi tiết từng bước và các ví dụ minh họa cụ thể trong cuốn sách này, các bạn sẽ tự tin khám phá và chinh phục thế giới IoT đầy tiềm năng. Hãy sẵn sàng để biến những ý tưởng sáng tạo của bạn thành hiện thực, mà không cần phải lo lắng về những dòng code khô khan!

Chúc các bạn có một hành trình học tập và khám phá IoT thật thú vị và thành công!

MỤC LỤC

Module 1: Giới thiệu về Internet of Things (IoT).....	5
Mục tiêu.....	5
1.1 IoT là gì? Các khái niệm cốt lõi và thuật ngữ.....	5
1.2 Tại sao IoT quan trọng?.....	6
1.3 Các thành phần chính của một hệ thống IoT.....	8
1.4 Giới thiệu về Vi điều khiển cho IoT.....	9
1.5 Local Control vs Cloud Dependency.....	11
Module 2: Giới thiệu về Tasmota.....	13
Mục tiêu.....	13
2.1 Tasmota là gì?.....	13
2.2 Tại sao chọn Tasmota?.....	14
2.3 Các phần cứng được hỗ trợ.....	16
2.4 So sánh Tasmota với Firmware gốc của nhà sản xuất.....	17
Module 3: Cơ bản về Phần cứng & Điện tử.....	20
3.1 Các bo mạch phát triển, sơ đồ chân (Pinouts) và tính năng.....	21
Bo mạch phát triển (Development Board) là gì?.....	21
Giới thiệu bo mạch Yolo UNO.....	22
Giới thiệu bo mạch Yolo Node.....	23
3.2 Các khái niệm Điện tử cơ bản.....	24
3.3 Tổng quan về các linh kiện IoT phổ biến.....	26
Module 4: Chuẩn bị môi trường làm việc.....	31
Mục tiêu.....	31
4.1 Cài đặt Trình điều khiển (Drivers).....	31
4.2 Các công cụ nạp firmware (Flashing Tools).....	35
4.3 Các phiên bản Firmware Tasmota.....	36
4.4 Kết nối thiết bị ESP của bạn với máy tính.....	37
Module 5: Nạp Tasmota vào thiết bị.....	39
Mục tiêu.....	40
5.1 Hướng dẫn nạp bằng Tasmota Web Installer.....	40
5.4 Cấu hình ban đầu cho thiết bị.....	44
5.6 Các vấn đề phổ biến khi nạp firmware.....	47
Module 6: Làm quen giao diện Web Tasmota.....	49
Mục tiêu.....	49
6.1 Truy cập Thiết bị Tasmota của bạn qua địa chỉ IP.....	50
6.2 Khám phá Menu chính và các thông tin hiển thị.....	50
6.3 Tìm hiểu Menu Cấu hình.....	54
6.4 Sử dụng công cụ Console để gửi lệnh & xem Log.....	55
6.5 Sao lưu và khôi phục cấu hình Tasmota.....	57

Module 7: Cấu hình Phần cứng trong Tasmota.....	60
Mục tiêu.....	60
7.1 Templates và Module trong Tasmota.....	60
Tại sao cần cấu hình phần cứng?.....	60
7.2 Cấu hình GPIO thủ công.....	61
7.4 Kiểm tra cấu hình phần cứng của bạn.....	63
Module 8: Giao thức MQTT trong IoT.....	67
Mục tiêu.....	67
8.1 MQTT là gì?.....	67
8.2 Tại sao MQTT cần thiết cho IoT?.....	69
8.3 Thiết lập một MQTT Broker.....	70
8.4 Cấu hình cài đặt MQTT của Tasmota.....	71
8.5 Hiểu cấu trúc các topic MQTT của Tasmota.....	73
8.6 Kiểm tra giao tiếp MQTT.....	74
Module 9: Điều khiển thiết bị và đọc Cảm biến.....	78
Mục tiêu.....	78
9.1 Điều khiển thiết bị Cơ bản qua WebUI, MQTT và HTTP.....	78
9.2 Điều khiển Relay bằng Power command.....	80
9.3 Đọc dữ liệu cảm biến.....	81
9.4 Làm việc với Nút nhấn và Công tắc.....	83
Module 10: Tự động hóa với Tasmota Rules.....	86
Mục tiêu.....	86
10.1 Giới thiệu về Tasmota Rules.....	86
10.2 Cú pháp Rule.....	87
Cấu trúc cơ bản.....	87
Giải thích các thành phần.....	87
Các lệnh để quản lý Rule trên Console.....	89
10.3 Ví dụ một số Rule trong thực tế.....	90
10.4 Gỡ lỗi Rule qua giao diện Console.....	91
Module 11: Tích hợp Tasmota với Core IoT.....	93
Mục tiêu.....	93
11.1 Giới thiệu về Core IoT.....	93
11.2 Thao tác cơ bản với Core IoT Platform.....	94
Tạo Device trên Core IoT.....	96
11.3 Flash Firmware Tasmota hỗ trợ Core IoT.....	98
11.4 Thiết lập MQTT kết nối với Core IoT.....	99
11.5 Tạo bảng điều khiển (Dashboard) trong Core IoT.....	102
Hiển thị thông tin cảm biến nhiệt độ và độ ẩm.....	103
Điều khiển relay và đồng bộ trạng thái.....	104
Module 12: Các tính năng nâng cao.....	108
Mục tiêu.....	108

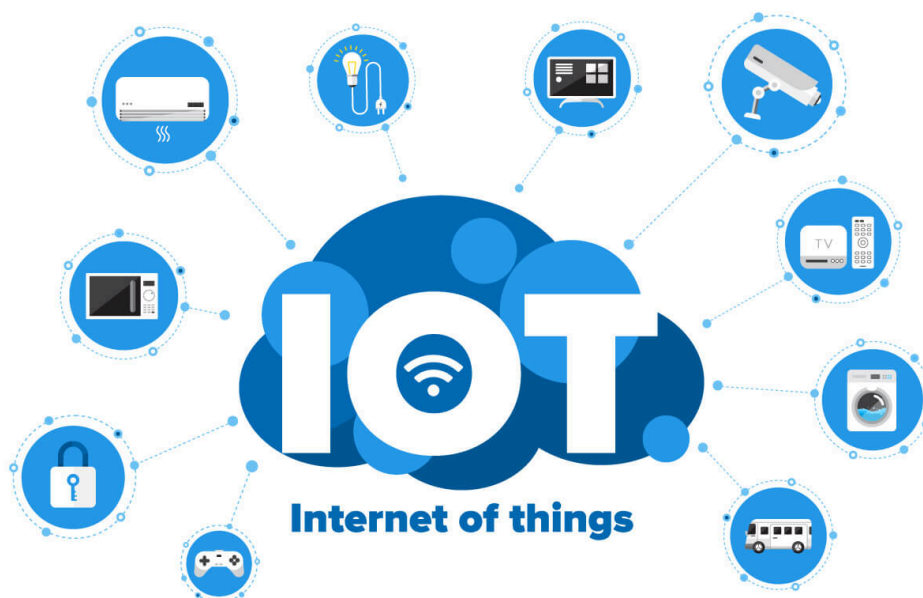
12.1 Hẹn giờ các tác vụ với Timers.....	108
12.2 Device Groups: Đồng bộ Nhóm Thiết bị.....	110
12.3 Energy Monitoring: Giám sát năng lượng & Hiệu chỉnh.....	111
12.4 Over-the-Air (OTA): Cập nhật Firmware qua mạng.....	112
12.5 Displays: Cấu hình màn hình hiển thị cơ bản.....	113
12.6 Deep Sleep Modes: Chế độ Ngủ cho thiết bị dùng Pin.....	114
Module 13: Tùy chỉnh Firmware Tasmota.....	115
Mục tiêu.....	115
13.1 Tại sao nên tự biên dịch Firmware?.....	115
13.2 Thiết lập Môi trường phát triển PlatformIO (VS Code).....	116
13.3 Tùy chỉnh Build Flags và user_config_override.h.....	117
Tùy chỉnh bằng file user_config_override.h:.....	118
13.4 Biên dịch và nạp Firmware tùy chỉnh của bạn.....	120
13.5 Giới thiệu về Berry Scripting cho các tùy chỉnh Nâng cao.....	121
Module 14: Xây dựng Smart Home thực tế.....	124
Mục tiêu.....	124
14.1 Dự án 1: Ổ cắm Thông minh Wi-Fi.....	124
14.2 Dự án 2: Thiết bị giám sát môi trường.....	125
14.3 Dự án 3: Công tắc tường thông minh.....	125
14.4 Dự án 4: Điều khiển dây LED màu RGB.....	126
14.5 Dự án 5: Bộ điều khiển hồng ngoại cho TV và máy lạnh.....	126
14.6 Dự án 6: Bộ điều khiển trung tâm.....	127
Module 15: Bảo mật trong Tasmota.....	128
Mục tiêu.....	128
15.1 Bảo mật Mạng Wi-Fi của bạn (WPA2/WPA3).....	128
15.2 Đặt mật khẩu giao diện Web quản trị của Tasmota.....	129
15.3 Bảo mật cho giao thức MQTT.....	130
15.4 Network Segmentation (tùy chọn).....	131
Module 16: Khắc phục các sự cố.....	133
Mục tiêu.....	133
16.1 Các vấn đề phổ biến và cách chẩn đoán.....	133
16.2 Sử dụng hiệu quả công cụ Console và Syslog của Tasmota.....	135
16.3 Tận dụng tài liệu chính thức của Tasmota.....	137
16.4 Hướng đi tiếp theo: Mở rộng kỹ năng IoT của bạn.....	137

Module 1: Giới thiệu về Internet of Things (IoT)

Mục tiêu

Chào mừng các bạn đến với Module đầu tiên của bộ tài liệu hướng dẫn xây dựng dự án IoT zero code với Tasmota.! Trong module đầu tiên này, chúng ta sẽ cùng nhau tìm hiểu những khái niệm cơ bản nhất về Internet of Things (IoT) - một công nghệ đang định hình lại thế giới xung quanh chúng ta.

1.1 IoT là gì? Các khái niệm chính và thuật ngữ



- **Internet of Things (IoT):**

Mạng lưới Vạn vật Kết nối Internet, là một mạng lưới gồm các đồ vật vật lý (thiết bị, máy móc, phương tiện, đồ gia dụng, và nhiều thứ khác) được nhúng các cảm biến, phần mềm và công nghệ kết nối mạng. Điều này cho phép chúng thu thập, trao đổi dữ liệu với

nhau và với các hệ thống khác qua Internet mà không cần sự tương tác trực tiếp của con người.

- **"Things" (Vạn vật/Đồ vật):**

Đây là thành phần cốt lõi của IoT. "Thing" có thể là bất cứ thứ gì bạn có thể tưởng tượng ra được gắn khả năng kết nối và "thông minh" hơn:

- Thiết bị đeo tay (đồng hồ thông minh, vòng theo dõi sức khỏe)
- Thiết bị gia dụng (tủ lạnh, máy giặt, bóng đèn, ổ cắm)
- Phương tiện (ô tô kết nối)
- Thiết bị công nghiệp (máy móc trong nhà máy, cảm biến môi trường)
- Và nhiều hơn nữa...

- **Kết nối (Connectivity):**

Yếu tố then chốt giúp các "Things" giao tiếp. Các công nghệ kết nối phổ biến bao gồm: Wi-Fi, Bluetooth, Zigbee, Z-Wave, LoRaWAN, NB-IoT, mạng di động (4G/5G)... Việc lựa chọn công nghệ phụ thuộc vào yêu cầu về khoảng cách, băng thông, và mức tiêu thụ năng lượng.

- **Dữ liệu (Data):**

Các thiết bị IoT liên tục thu thập dữ liệu từ môi trường xung quanh thông qua các cảm biến (ví dụ: nhiệt độ, độ ẩm, ánh sáng, chuyển động, vị trí...). Dữ liệu này là nguồn thông tin quý giá.

- **Xử lý & Phân tích (Processing & Analytics):**

Dữ liệu thô thu thập được thường được gửi đến một nơi để xử lý – có thể là một máy chủ cục bộ trong mạng nhà bạn, hoặc phổ biến hơn là lên nền tảng đám mây (Cloud). Tại đây, dữ liệu được lưu trữ, phân tích để rút ra thông tin hữu ích, đưa ra quyết định, kích hoạt hành động hoặc hiển thị cho người dùng.

- **Một số thuật ngữ quan trọng:**

- **Smart Devices (Thiết bị thông minh):** Các thiết bị có khả năng kết nối mạng và thực hiện một số tác vụ tự động hoặc từ xa.

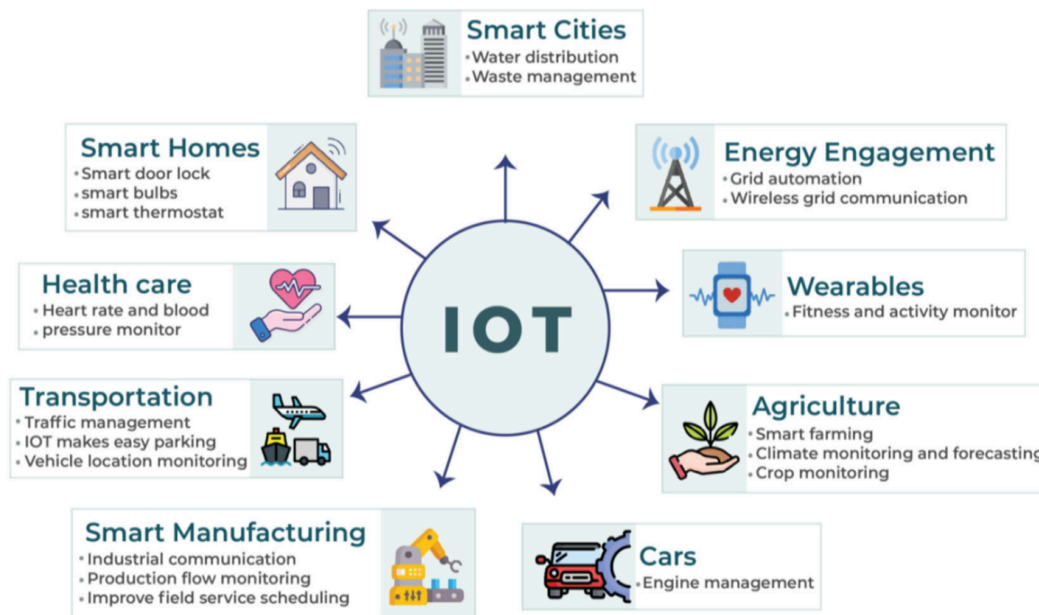
- **Embedded Systems (Hệ thống nhúng):** Hệ thống máy tính chuyên dụng được thiết kế để thực hiện một chức năng cụ thể trong một hệ thống lớn hơn (ví dụ: bộ não bên trong ổ cứng thông minh).
- **Sensors (Cảm biến):** Thiết bị phát hiện và phản hồi với các tín hiệu đầu vào từ môi trường vật lý (ánh sáng, nhiệt, chuyển động, độ ẩm, áp suất...).
- **Actuators (Bộ truyền động/Thiết bị chấp hành):** Thiết bị thực hiện hành động vật lý dựa trên tín hiệu điều khiển (ví dụ: bật/tắt relay (relay), điều khiển động cơ, thay đổi độ sáng đèn).
- **Cloud Computing (Điện toán đám mây):** Việc sử dụng các tài nguyên máy tính (máy chủ, lưu trữ, cơ sở dữ liệu, mạng, phần mềm...) qua Internet.
- **Big Data (Dữ liệu lớn):** Tập hợp dữ liệu cực lớn và phức tạp mà các công cụ xử lý dữ liệu truyền thống không thể xử lý được. IoT là một nguồn tạo ra Big Data khổng lồ.

1.2 Tại sao IoT quan trọng?

IoT không chỉ là một xu hướng công nghệ; nó đang thực sự thay đổi cách chúng ta sống, làm việc và tương tác. Tầm quan trọng của nó thể hiện qua:

- **Sự tiện nghi và tự động hóa:** Giúp cuộc sống dễ dàng hơn bằng cách tự động hóa các công việc lặp đi lặp lại và cho phép điều khiển mọi thứ từ xa.
- **Hiệu quả và tiết kiệm:** Tối ưu hóa việc sử dụng tài nguyên (năng lượng, nước), giảm lãng phí, và tiết kiệm chi phí vận hành.
- **An toàn và an ninh:** Cải thiện khả năng giám sát và cảnh báo sớm các vấn đề tiềm ẩn.
- **Ra quyết định tốt hơn:** Dữ liệu thu thập được cung cấp cái nhìn sâu sắc để đưa ra các quyết định dựa trên bằng chứng.
- **Mô hình kinh doanh mới:** Tạo ra các dịch vụ và sản phẩm sáng tạo.

Các lĩnh vực ứng dụng thực tế của công nghệ IoT có thể kể đến như:



- **Nhà thông minh (Smart Home):**

- Ví dụ: Điều khiển đèn, quạt, máy lạnh qua điện thoại; khóa cửa tự động; nhận cảnh báo khi có chuyển động lạ; tự động tưới cây dựa vào độ ẩm đất; bật bình nóng lạnh trước khi về nhà.
- Lợi ích: Tiện nghi, tiết kiệm năng lượng, tăng cường an ninh.

- **Công nghiệp (Industrial IoT - IIoT):**

- Ví dụ: Theo dõi hiệu suất máy móc trong nhà máy để phát hiện sớm nguy cơ hỏng hóc (bảo trì dự đoán); tự động hóa dây chuyền sản xuất; quản lý kho hàng thông minh; theo dõi vị trí tài sản.
- Lợi ích: Tăng năng suất, giảm thời gian chết máy, tối ưu hóa chuỗi cung ứng, đảm bảo an toàn lao động.

- **Thành phố thông minh (Smart City):**

- Ví dụ: Hệ thống đèn đường tự điều chỉnh độ sáng; quản lý giao thông thông minh để giảm ùn tắc; thùng rác thông minh báo đầy; giám sát chất lượng không khí và nước.
- Lợi ích: Nâng cao chất lượng cuộc sống đô thị, tiết kiệm năng lượng, bảo vệ môi trường.

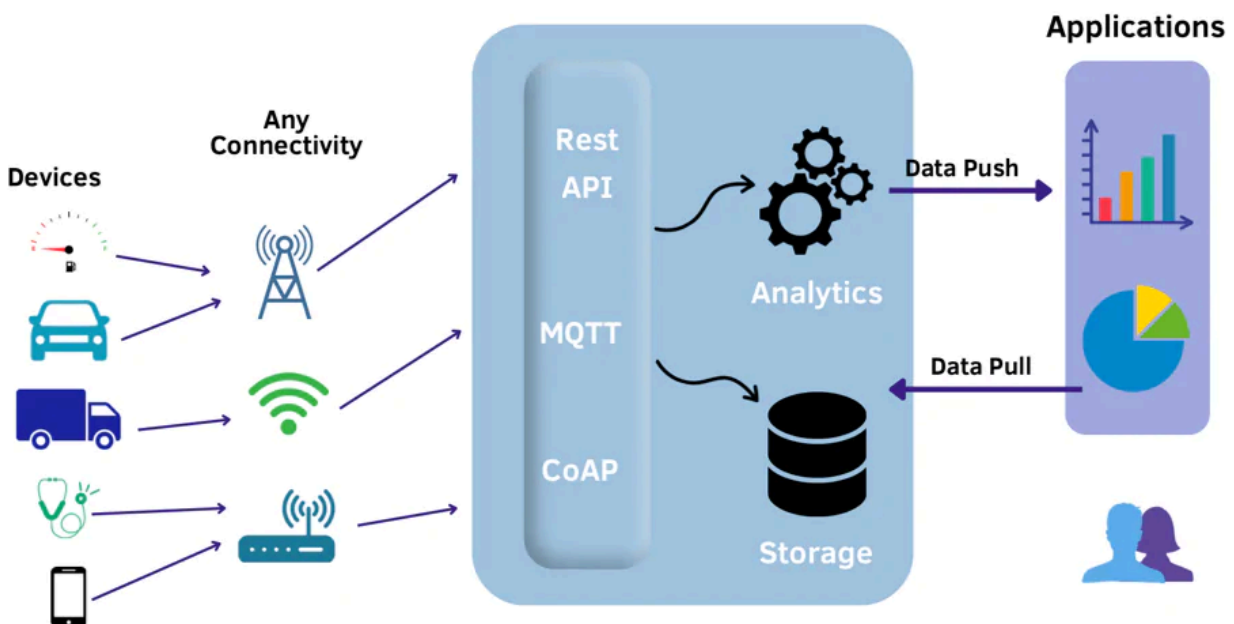
- **Nông nghiệp thông minh (Smart Agriculture):**

- Ví dụ: Cảm biến độ ẩm đất và dinh dưỡng; hệ thống tưới tiêu tự động; theo dõi sức khỏe vật nuôi; máy bay không người lái phun thuốc.

- *Lợi ích*: Tăng năng suất cây trồng, giảm sử dụng nước và phân bón, nâng cao chất lượng nông sản.
- **Y tế thông minh (Smart Healthcare)**:
 - *Ví dụ*: Thiết bị đeo theo dõi nhịp tim, giấc ngủ; giường bệnh thông minh báo cáo tình trạng bệnh nhân; hệ thống nhắc uống thuốc tự động; khám bệnh từ xa.
 - *Lợi ích*: Chăm sóc sức khỏe chủ động, giám sát bệnh nhân hiệu quả, giảm tải cho bệnh viện.

1.3 Các thành phần chính của một hệ thống IoT

Một hệ thống IoT điển hình thường bao gồm các thành phần cơ bản sau:



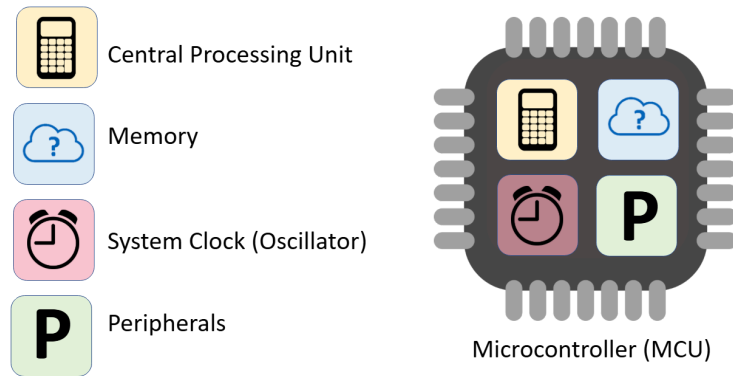
1. **Thiết bị/Cảm biến (Devices/Sensors)**: Đây là "giác quan" và "tay chân" của hệ thống. Chúng thu thập dữ liệu (cảm biến nhiệt độ, độ ẩm, ánh sáng, PIR...) hoặc thực hiện hành động (relay bật/tắt đèn, động cơ kéo rèm...). Trong tài liệu này, chúng ta sẽ tập trung vào các thiết bị dựa trên ESP8266/ESP32.
2. **Kết nối (Connectivity)**: Phương tiện để các thiết bị "nói chuyện" với nhau và với thế giới bên ngoài. Đối với các thiết bị Tasmota, kết nối chính thường là **Wi-Fi**. Các giao

thức khác như Bluetooth, Zigbee, Z-Wave cũng phổ biến, nhưng thường cần một Gateway.

3. **Cổng kết nối/Gateway (Gateway):** (Không phải lúc nào cũng cần thiết) Đóng vai trò như một trạm trung chuyển. Nó thu thập dữ liệu từ các thiết bị dùng giao thức tầm ngắn (như Zigbee, Bluetooth) và chuyển tiếp lên mạng IP (thường là Internet) thông qua Wi-Fi hoặc Ethernet. Với Tasmota chạy trên ESP8266/ESP32 có Wi-Fi, thiết bị thường có thể kết nối trực tiếp đến mạng Wi-Fi nhà bạn mà không cần Gateway riêng.
4. **Nền tảng đám mây/Xử lý dữ liệu (Cloud Platform/Data Processing):** "Bộ não" trung tâm của hệ thống. Đây là nơi dữ liệu được gửi đến để lưu trữ, phân tích, xử lý. Các nền tảng đám mây cung cấp công cụ quản lý thiết bị, xây dựng quy tắc tự động hóa, và hiển thị dữ liệu. Ví dụ: AWS IoT, Google Cloud IoT, Microsoft Azure IoT, Blynk Cloud, Thingspeak. **Quan trọng:** Hệ thống cũng có thể được xử lý hoàn toàn cục bộ (ví dụ: sử dụng Home Assistant chạy trên máy tính tại nhà) mà không cần đám mây công cộng.
5. **Ứng dụng Người dùng (User Application/Interface):** Cách bạn tương tác với hệ thống IoT. Đây có thể là một ứng dụng trên điện thoại, một trang web, hoặc thậm chí là giao diện điều khiển bằng giọng nói. Nó cho phép bạn xem trạng thái, điều khiển thiết bị, và cấu hình hệ thống. Giao diện WebUI của Tasmota là một ví dụ về giao diện người dùng cục bộ.

1.4 Giới thiệu về Vi điều khiển cho IoT

Trái tim của hầu hết các thiết bị IoT nhỏ gọn chính là Vi điều khiển (Microcontroller - MCU).



- **Vi điều khiển là gì?** Hãy tưởng tượng nó như một máy tính siêu nhỏ, tất cả được tích hợp trên một con chip duy nhất. Nó có bộ xử lý (CPU), bộ nhớ tạm (RAM), bộ nhớ lưu trữ chương trình (Flash), và các chân giao tiếp (GPIO) để kết nối với cảm biến, relay, và các linh kiện khác.
- **Tại sao dùng MCU cho IoT?** Chúng nhỏ gọn, tiêu thụ ít năng lượng (quan trọng cho thiết bị dùng pin), giá thành rẻ, và đủ mạnh để thực hiện các tác vụ cần thiết cho một thiết bị IoT.

Trong thế giới IoT DIY (Do It Yourself) và Maker, có hai dòng chip của Espressif Systems cực kỳ phổ biến và cũng là nền tảng được hỗ trợ chính bởi Tasmota:

- **ESP8266:**
 - Một MCU 32-bit giá rẻ đột phá với **Wi-Fi tích hợp sẵn**, sử dụng lõi xử lý Tensilica L106.
 - Ra đời trước và cực kỳ phổ biến trong cộng đồng do giá thành rẻ và dễ tiếp cận.
 - Số lượng chân GPIO và các ngoại vi tích hợp khá hạn chế.
 - Rất phù hợp cho các dự án IoT đơn giản chỉ cần kết nối Wi-Fi.
 - Các bo mạch phát triển phổ biến: **NodeMCU**, **Wemos D1 Mini**.
- **ESP32:**
 - Phiên bản kế nhiệm mạnh mẽ hơn của ESP8266, tích hợp cả **Wi-Fi và Bluetooth**.
 - Thường có **lõi kép** (Dual-core Tensilica LX6 hoặc LX7), cho khả năng xử lý cao hơn.

- Có nhiều chân GPIO hơn và nhiều ngoại vi tích hợp hơn (cảm biến chạm, bộ chuyển đổi Analog-Digital (ADC), Digital-Analog (DAC) chất lượng cao hơn...).
- Phù hợp cho các ứng dụng phức tạp hơn, cần hiệu năng cao, đa nhiệm, hoặc yêu cầu kết nối Bluetooth.
- Có nhiều dòng khác nhau như ESP32, ESP32-C3, ESP32-S3, ESP32-C6...

Tại sao chúng quan trọng với Tasmota? Tasmota là firmware được viết để chạy trực tiếp trên các chip ESP8266 và ESP32. Nó khai thác tối đa khả năng kết nối mạng (đặc biệt là Wi-Fi) và các chân GPIO của chúng để biến các thiết bị điện tử thông thường thành các thiết bị IoT thông minh, có thể tùy biến cao.

1.5 Local Control vs Cloud Dependency

Đây là một khái niệm quan trọng khi bạn làm việc với các thiết bị IoT, đặc biệt là với Tasmota.

- **Phụ thuộc vào đám mây (Cloud Dependency):**

- *Cách hoạt động:* Phần lớn các thiết bị nhà thông minh bạn mua ngoài cửa hàng (ví dụ: bóng đèn, ổ cắm của các nhà sản xuất Trung Quốc và cả của Việt Nam) yêu cầu phải kết nối liên tục đến máy chủ của nhà sản xuất qua Internet để hoạt động đầy đủ chức năng. Khi bạn bấm nút trên điện thoại, lệnh đi qua Internet -> đến máy chủ hãng -> rồi mới quay về thiết bị nhà bạn.
- *Ưu điểm:* Thường dễ cài đặt ban đầu, dễ truy cập điều khiển từ xa qua ứng dụng của hãng.
- *Nhược điểm:*
 - **Mất Internet = Mất kiểm soát:** Nếu nhà bạn mất mạng, hoặc máy chủ của hãng gặp sự cố, thiết bị có thể không hoạt động hoặc bạn không thể điều khiển được nữa.
 - **Quyền riêng tư:** Dữ liệu về thói quen sử dụng của bạn được gửi và lưu trữ trên máy chủ của bên thứ ba.
 - **Độ trễ (Latency):** Việc lệnh phải đi một vòng qua Internet có thể gây ra độ trễ khi điều khiển.

- **Vòng đời sản phẩm không chắc chắn:** Nếu nhà sản xuất ngừng kinh doanh hoặc ngừng hỗ trợ dịch vụ đám mây, thiết bị của bạn có nguy cơ trở thành "cục gạch".

- **Điều khiển cục bộ (Local Control):**

- *Cách hoạt động:* Thiết bị có khả năng hoạt động và được điều khiển hoàn toàn trong mạng nội bộ (mạng LAN/Wi-Fi nhà bạn) mà **không cần** kết nối Internet. Lệnh điều khiển đi trực tiếp từ điện thoại/máy tính của bạn đến thiết bị.
- *Ưu điểm:*
 - **Độ tin cậy cao:** Hoạt động ổn định ngay cả khi không có Internet. Bạn luôn kiểm soát được thiết bị của mình.
 - **Quyền riêng tư được đảm bảo:** Dữ liệu thường chỉ lưu hành trong mạng nội bộ của bạn (trừ khi bạn chủ động cấu hình để gửi đi).
 - **Phản hồi nhanh:** Độ trễ điều khiển gần như tức thì vì lệnh không phải đi vòng qua Internet.
 - **Tự chủ:** Bạn không phụ thuộc vào máy chủ hay dịch vụ của bất kỳ công ty nào.
- *Nhược điểm:*
 - Việc cài đặt ban đầu có thể đòi hỏi nhiều kiến thức kỹ thuật hơn một chút.
 - Để điều khiển từ xa (khi bạn không ở nhà) cần cấu hình thêm các giải pháp như mở port trên Wi-Fi Router, VPN, hoặc sử dụng các nền tảng trung gian hỗ trợ (như Home Assistant Cloud, Nabu Casa).

Tasmota và Local Control: Một trong những ưu điểm lớn nhất của Tasmota là nó được thiết kế để **ưu tiên điều khiển cục bộ**. Khi bạn thay thế firmware gốc bằng Tasmota, bạn giải phóng thiết bị khỏi sự trói buộc vào đám mây của nhà sản xuất, mang lại cho bạn sự tự do, tin cậy và quyền kiểm soát hoàn toàn. Đây chính là lý do tại sao Tasmota rất được yêu thích trong cộng đồng DIY/Maker và những người quan tâm đến quyền riêng tư, độ tin cậy trong hệ thống nhà thông minh.

Kết thúc Module 1, bạn đã có cái nhìn tổng quan về thế giới IoT, các thành phần cấu tạo nên nó, vai trò của vi điều khiển ESP8266/ESP32, và đặc biệt là sự khác biệt quan trọng giữa điều khiển cục bộ và phụ thuộc vào đám mây. Kiến thức nền tảng này rất quan trọng để chúng ta tiếp tục khám phá Tasmota ở các module tiếp theo.

Module 2: Giới thiệu về Tasmota

Mục tiêu

Ở Module 1, chúng ta đã tìm hiểu về thế giới rộng lớn của IoT. Bây giờ, hãy cùng tập trung vào nhân vật chính của bộ tài liệu này: **Tasmota**. Module 2 này sẽ giới thiệu Tasmota là gì, tại sao nó lại là lựa chọn hấp dẫn và những điều cơ bản bạn cần biết trước khi bắt đầu.

2.1 Tasmota là gì?

- **Tasmota là gì?** Hiểu một cách đơn giản, Tasmota là một **firmware thay thế**. Firmware là phần mềm cốt lõi chạy trực tiếp trên phần cứng của một thiết bị điện tử. Tasmota được tạo ra để **thay thế hoàn toàn** firmware gốc mà nhà sản xuất cài đặt sẵn trên các thiết bị nhà thông minh, đặc biệt là các thiết bị sử dụng chip Wi-Fi ESP8266 và ESP32.



- **Lịch sử ra đời:**
 - Tasmota được tạo ra bởi **Theo Arends**.
 - Ban đầu, mục đích chính là để "giải phóng" các thiết bị nhà thông minh giá rẻ của hãng ITEAD (nổi tiếng với thương hiệu Sonoff) khỏi sự phụ thuộc vào máy chủ đám mây eWeLink của họ. Cái tên **TASMOTA** là viết tắt của **Theo Arends Sonoff MQTT OTA**.
 - **Sonoff:** Thương hiệu thiết bị mục tiêu ban đầu.
 - **MQTT:** Giao thức nhắn tin mà Tasmota sử dụng để giao tiếp (sẽ tìm hiểu kỹ ở Module 8).
 - **OTA (Over-The-Air):** Khả năng cập nhật firmware qua mạng Wi-Fi.
 - Từ khởi đầu tập trung vào Sonoff, dự án Tasmota đã phát triển vượt bậc, mở rộng hỗ trợ cho hàng nghìn mẫu thiết bị khác nhau từ rất nhiều nhà sản xuất trên thế giới.
- **Triết lý Mã nguồn mở (Open Source):** Tasmota được phát triển theo triết lý mã nguồn mở, mang lại nhiều lợi ích:
 - **Miễn phí:** Bạn có thể tải về, sử dụng và sửa đổi Tasmota mà không tốn bất kỳ chi phí nào.
 - **Công khai mã nguồn:** Toàn bộ mã nguồn của Tasmota được công khai trên nền tảng GitHub. Điều này cho phép bất kỳ ai cũng có thể xem xét, kiểm tra cách nó hoạt động, học hỏi và thậm chí đóng góp cải tiến.
 - **Minh bạch:** Bạn hoàn toàn biết được phần mềm chạy trên thiết bị của mình đang làm gì, không có những hoạt động ẩn giấu nào.
 - **Sức mạnh cộng đồng:** Tasmota có được sự phát triển và hỗ trợ mạnh mẽ như ngày nay là nhờ vào nỗ lực đóng góp của một cộng đồng đông đảo người dùng và các nhà phát triển trên toàn cầu.

2.2 Tại sao chọn Tasmota?

Vậy tại sao nhiều người lại chọn cài Tasmota thay vì sử dụng firmware gốc? Dưới đây là những lý do chính:

1. Giải phóng khỏi dịch vụ đám mây với Local Control:

- Đây là lợi ích **quan trọng nhất**. Tasmota cho phép thiết bị hoạt động độc lập, **điều khiển hoàn toàn cục bộ** trong mạng nhà bạn.
- Không còn phụ thuộc vào máy chủ của nhà sản xuất.
- Thiết bị vẫn hoạt động ổn định ngay cả khi **mất kết nối Internet**.
- **Tăng cường quyền riêng tư**: Dữ liệu không bị gửi đi đâu nếu bạn không muốn.
- **Phản hồi nhanh hơn**: Giảm độ trễ khi điều khiển.

2. Khả năng tùy biến cực cao:

- Bạn có thể cấu hình gần như mọi thứ: gán chức năng cho các chân GPIO, điều chỉnh hoạt động của nút bấm, đèn LED, relay.
- Hỗ trợ một danh sách khổng lồ các loại cảm biến và thiết bị ngoại vi.
- Tạo ra các kịch bản tự động hóa phức tạp ngay trên thiết bị bằng **Rules** hoặc ngôn ngữ lập trình **Berry**.
- Giao diện web có thể được tùy chỉnh.
- Bạn có thể tự biên dịch (compile) phiên bản Tasmota của riêng mình, chỉ bao gồm những tính năng bạn cần để tối ưu hiệu năng và bộ nhớ.

3. Tích hợp dễ dàng với các nền tảng tự động hóa:

- Hỗ trợ chuẩn giao tiếp **MQTT**, giúp việc kết nối và trao đổi dữ liệu với các hệ thống nhà thông minh trung tâm như **Core IoT**, **Home Assistant**, **openHAB**, **Node-RED** trở nên cực kỳ đơn giản và linh hoạt.
- Cung cấp giao diện điều khiển qua **HTTP API** và **Serial**.

4. Cập nhật OTA (Over-The-Air):

- Việc nâng cấp lên phiên bản Tasmota mới rất dễ dàng, thực hiện trực tiếp qua mạng Wi-Fi mà không cần phải tháo thiết bị ra để kết nối cáp.

5. Cộng đồng hỗ trợ lớn:

- Khi gặp khó khăn, bạn có thể tìm thấy vô vàn tài liệu hướng dẫn chi tiết trên trang Tasmota Docs chính thức.
- Có rất nhiều diễn đàn, kênh Discord, nhóm trên mạng xã hội nơi bạn có thể đặt câu hỏi, chia sẻ kinh nghiệm và nhận được sự giúp đỡ từ những người dùng khác.
- Dự án liên tục được cập nhật, sửa lỗi và bổ sung tính năng mới nhờ sự đóng góp không ngừng của cộng đồng.

6. Hỗ trợ rất nhiều loại phần cứng:

- Tasmota có thể chạy trên rất nhiều thiết bị Wi-Fi giá rẻ dựa trên chip ESP có sẵn trên thị trường, giúp bạn tiết kiệm chi phí khi xây dựng nhà thông minh.

2.3 Các phần cứng được hỗ trợ

- **Nền tảng chính:** Như đã nói, Tasmota được thiết kế để chạy chủ yếu trên các chip **ESP8266** và **ESP32** do công ty Espressif Systems sản xuất. Nếu một thiết bị sử dụng một trong hai loại chip này, khả năng cao là nó có thể chạy được Tasmota.
- **Các loại thiết bị phổ biến có thể chạy Tasmota:**
 - **Ổ cắm thông minh (Smart Plugs):** Các dòng Sonoff Sxx, BlitzWolf SHP, Teckin, Gosund, và nhiều thương hiệu khác sử dụng chip ESP.
 - **Công tắc âm tường (Wall Switches):** Sonoff Basic/Mini/TX/T-series, Shelly 1/1PM/2.5, nhiều mẫu công tắc của Tuya/Smart Life (đời cũ dùng ESP).
 - **Bóng đèn thông minh (Smart Bulbs):** Các loại bóng đèn Wi-Fi có thể điều chỉnh độ sáng, màu sắc (RGB), nhiệt độ màu (CCT) dùng chip ESP.
 - **Module Relay (Relay Modules):** Sonoff Basic/Mini/4CH, Shelly, các module ESP Relay bán rời.
 - **Thiết bị tích hợp cảm biến:** Sonoff TH (nhiệt độ/độ ẩm), các thiết bị khác có cảm biến tích hợp sẵn dùng ESP.
 - **Bo mạch phát triển (Development Boards):** Đây là lựa chọn tuyệt vời cho các dự án DIY. Các bo mạch như **NodeMCU (ESP8266)**, **Wemos D1 Mini**

(ESP8266), và các loại **bo mạch ESP32** (DevKitC, WROOM, WROVER, Yolo UNO, Yolo Node...) là nền tảng lý tưởng để bạn tự tạo ra thiết bị Tasmota độc đáo của riêng mình.

- **Làm sao để biết thiết bị có tương thích Tasmota?**
 - Cách tốt nhất là truy cập **Tasmota Devices Repository** (templates.blakadder.com). Đây là một cơ sở dữ liệu khổng lồ do cộng đồng đóng góp, liệt kê các thiết bị đã được xác nhận chạy Tasmota cùng với hướng dẫn cấu hình (template).
 - Tìm kiếm trên Google tên thiết bị của bạn kèm theo từ khóa "Tasmota", "ESP8266", "ESP32".
 - **Lưu ý quan trọng:** Các nhà sản xuất (đặc biệt là Tuya) ngày càng có xu hướng chuyển sang sử dụng các chip Wi-Fi khác không phải của Espressif cho các thiết bị mới. Việc cài Tasmota lên các chip này thường rất khó hoặc không thể. Do đó, nếu bạn có ý định mua thiết bị mới để chạy Tasmota, hãy **kiểm tra kỹ thông tin tương thích** trước khi mua!

2.4 So sánh Tasmota với Firmware gốc của nhà sản xuất

Việc lựa chọn firmware phù hợp phụ thuộc vào nhu cầu và kiến thức của bạn. Dưới đây là bảng so sánh giữa việc sử dụng firmware Tasmota với Firmware gốc (Stock Firmware) của nhà sản xuất thiết bị.

Tiêu chí	Tasmota	Firmware gốc (thường gặp) của NSX
Điều khiển	Cục bộ (Local), không cần Internet	Thường phụ thuộc Cloud, cần Internet

Tùy biến	Rất cao , cấu hình mọi thứ	Hạn chế theo nhà sản xuất
Tích hợp	Linh hoạt (MQTT, HTTP, API)	Qua App/Cloud độc quyền, tích hợp hạn chế
Quyền riêng tư	Cao hơn , dữ liệu trong mạng nhà bạn	Dữ liệu có thể gửi về máy chủ của nhà sản xuất
Cập nhật	Người dùng chủ động thông qua OTA	Nhà sản xuất đẩy tự động (có thể bị ép)
Ổn định	Rất ổn định (khi cấu hình đúng)	Phụ thuộc vào máy chủ Cloud
Cài đặt	Cần hiểu biết kỹ thuật (nạp lại firmware)	Dễ dàng qua App của nhà sản xuất

Kết thúc Module 2, bạn đã có một bức tranh rõ ràng hơn về Tasmota, những lợi ích nó mang lại, các thiết bị tương thích, so sánh với các lựa chọn khác và những lưu ý an toàn không thể bỏ qua. Với kiến thức này, chúng ta đã sẵn sàng để chuẩn bị những công cụ và linh kiện cần thiết trong Module 3.

Module 3: Cơ bản về Phần cứng & Điện tử

Mục tiêu

Sau khi tìm hiểu về IoT và Tasmota, Module này sẽ trang bị cho bạn những kiến thức nền tảng về phần cứng và điện tử. Đây là những kỹ năng và hiểu biết cần thiết để bạn có thể kết nối, cấu hình và sửa lỗi các thiết bị Tasmota một cách hiệu quả.

3.1 Các bo mạch phát triển, sơ đồ chân (Pinouts) và tính năng

Bo mạch phát triển (Development Board) là gì?

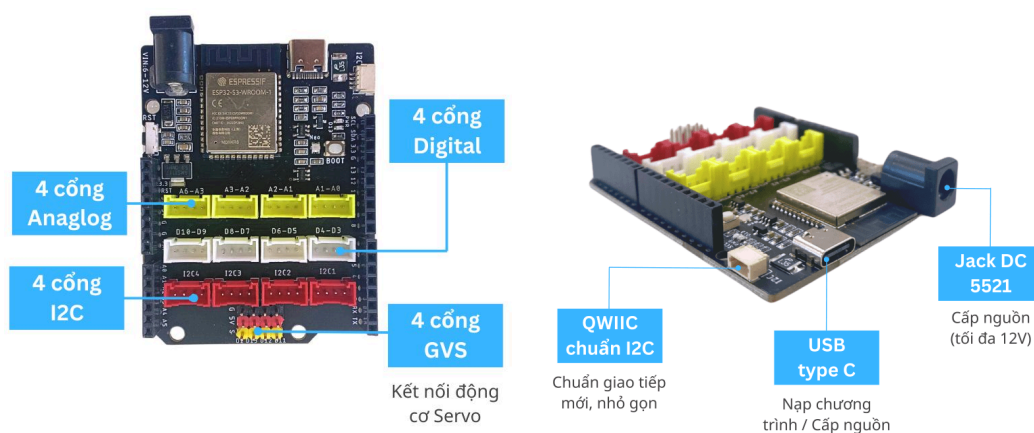
Đây là những bo mạch được thiết kế đặc biệt để giúp việc thử nghiệm và phát triển ứng dụng với chip ESP8266 hoặc ESP32 trở nên dễ dàng hơn. Chúng thường tích hợp sẵn:

- Chip ESP8266 hoặc ESP32.
- Mạch cấp nguồn và ổn áp (thường là 3.3V cho chip ESP).
- Cổng USB để cấp nguồn và nạp chương trình (firmware).
- Chip chuyển đổi USB-to-Serial (để máy tính giao tiếp được với chip ESP qua cổng USB).
- Các chân GPIO (General Purpose Input/Output) của chip ESP được đưa ra ngoài dưới dạng các chân cắm (header pins) để dễ dàng kết nối với các linh kiện khác.

Có rất nhiều loại bo mạch ESP8266 và ESP32 mà bạn có thể dễ dàng tìm thấy trên các cửa hàng điện tử hoặc các sàn thương mại điện tử. Đặc điểm chung của các bo mạch này ra chỉ ra các chân IO để sử dụng với các dây nối jumper, breadboard hoặc hàn trực tiếp. Điều này gây ra rất nhiều khó khăn cho những người mới tiếp cận.

Trong tài liệu này, chúng tôi sẽ sử dụng mạch Yolo UNO và Yolo Node để hướng dẫn vì sự dễ dàng của nó với người mới.

Giới thiệu bo mạch Yolo UNO



Bo mạch Yolo UNO

Yolo UNO là một bo mạch do OhStem Education thiết kế, phục vụ cho giáo dục và phù hợp với những người mới bắt đầu làm quen với thế giới phần cứng IoT. Bo mạch được thiết kế theo form của Arduino Uno, nhưng sử dụng dòng chip ESP32-S3 mới nhất hiện nay của Espressif, với bộ nhớ Flash 16MB và bộ nhớ PSRAM lên đến 8MB.

- ESP32-S3 là một dòng System-on-Chip (SoC) có bộ xử lý lõi kép 32-bit, tốc độ xung nhịp lên đến 240MHz, low-power, hỗ trợ WiFi chuẩn 2.4 GHz và BLE+Mesh, được tích hợp nhiều tính năng như USB native, công nghệ đo khoảng cách Time of Flight và tăng tốc xử lý AI.
- Mạnh mẽ hơn ESP8266: Lõi kép, tích hợp **Wi-Fi và Bluetooth** (Classic & BLE) và có **nhiều chân GPIO** hơn đáng kể so với ESP8266.
- Tích hợp sẵn 12 cổng kết nối chuẩn Grove, giúp bạn dễ dàng kết nối với các module và cảm biến trong hệ sinh thái thiết bị AI/IoT của OhStem.
- Tích hợp nút nhấn, đèn LED báo nguồn, LED chân D13 và LED RGB NeoPixel.

- Kiến trúc RISC-V, mã nguồn mở, mang lại hiệu suất cao và khả năng tiết kiệm năng lượng vượt trội.
- 6 cổng Grove giúp kết nối các cảm biến và module ngoại vi dễ dàng, đặc biệt là cho người mới bắt đầu, bao gồm:
 - 3 x cổng Analog/Digital
 - 2 x cổng I2C
 - 1 x cổng UART
- Tích hợp nút nhấn, đèn LED, cảm biến nhiệt độ độ ẩm DHT20 và cảm biến ánh sáng quang trở.

Các bạn có thể sử dụng 1 trong 2 bo mạch trên để có thể làm các hướng dẫn trong tài liệu này, nhưng sẽ cần chú ý đến việc cấu hình các chân GPIO cho đúng với từng mạch.

Lưu ý quan trọng khi xem Sơ đồ chân:

- Luôn **tìm đúng sơ đồ chân (pinout diagram)** cho phiên bản bo mạch **cụ thể** mà bạn đang sử dụng. Có thể có những khác biệt nhỏ về cách đánh dấu hoặc vị trí chân giữa các nhà sản xuất hoặc các phiên bản khác nhau của cùng một loại bo mạch.
- Hiểu rõ sự khác biệt giữa **tên chân trên bo mạch** (D1, D2...) và **số GPIO thực tế** của chip ESP là rất quan trọng khi cấu hình trong Tasmota.

3.2 Các khái niệm Điện tử cơ bản

Để làm việc hiệu quả với các bo mạch và linh kiện, bạn cần nắm vững một vài khái niệm điện tử cơ bản:

- **Điện áp (Voltage - Ký hiệu V):**
 - Giống như "áp lực nước" trong ống. Nó là sự chênh lệch điện thế giữa hai điểm, tạo ra "lực đẩy" làm các hạt điện tích (electron) di chuyển.
 - Đơn vị đo: **Volt (V)**.
 - Trong các dự án với ESP, chúng ta thường làm việc với điện áp một chiều (DC) thấp: **3.3V** (điện áp hoạt động của chip ESP) và **5V** (điện áp từ cổng USB hoặc nguồn ngoài cấp cho bo mạch). Điện áp xoay chiều (AC) 220V/110V của điện lưới là **rất nguy hiểm** và chỉ nên làm việc khi đã hiểu rõ và tuân thủ an toàn tuyệt

đối.

- **Dòng điện (Current - Ký hiệu I):**

- Giống như "lưu lượng nước" chảy trong ống. Nó là tốc độ dòng chảy của các hạt điện tích.
- Đơn vị đo: **Ampere (A)** hoặc **miliAmpe (mA)** ($1A = 1000mA$).
- Dòng điện chỉ chảy khi có mạch kín và có điện áp. Mỗi thiết bị sẽ tiêu thụ (hay "rút") một lượng dòng điện nhất định để hoạt động. Nguồn cấp phải có khả năng cung cấp đủ dòng điện yêu cầu. Chip ESP có thể tiêu thụ từ vài chục mA đến vài trăm mA tùy vào hoạt động (đặc biệt là khi phát Wi-Fi).

- **Điện trở (Resistance - Ký hiệu R):**

- Giống như "vật cản" làm hẹp đường ống, cản trở dòng nước chảy. Nó là đại lượng đo lường khả năng cản trở dòng điện của một vật liệu hoặc linh kiện.
- Đơn vị đo: **Ohm (Ω)**.
- Điện trở (linh kiện điện trở) được sử dụng rất nhiều trong mạch điện tử để: hạn chế dòng điện (ví dụ: bảo vệ đèn LED), chia điện áp, tạo điện trở kéo lên/kéo xuống (pull-up/pull-down).
- **Định luật Ohm:** Mối quan hệ nền tảng giữa V, I, R: **$V = I \times R$** . Hiểu định luật này giúp bạn tính toán và phân tích mạch đơn giản.

- **GPIO (General Purpose Input/Output - Chân Vào/Ra Đa dụng):**

- Đây là các chân vật lý trên chip vi điều khiển (ESP8266/ESP32) mà chúng ta có thể lập trình để điều khiển hoặc đọc tín hiệu từ thế giới bên ngoài. Chúng rất linh hoạt.
- **Chế độ Output (Ngõ ra):** Bạn lập trình để chân này xuất ra một mức điện áp logic:
 - **HIGH (Mức cao):** Thường là mức điện áp hoạt động của chip (3.3V đối với ESP). Dùng để cấp nguồn cho LED sáng, kích hoạt một module relay, gửi tín hiệu '1'.
 - **LOW (Mức thấp):** Thường là 0V (nối với GND). Dùng để tắt LED, ngừng kích hoạt relay, gửi tín hiệu '0'.
- **Chế độ Input (Ngõ vào):** Bạn lập trình để chân này đọc trạng thái điện áp từ bên ngoài:

- Đọc xem chân đang được nối với mức điện áp HIGH (3.3V) hay LOW (0V).
 - Dùng để nhận biết trạng thái của nút nhấn, công tắc, hoặc đọc tín hiệu số từ cảm biến.
 - **Input Pull-up / Pull-down:** Đây là các điện trở tích hợp sẵn trong chip ESP. Khi một chân Input không được nối với đầu cả (trạng thái "thả nổi" - floating), nó có thể bị nhiễu và đọc giá trị không ổn định. Điện trở Pull-up sẽ "kéo" điện áp của chân lên mức HIGH (3.3V) khi không có tín hiệu nào khác. Điện trở Pull-down sẽ "kéo" xuống mức LOW (0V). Trong Tasmota và nhiều ứng dụng ESP, **Pull-up** rất hay được dùng với nút nhấn (nút nhấn sẽ nối chân GPIO xuống GND khi được nhấn).
 - **Analog Input (Ngõ vào Analog - ADC):** Một số chân GPIO đặc biệt (đánh dấu A0 trên ESP8266, nhiều chân trên ESP32) có khả năng đọc các mức điện áp thay đổi liên tục (ví dụ từ 0V đến 1V hoặc 0V đến 3.3V), chứ không chỉ là HIGH/LOW. Tính năng này gọi là Bộ chuyển đổi Analog sang Số (Analog-to-Digital Converter - ADC). Dùng để đọc giá trị từ các cảm biến analog như cảm biến ánh sáng (quang trở LDR), biến trở...
- **GND (Ground - Mass / Đất):**
 - Là điểm tham chiếu chung có điện thế 0V trong một mạch điện. Tất cả các thành phần trong cùng một mạch cần phải được nối chung GND để đảm bảo hoạt động chính xác.

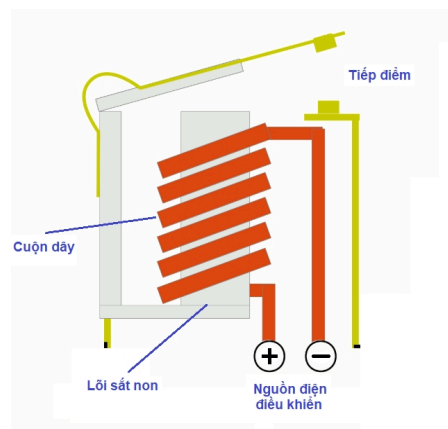
3.3 Tổng quan về các linh kiện IoT phổ biến

Khi làm việc với Tasmota và các bo mạch ESP, bạn sẽ thường xuyên gặp và sử dụng các linh kiện sau:

- **Relay (Rơ le):**



- Hoạt động như một **công tắc điện tử công suất lớn**. Nó cho phép một tín hiệu điện áp thấp (ví dụ 3.3V từ chân GPIO của ESP) điều khiển việc đóng/cắt một mạch điện có điện áp cao (như AC 220V) hoặc dòng điện lớn mà không cần tiếp xúc trực tiếp về điện.
- Rất cần thiết khi bạn muốn dùng ESP để điều khiển bật/tắt đèn điện lưới, quạt, máy bơm, ổ cắm...
- **Cấu tạo:** Bên trong có một cuộn dây (coil) và các tiếp điểm cơ khí (contacts). Khi cấp điện áp điều khiển vào cuộn dây, nó tạo ra từ trường hút hoặc đẩy các tiếp điểm, làm thay đổi trạng thái mạch điện áp cao (đóng hoặc mở).



- **Các chân chính:** Chân cấp nguồn cho cuộn dây (VCC, GND), chân tín hiệu điều khiển (IN), và các chân tiếp điểm công suất (COM - Chân chung, NC - Thường đóng, NO - Thường mở).

- **Nút nhấn (Button / Push Button):**



- Là loại công tắc **nhấn nhả (momentary)**. Nó chỉ đóng mạch (dẫn điện) khi bạn nhấn và giữ nút, và sẽ tự động ngắt mạch (hở ra) khi bạn thả tay.
- Thường được dùng làm **tín hiệu đầu vào** cho vi điều khiển để người dùng tương tác (ví dụ: nhấn nút để bật/tắt đèn, kích hoạt một chức năng).
- **Cách kết nối phổ biến với ESP (dùng INPUT_PULLUP)**: Nối một chân của nút nhấn với chân **GND**. Chân còn lại nối với một chân **GPIO** của ESP. Cấu hình chân GPIO đó trong Tasmota (hoặc code) là **INPUT_PULLUP**. Khi nút không được nhấn, điện trở kéo lên ảo sẽ giữ chân GPIO ở mức HIGH. Khi bạn nhấn nút, chân GPIO bị nối trực tiếp xuống GND, và vi điều khiển sẽ đọc được mức LOW.

- **Công tắc (Switch):**



- Khác với nút nhấn, công tắc là loại **giữ trạng thái (maintained)**. Nó có hai hoặc nhiều vị trí ổn định (ví dụ: ON và OFF). Khi bạn gạt công tắc sang một vị trí, nó sẽ giữ nguyên trạng thái đó cho đến khi bạn gạt sang vị trí khác.
- Ví dụ: công tắc bật/tắt đèn tường, công tắc gạt nhỏ trên các bo mạch.

- Cách kết nối và đọc trạng thái tương tự như nút nhấn, nhưng trạng thái HIGH/LOW sẽ được duy trì.

- **Đèn LED (Light Emitting Diode):**



- Linh kiện bán dẫn phát ra ánh sáng khi có dòng điện chạy qua đúng chiều.
 - Có **phân cực**: Chân dài hơn thường là cực dương (Anode, +), chân ngắn hơn là cực âm (Cathode, -). Phải nối đúng chiều thì LED mới sáng.
 - **Luôn cần một điện trở hạn dòng (current limiting resistor)** mắc nối tiếp với LED khi nối vào chân GPIO. Điện trở này bảo vệ cả LED và chân GPIO không bị dòng quá lớn chạy qua gây hỏng. Giá trị điện trở thường dùng là 220Ω đến 1kΩ cho nguồn 3.3V hoặc 5V (tùy thuộc vào loại LED).
 - Thường dùng làm đèn báo trạng thái (ví dụ: báo có nguồn, báo kết nối Wi-Fi, báo relay đang bật...).
- **Addressable RGB LED:** Như **WS2812B (Neopixel)**. Nhiều LED được mắc nối tiếp nhưng bạn có thể điều khiển màu sắc và độ sáng của từng LED riêng lẻ thông qua một chân dữ liệu duy nhất. Tasmota hỗ trợ rất tốt loại LED này.
- **Cảm biến (Sensors):** Là "giác quan" giúp thiết bị IoT của bạn nhận biết môi trường xung quanh.
 - **Cảm biến Nhiệt độ & Độ ẩm:** như DHT20, DHT11/DHT22, DS18B20.
 - **Cảm biến Chuyển động (PIR Motion Sensor):** Phát hiện sự thay đổi bức xạ hồng ngoại phát ra từ các vật thể ẩm chuyển động (như người, động vật).

- **Cảm biến Ánh sáng quang trở (LDR - Light Dependent Resistor):** Là một điện trở có giá trị thay đổi phụ thuộc vào cường độ ánh sáng chiếu vào nó (sáng mạnh thì điện trở giảm, tối thì điện trở tăng).
- **Các loại cảm biến khác:** Có vô vàn loại cảm biến khác mà bạn có thể kết nối với Tasmota như: cảm biến khoảng cách (siêu âm HC-SR04, hồng ngoại), cảm biến khí gas, cảm biến chất lượng không khí (PM2.5), cảm biến mưa, cảm biến dòng điện/công suất (PZEM-004T), cảm biến mực nước...

Kết thúc Module 3, bạn đã được trang bị những kiến thức và kỹ năng cơ bản về phần cứng và điện tử. Hiểu về bo mạch ESP, các khái niệm V-I-R-GPIO, biết nhận diện các linh kiện phổ biến sẽ giúp bạn tự tin hơn rất nhiều khi bắt tay vào thực hành và cấu hình thiết bị ở các module sau. Hãy chuẩn bị sẵn sàng các công cụ và bo mạch ESP của bạn cho Module 4.

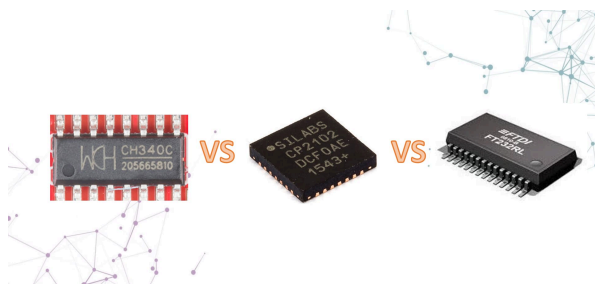
Module 4: Chuẩn bị môi trường làm việc

Mục tiêu

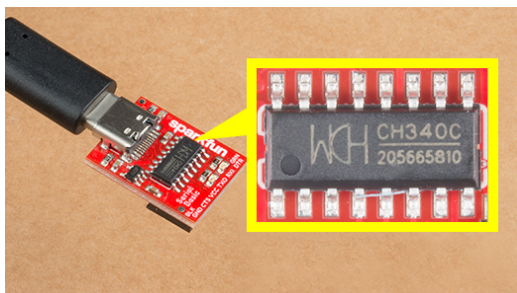
Trước khi chúng ta có thể nạp Tasmota vào thiết bị ESP, chúng ta cần chuẩn bị máy tính và hiểu rõ về các công cụ cũng như firmware sẽ sử dụng. Module này sẽ hướng dẫn bạn qua các bước chuẩn bị cần thiết này.

4.1 Cài đặt Trình điều khiển (Drivers)

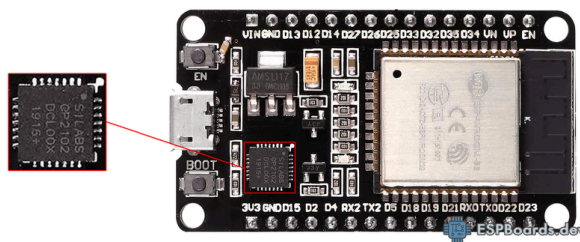
- **Tại sao cần Driver?** Các bo mạch phát triển như NodeMCU, Wemos D1 Mini, ESP32 DevKitC thường sử dụng một con chip nhỏ gọi là "USB-to-Serial Bridge" để cho phép máy tính của bạn giao tiếp với chip ESP chính thông qua cổng USB. Máy tính của bạn cần một phần mềm đặc biệt, gọi là **driver (trình điều khiển)**, để nhận biết và "nói chuyện" được với con chip cầu nối này. Nếu không có driver đúng, máy tính sẽ không thể tìm thấy bo mạch ESP khi bạn cắm nó vào.
- **Lưu ý:** Nếu bạn sử dụng mạch **Yolo UNO** hoặc **Yolo Node** thì nhiều khả năng hệ điều hành Windows **sẽ tự nhận mà không cần cài đặt thêm driver**.
- **Xác định loại chip USB-to-Serial trên bo mạch của bạn:** Cách dễ nhất là nhìn vào con chip hình chữ nhật nằm gần cổng USB trên bo mạch. Các loại phổ biến nhất là:



1. **CH340 / CH341:** Rất phổ biến trên các bo mạch NodeMCU và Wemos D1 Mini giá rẻ.



2. **CP2102 / CP210x:** Thường có trên các bo mạch NodeMCU, ESP32 DevKitC chất lượng cao hơn.



3. **FTDI FT232RL:** Ít gặp hơn trên các bo mạch ESP phổ thông vì giá thành cao.
- **Tải và Cài đặt Driver:** Bạn cần tải và cài đặt driver tương ứng với loại chip trên bo mạch của mình.
 1. **Đối với Windows:**
 - **CH340/CH341:** Bạn có thể tìm kiếm "CH340 driver download" hoặc tải từ trang của nhà sản xuất bo mạch (như Wemos). Một link tin cậy thường là từ nhà sản xuất chip WCH:
http://www.wch-ic.com/downloads/CH341SER_EXE.html
 - **CP2102/CP210x:** Tải từ trang của Silicon Labs (nhà sản xuất chip):
<https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers>¹ (Chọn đúng phiên bản cho Windows của bạn).
 - Sau khi tải về, chạy file cài đặt (.exe) và làm theo hướng dẫn. Thông thường bạn cần khởi động lại máy tính sau khi cài đặt.

2. Đối với macOS:

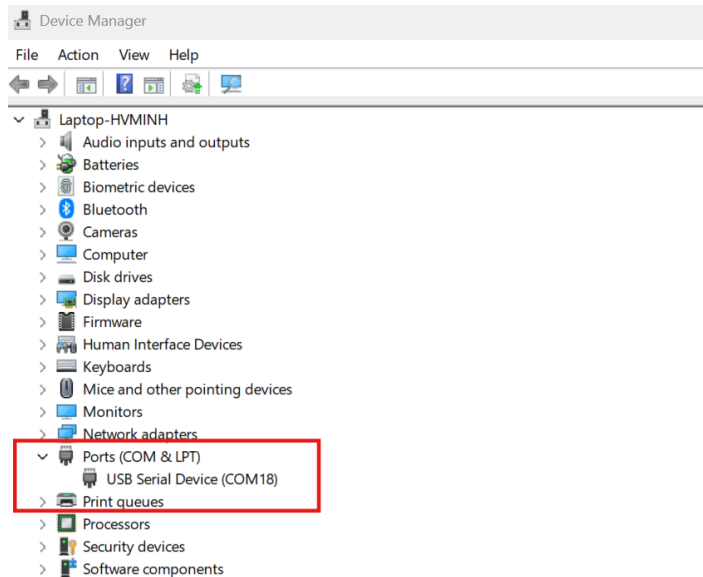
- Tải driver phiên bản cho macOS từ các link tương tự như trên.
- **Lưu ý quan trọng:** Từ các phiên bản macOS gần đây, hệ điều hành có các cơ chế bảo mật chặt chẽ hơn. Sau khi chạy cài đặt, bạn có thể cần vào **System Preferences** -> **Security & Privacy** -> **General** và nhấn nút "Allow" (Cho phép) để driver được hoạt động.

3. Đối với Linux:

- Hầu hết các bản phân phối Linux hiện đại (Ubuntu, Fedora, Mint...) thường **đã tích hợp sẵn driver** cho CP210x và FTDI.
- Driver cho CH340 cũng thường có sẵn trong kernel mới. Bạn có thể cắm bo mạch vào và kiểm tra xem hệ thống có nhận diện không (xem phần kiểm tra bên dưới).
- Nếu không được nhận diện, bạn có thể cần tìm kiếm hướng dẫn cài đặt driver CH340 cho bản phân phối Linux cụ thể của mình.
- Bạn cũng có thể cần thêm tài khoản người dùng của mình vào nhóm **dialout** để có quyền truy cập cổng nối tiếp: `sudo usermod -a -G dialout $USER` (thay **\$USER** bằng tên đăng nhập của bạn), sau đó đăng xuất và đăng nhập lại.

• Kiểm tra Driver đã cài đặt thành công (trên Windows):

1. Cắm bo mạch ESP của bạn vào cổng USB của máy tính.
2. Nhấn chuột phải vào nút Start -> chọn **Device Manager**.
3. Trong cửa sổ Device Manager, tìm và mở rộng mục **Ports (COM & LPT)**.
4. Nếu driver đã được cài đặt đúng và máy tính nhận diện bo mạch, bạn sẽ thấy một thiết bị mới xuất hiện trong danh sách này, ví dụ như:
 - **USB-SERIAL CH340 (COM3)**
 - **Silicon Labs CP210x USB to UART Bridge (COM4)**
 - **USB Serial Port (COM5)**



Cổng COM xuất hiện khi kết nối Yolo Node với máy tính

5. Hãy **ghi nhớ số cổng COM** này (ví dụ: COM3, COM4, COM5...). Bạn sẽ cần nó khi sử dụng các công cụ nạp firmware. Nếu không thấy mục Ports hoặc có thiết bị không nhận diện (có dấu chấm than màu vàng), driver có thể chưa được cài đúng.

4.2 Các công cụ nạp firmware (Flashing Tools)

"Flashing" hay "Nạp firmware" là quá trình ghi file firmware Tasmota (dạng file .bin) vào bộ nhớ Flash bên trong chip ESP, ghi đè lên firmware cũ đang có. Có nhiều công cụ giúp bạn thực hiện việc này:

1. Tasmota Web Installer:

- **Đặc điểm:** Công cụ dựa trên trình duyệt web, không cần cài đặt phần mềm riêng.
- **Ưu điểm: Dễ sử dụng nhất** cho người mới. Tự động phát hiện cổng COM, cung cấp sẵn các phiên bản Tasmota mới nhất để chọn và nạp. Chỉ cần trình duyệt hỗ trợ Web Serial API (như Google Chrome, Microsoft Edge phiên bản mới).

- **Nhược điểm:** Ít tùy chọn nâng cao (ví dụ: không có chức năng sao lưu firmware gốc).
- **Truy cập:** <https://tasmota.github.io/install/>

2. Tasmotizer:

- **Đặc điểm:** Phần mềm ứng dụng đồ họa (GUI) độc lập, có phiên bản cho Windows, macOS và Linux.
- **Ưu điểm:** Giao diện trực quan, dễ dùng. Tích hợp nhiều tính năng hữu ích: tự động phát hiện cổng COM, **tải firmware** Tasmota (bản ổn định và bản phát triển), tùy chọn **xóa sạch bộ nhớ flash (Erase flash)** trước khi nạp, và đặc biệt là khả năng **sao lưu (Backup) firmware gốc** của thiết bị trước khi thay thế bằng Tasmota.
- **Nhược điểm:** Cần tải về và chạy file thực thi Tasmotizer trên máy tính.
- **Nguồn tải:** Tìm kiếm "Tasmotizer github" để tải từ trang phát hành của dự án.

3. Esptool.py:

- **Đặc điểm:** Công cụ **dòng lệnh (command-line)** chính thức do Espressif (nhà sản xuất chip ESP) phát triển.
- **Ưu điểm:** **Mạnh mẽ, linh hoạt nhất.** Cung cấp toàn quyền kiểm soát quá trình nạp, xóa, đọc bộ nhớ flash với rất nhiều tùy chọn chi tiết. Hoạt động trên mọi hệ điều hành có cài đặt Python. Đây là công cụ nền tảng mà Tasmotizer và nhiều công cụ khác sử dụng bên trong.
- **Nhược điểm:** Yêu cầu người dùng phải quen thuộc với việc gõ lệnh trong cửa sổ Terminal hoặc Command Prompt. Cần cài đặt Python và cài đặt esptool thông qua pip (`pip install esptool`).
- **Đối tượng:** Phù hợp hơn với người dùng có kinh nghiệm hoặc khi cần xử lý các tác vụ phức tạp.

● Nên chọn công cụ nào?

- **Người mới:** Hãy bắt đầu với **Tasmota Web Installer**. Tài liệu này cũng sẽ ưu tiên hướng dẫn sử dụng công cụ này vì tính đơn giản và thuận tiện của nó.
- **Người dùng nâng cao:** **Esptool.py** là công cụ tối ưu.

4.3 Các phiên bản Firmware Tasmota

Firmware Tasmota là một file binary (có đuôi `.bin`) chứa mã lệnh để chip ESP thực thi. Bạn cần có file `.bin` này để nạp vào thiết bị. Các firmware Tasmota đã được build sẵn để có thể nạp có rất nhiều biến thể khác, mỗi biến thể chứa một tập hợp tính năng nhất định để tối ưu dung lượng cũng như bộ nhớ của thiết bị.

Dưới đây là một vài biến thể chính giúp bạn hình dung được phần nào khi chọn lựa để sử dụng:

- **tasmota.bin**: Phiên bản **chuẩn**, chứa các tính năng cốt lõi nhất (WebUI, Wi-Fi, MQTT, OTA, Rules cơ bản) và hỗ trợ một số cảm biến thông dụng (DHTxx, DS18B20...). **Đây thường là lựa chọn tốt nhất để bắt đầu cho hầu hết thiết bị sử dụng ESP8266.**
- **tasmota-sensors.bin**: Phiên bản mở rộng từ **tasmota.bin**, **bổ sung hỗ trợ cho nhiều loại cảm biến hơn** (I2C, analog, các cảm biến đặc biệt khác...). Cần nhiều bộ nhớ hơn, thường dùng cho thiết bị có 4MB Flash trở lên hoặc ESP32.
- **tasmota-lite.bin**: Phiên bản **siêu nhẹ**, đã loại bỏ một số tính năng (như WebUI hoặc một số lệnh) để **tiết kiệm bộ nhớ tối đa**. Thường dùng làm bước đệm khi cập nhật OTA cho thiết bị 1MB, hoặc khi bạn chỉ cần thiết bị giao tiếp qua MQTT mà không cần giao diện web.
- **Các phiên bản chuyên dụng**: **tasmota-display.bin** (hỗ trợ màn hình), **tasmota-ir.bin** (hỗ trợ điều khiển hồng ngoại), **tasmota-knx.bin**, **tasmota-zbbridge.bin** (cho Sonoff Zigbee Bridge), **tasmota-bluetooth.bin** (thêm hỗ trợ Bluetooth trên ESP32)...
- **Phiên bản cho ESP32**: Thường có tên chứa **tasmota32** (ví dụ: **tasmota32.bin**, **tasmota32-sensors.bin**, **tasmota32-bluetooth.bin**...). Do ESP32 có nhiều bộ nhớ hơn, các phiên bản này thường tích hợp nhiều tính năng hơn.

Vậy chúng ta nên chọn phiên bản nào khi nạp lần đầu?

- **ESP8266**: Bắt đầu với **tasmota.bin**.
- **ESP32**: Bắt đầu với **tasmota32.bin**.

- Nếu bạn biết chắc mình cần hỗ trợ nhiều cảm biến đặc biệt, hãy cân nhắc `tasmota-sensors.bin` (hoặc `tasmota32-sensors.bin`), nhưng hãy đảm bảo thiết bị của bạn có đủ bộ nhớ (thường là $\geq 4\text{MB}$ Flash).

4.4 Kết nối thiết bị ESP của bạn với máy tính

Đây là bước kết nối vật lý giữa bo mạch ESP và máy tính của bạn.

1. Chuẩn bị:

- Bo mạch phát triển Yolo Node.
- Cáp USB **có khả năng truyền dữ liệu** phù hợp với cổng trên bo mạch.
- Máy tính đã cài đặt driver USB-to-Serial thành công (Bước 4.1).

2. Kết nối:

- Cắm một đầu cáp USB vào bo mạch Yolo Node.
- Cắm đầu còn lại vào cổng USB trên máy tính của bạn.
- Quan sát đèn LED báo nguồn trên bo mạch, nó phải sáng lên.

3. Đưa ESP vào Chế độ Nạp Firmware (Flashing Mode / Bootloader Mode):

- Đây là bước **RẤT QUAN TRỌNG**. Để chip ESP trên Yolo Node có thể nhận firmware mới từ máy tính qua cổng Serial, nó cần được khởi động vào một chế độ đặc biệt gọi là "Bootloader Mode" hay "Flashing Mode".
- **Các bước thực hiện:**
 - Đầu tiên, nhấn và giữ nút "BOOT" trên mạch
 - Sau đó, trong khi vẫn giữ nút đó, kết nối mạch với máy tính bằng cáp USB.
- **Ngoài ra, bạn cũng có thể thực hiện theo cách sau mà không cần rút cáp USB:**
 - Tìm nút **BOOT** và **RESET** trên bo mạch.
 - **Nhấn và GIỮ** nút **BOOT**.

- Trong khi vẫn đang giữ nút BOOT, **nhấn rồi THẢ nút RESET.**
- Sau khoảng 1-2 giây, **thả nút BOOT.**
- **Lưu ý:** Thứ tự nhấn/giữ/thả là rất quan trọng.

4. Xác nhận lại kết nối:

- Sau khi kết nối cáp và đưa vào chế độ nạp, hãy kiểm tra lại trong Device Manager (Windows) hoặc dùng lệnh `ls /dev/ttyUSB*` hoặc `ls /dev/ttyACM*` (Linux/macOS) để đảm bảo máy tính vẫn đang nhận diện đúng cổng COM/tty của bo mạch.

Bây giờ, mọi thứ đã sẵn sàng! Máy tính đã nhận diện được bo mạch, bạn đã hiểu về các công cụ và phiên bản firmware. Chúng ta đã chuẩn bị đầy đủ để thực hiện quá trình nạp Tasmota vào thiết bị trong Module 5.

Module 5: Nạp Tasmota vào thiết bị

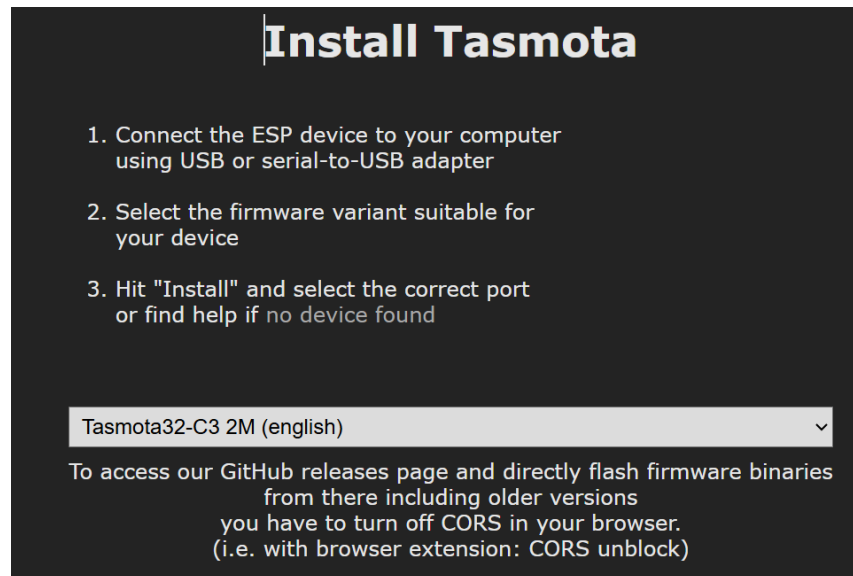
Mục tiêu

Đây là phần thực hành rất quan trọng, nơi chúng ta sẽ thực hiện quá trình "thay não" cho thiết bị ESP của bạn bằng firmware Tasmota. Sau đó, chúng ta sẽ thực hiện các bước cấu hình ban đầu để kết nối thiết bị vào mạng Wi-Fi.

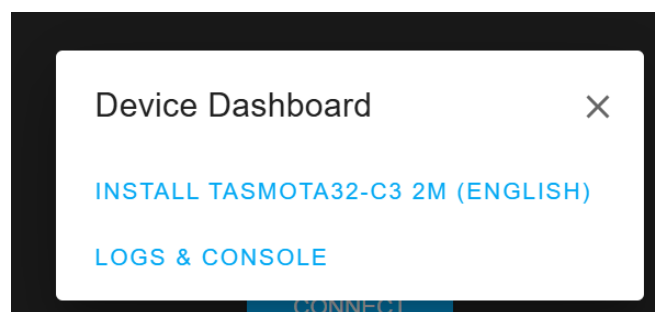
5.1 Hướng dẫn nạp bằng Tasmota Web Installer

Đây là phương pháp **dễ nhất** và được khuyến nghị cho người mới bắt đầu, không yêu cầu cài đặt phần mềm phức tạp.

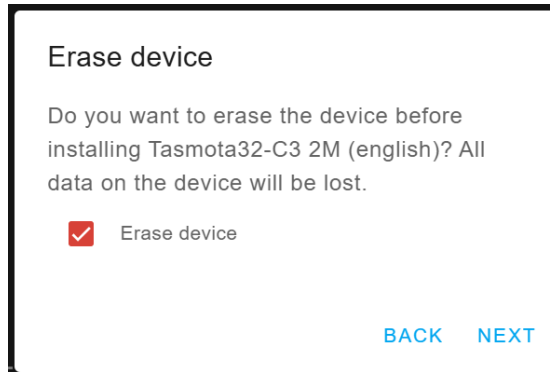
- **Chuẩn bị:**
 1. Máy tính đã cài đặt driver USB-to-Serial (từ Module 4).
 2. Trình duyệt web Google Chrome hoặc Microsoft Edge (phiên bản mới nhất).
 3. Cáp USB data tốt.
 4. Bo mạch Yolo Node đã được kết nối với máy tính qua cáp USB.
- **Các bước thực hiện:**
 1. Mở trình duyệt Chrome hoặc Edge và truy cập địa chỉ:
<https://tasmota.github.io/install/>
 2. Trong danh sách xổ xuống ("Select device type to install"), hãy **chọn phiên bản Tasmota** bạn muốn nạp.
 - Nếu bạn dùng bo mạch Yolo Node: Chọn phiên bản **Tasmota32-C3 2M** ở phần unofficial.
 - Nếu bạn dùng bo mạch Yolo UNO: Chọn phiên bản **Tasmota32-S3 (QIO Flash/OPI PSRAM)** ở phần unofficial.



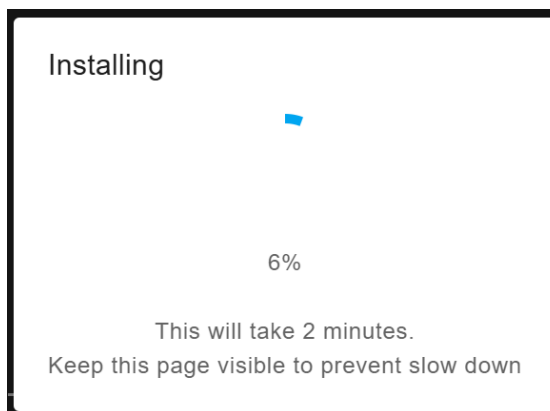
3. Nhấn vào nút màu xanh **"CONNECT"**.
4. Trình duyệt sẽ hiển thị một cửa sổ popup liệt kê các cổng Serial/COM khả dụng mà nó phát hiện được. **Chọn đúng cổng COM** tương ứng với bo mạch ESP của bạn (bạn đã xác định ở Module 4, ví dụ: COM3, COM4...). Sau đó nhấn nút **"Connect"** trên cửa sổ popup này.
5. Nếu kết nối thành công, nút "CONNECT" sẽ chuyển thành "INSTALL TASMOTA" (hoặc tên tương tự). Nhấn vào nút **"INSTALL TASMOTA"**.



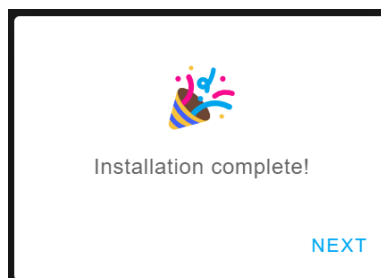
6. Một cửa sổ mới sẽ hiện ra hỏi bạn có muốn xóa thiết bị trước không. **Đánh dấu tích vào ô "Erase device"**. Việc này **rất quan trọng** khi nạp lần đầu để đảm bảo không còn dữ liệu cũ gây xung đột. Sau đó nhấn **"NEXT"**.



7. Nhấn **Next** và **INSTALL** để xác nhận và bắt đầu quá trình nạp firmware.
8. Bạn sẽ thấy thanh tiến trình chạy từ 0% đến 100%. Quá trình này thường mất khoảng 1-2 phút. **TUYỆT ĐỐI KHÔNG** rút cáp USB, đóng trình duyệt, hay tắt máy tính trong suốt quá trình này.

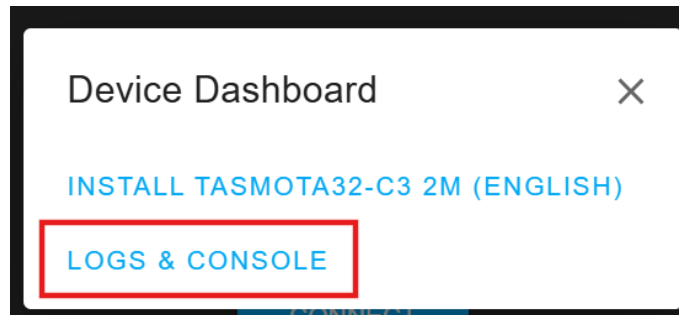


9. Khi thanh tiến trình đạt 100%, thông báo "**Installation complete**" sẽ xuất hiện. Nhấn "**NEXT**".



10. **Reset lại bo mạch bằng cách nhấn nút Reset trên mạch**

11. Sau khi reset thì kết nối trên công cụ web nạp firmware cũng sẽ bị ngắt. Bạn có thể Connect lại và vào Logs & Console để có thể xem được thông tin xuất ra trong quá trình chúng ta cấu hình Yolo Node.



Nhấn nút Reset Device để reset thiết bị một lần nữa giúp chúng ta thấy được các thông tin in ra khi Yolo Node khởi động.



Sau khi nhấn nút Reset Device trong cửa sổ Logs & Console, bạn sẽ thấy Yolo Node bắt đầu in ra một số thông tin cho biết thiết bị đã chạy và phát ra một Wi-Fi để bạn có thể bắt đầu cấu hình ở bước sau.

Logs

```
rst:0x15 (USB_UART_CHIP_RESET),boot:0xc (SPI_FAST_FLASH_BOOT)
Saved PC:0x4038025e
SPIWP:0xee
mode:DIO, clock div:1
load:0x3fcd5820,len:0x98
load:0x403cc710,len:0x870
load:0x403ce710,len:0x21bc
SHA-256 comparison failed:
Calculated: 92b968b1a2070f8c38a5825ad5afdde7ee3779b3564ef8905d1078e331c5f742
Expected: 96e29393a3c1e46d2daca62d8e5a267d4fa925513981d037bf9643a02fbafaff
Attempting to boot anyway...
entry 0x403cc710

00:00:00.001 CMD: Using USB CDC
00:00:00.001 HDW: ESP32-C3 v0.4
00:00:00.005 UFS: FlashFS mounted with 2228 kB free
00:00:00.011 CFG: Loaded from File, Count 4
00:00:00.014 SER: Set to 8M1 115200 bit/s
00:00:00.014 SER: HWCDC supports 115200 bit/s only
00:00:00.017 QPC: Count 1
00:00:00.043 BRV: Berry initialized, RAM used 3903 bytes
00:00:00.052 Project tasmota - Tasmota Version 14.5.0.3(c129743-tasmota32_2M)-3_1_3(2025-04-11T13:39:58)
00:00:00.355 WIF: WifiManager active for 3 minutes
00:00:01.037 HTTP: Web server active on tasmota-4B83BC-0956 with IP address 192.168.4.1
```

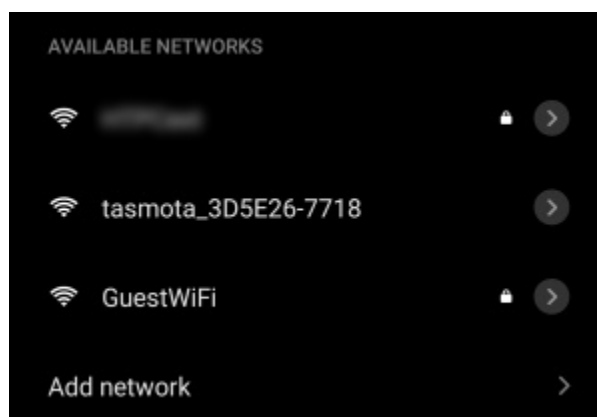
DOWNLOAD LOGS RESET DEVICE BACK

5.4 Cấu hình ban đầu cho thiết bị

Sau khi nạp Tasmota thành công và khởi động lại, thiết bị Yolo Node của bạn chưa biết cách kết nối vào mạng Wi-Fi của nhà bạn đang sử dụng. Thay vào đó, nó sẽ tự biến mình thành một **Điểm truy cập Wi-Fi (Access Point - AP)** tạm thời để bạn kết nối vào và tiến hành cài đặt.

Bạn thực hiện các bước sau để cấu hình:

1. Dùng **điện thoại thông minh** hoặc **máy tính xách tay** của bạn, mở phần cài đặt Wi-Fi.
2. **Tìm mạng Wi-Fi** có tên dạng **tasmota_XXXXXX-####** (XXXXXX-#### là một chuỗi ký tự và số duy nhất cho thiết bị của bạn).

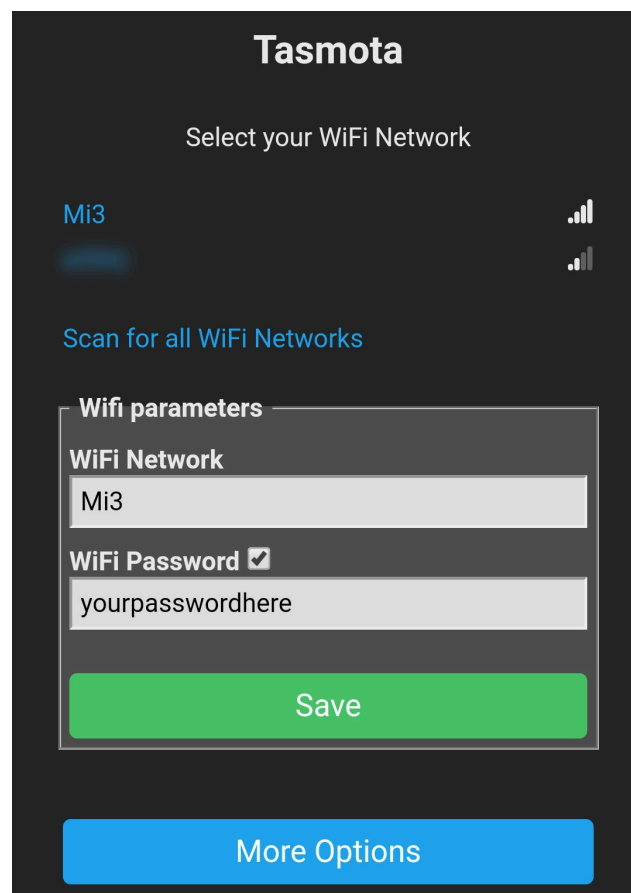


3. **Kết nối** vào mạng Wi-Fi **tasmota_...** này. Mạng này **không yêu cầu mật khẩu**.
4. Sau khi kết nối, điện thoại/máy tính của bạn có thể tự động mở một trang đăng nhập mạng (captive portal), hoặc bạn cần **mở trình duyệt web** (Chrome, Safari, Firefox...).
5. Trong thanh địa chỉ của trình duyệt, gõ địa chỉ IP **192.168.4.1** và nhấn Enter.
6. Bạn sẽ thấy trang cấu hình Wi-Fi ban đầu của Tasmota hiện ra.

5.5 Kết nối Tasmota vào Mạng Wi-Fi cục bộ của bạn

Bây giờ bạn sẽ "dạy" cho Tasmota cách kết nối vào mạng Wi-Fi chính của mình:

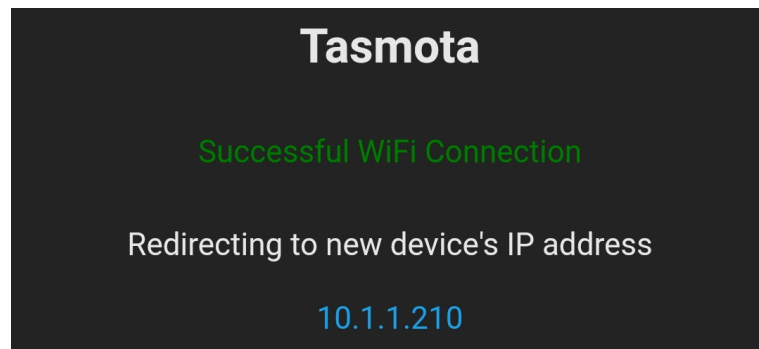
1. Trên trang cấu hình Tasmota (**192.168.4.1**), bạn sẽ thấy danh sách các mạng Wi-Fi mà thiết bị quét được. **Chọn tên mạng Wi-Fi (SSID)** của nhà bạn. (Nếu không thấy, nhấn "Scan again").



The screenshot shows the Tasmota web interface for configuring WiFi. At the top, it says "Tasmota" and "Select your WiFi Network". Below this, there's a section for "Mi3" with a signal strength indicator. A link "Scan for all WiFi Networks" is visible. The "Wifi parameters" section contains a "WiFi Network" dropdown menu with "Mi3" selected, a "WiFi Password" field with a checkmark icon and the placeholder text "yourpasswordhere", and a green "Save" button. At the bottom, there is a blue "More Options" button.

2. Trong ô **Password**, nhập **chính xác mật khẩu** mạng Wi-Fi của bạn. (Lưu ý chữ hoa/thường).

3. Bạn có thể bỏ qua các ô khác (Hostname, SSID2, Password2...).
4. Nhấn nút **"Save"**.
5. Thiết bị Tasmota sẽ lưu cấu hình, tự khởi động lại và cố gắng kết nối vào mạng Wi-Fi bạn vừa cung cấp. Quá trình này thường mất vài giây đến nửa phút. Điện thoại/máy tính của bạn sẽ tự động ngắt kết nối khỏi mạng **tasmota_...** Hãy **kết nối lại điện thoại/máy tính vào mạng Wi-Fi chính** của nhà bạn.
6. **Tìm địa chỉ IP mới của thiết bị Tasmota:** Sau khi Tasmota kết nối thành công vào mạng nhà bạn, nó sẽ nhận được một địa chỉ IP mới từ router của bạn. Có vài cách để tìm địa chỉ IP này:
 - Giao diện web cấu hình Wi-Fi cũng sẽ hiển thị địa chỉ IP này. Bạn cần lưu ý và ghi nhớ

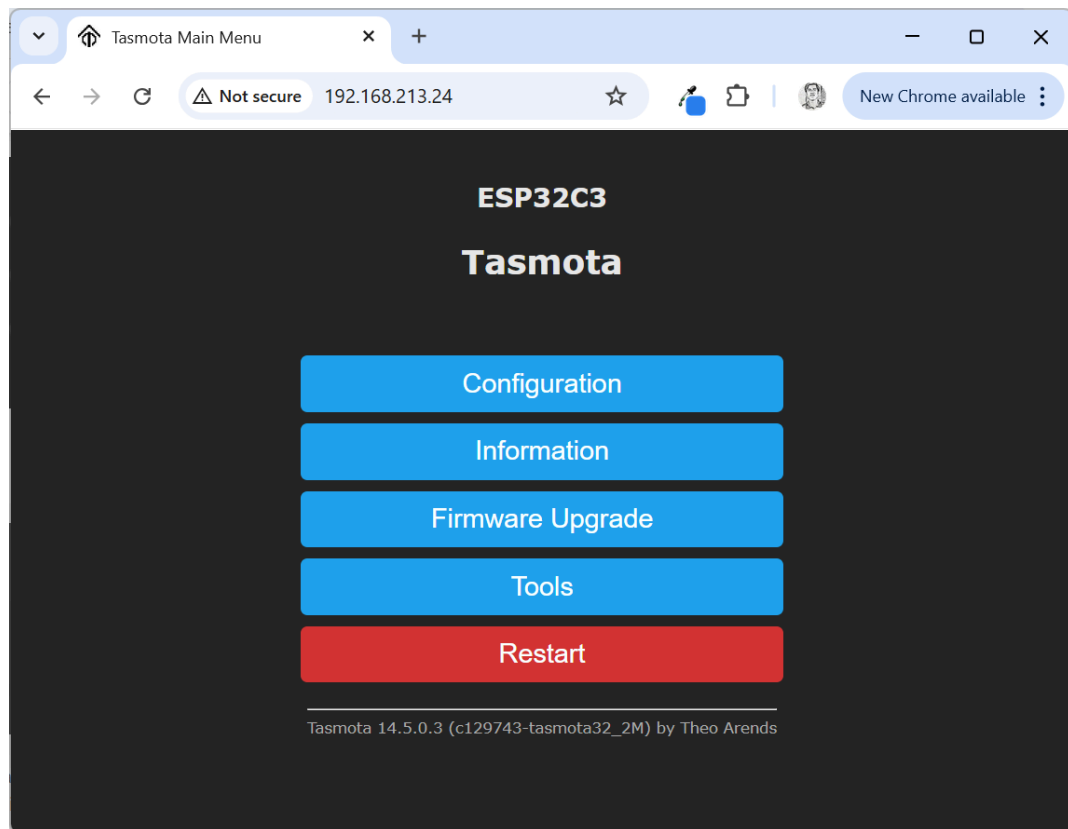


- **Xem trên cửa sổ Logs & Console:** Nếu bo mạch vẫn đang nối với máy tính, Tasmota sẽ in địa chỉ IP mới sau khi kết nối.

```
00:02:10.333 WIF: Connecting to AP1 test in mode HT40 as tasmota-4B83BC-0956...
00:02:17.214 WIF: Connected
00:02:17.476 HTP: Web server active on tasmota-4B83BC-0956 with IP address 192.168.213.24
00:02:18.433 RSL: INFO1 = {"Info1":{"Module":"ESP32C3","Version":"14.5.0.3(c129/43-tasmota32_2M)","FallbackTopic":"cmd/DVES_4B83BC_fb/", "GroupTopic":"cmd/tasmtas/"}}
00:02:18.436 RSL: INFO2 = {"Info2":{"WebServerMode":"Admin","Hostname":"tasmota-4B83BC-0956","IPAddress":"192.168.213.24","IP6Global":"","IP6Local":"fe80::2a37:2fff:fe4b:83bc%st8"}}
00:02:18.438 RSL: INFO3 = {"Info3":{"RestartReason":"Software reset CPU","BootCount":4}}
00:02:23.016 RSL: STATE = {"Time":"1970-01-01T00:02:23","Uptime":"0T00:02:24","UptimeSec":144,"Heap":196,"SleepMode":"Dynamic","Sleep":50,"Lo
{"HeapUsed":4,"Objects":36},"Wifi":
{"AP":1,"SSID":"test","BSSID":"9A:62:06:8F:5E:D4","Channel":6,"Mode":"HT40","RSSI":62,"Signal":-69
02:18"}}
```

- **Kiểm tra trên Router:** Đăng nhập vào trang quản trị router nhà bạn, tìm danh sách thiết bị đang kết nối (DHCP Client List), bạn sẽ thấy thiết bị có tên **tasmota_XXXXXX-####** và địa chỉ IP của nó.

- **Dùng ứng dụng quét mạng:** Sử dụng app như Fing (trên điện thoại) hoặc Advanced IP Scanner (trên máy tính) để quét mạng và tìm thiết bị Tasmota.
7. **Ghi nhớ địa chỉ IP mới** này (ví dụ: 192.168.1.105).
 8. **Truy cập WebUI Tasmota:** Mở trình duyệt web trên điện thoại hoặc máy tính (đảm bảo cùng mạng Wi-Fi với thiết bị), gõ địa chỉ IP mới bạn vừa tìm được vào thanh địa chỉ và nhấn Enter.
 9. **Thành công!** Nếu mọi việc suôn sẻ, bạn sẽ thấy giao diện WebUI chính của Tasmota hiện ra.



5.6 Các vấn đề phổ biến khi nạp firmware

Đôi khi quá trình nạp hoặc kết nối gặp trục trặc. Dưới đây là một số vấn đề thường gặp:

- **Lỗi không kết nối được khi nạp ("Failed to connect...", "Timed out..."):**
 - Kiểm tra lại cáp USB (phải là cáp data).

- Đảm bảo driver đã được cài đúng và chọn đúng cổng COM.
- **Thực hiện lại thao tác vào chế độ nạp** (Boot mode) cẩn thận, đúng thời điểm.
- Kiểm tra nguồn cấp cho bo mạch có ổn định không.
- **Lỗi trong quá trình ghi hoặc xác minh ("Error writing...", "Verify failed..."):**
 - **Xóa sạch flash (Erase flash)** rồi thử nạp lại.
 - Kiểm tra lại các tùy chọn nạp (Flash Size, Flash Mode nếu dùng esptool).
 - Thử dùng cáp USB khác hoặc cổng USB khác.
- **Nạp thành công nhưng Tasmota không phát AP `tasmota_...`:**
 - **Xóa sạch flash và nạp lại firmware.** Có thể firmware đã bị lỗi.
 - Kiểm tra nguồn cấp có đủ mạnh không.
 - Kết nối và mở cửa sổ Logs & Console xem có thông báo lỗi gì khi khởi động không.
- **Đã cấu hình Wi-Fi nhưng thiết bị không kết nối được (không tìm thấy IP mới):**
 - **Kiểm tra lại SSID và Mật khẩu Wi-Fi** đã nhập có chính xác 100% không (phân biệt hoa/thường, ký tự đặc biệt).
 - Đảm bảo router Wi-Fi của bạn đang hoạt động bình thường. Thử khởi động lại router.
 - Di chuyển thiết bị ESP đến gần router hơn để loại trừ vấn đề sóng yếu.
 - Kiểm tra xem router có cài đặt chặn thiết bị mới kết nối (MAC Filtering) hay không.
 - Kết nối và xem log kết nối Wi-Fi của Tasmota.

Xin chúc mừng! Bạn đã hoàn thành một trong những bước quan trọng và thú vị nhất: đưa Tasmota vào hoạt động trên thiết bị của mình và kết nối nó vào mạng. Ở Module tiếp theo, chúng ta sẽ cùng nhau khám phá Giao diện người dùng Web (WebUI) mạnh mẽ của Tasmota.

Module 6: Làm quen giao diện Web Tasmota

Mục tiêu

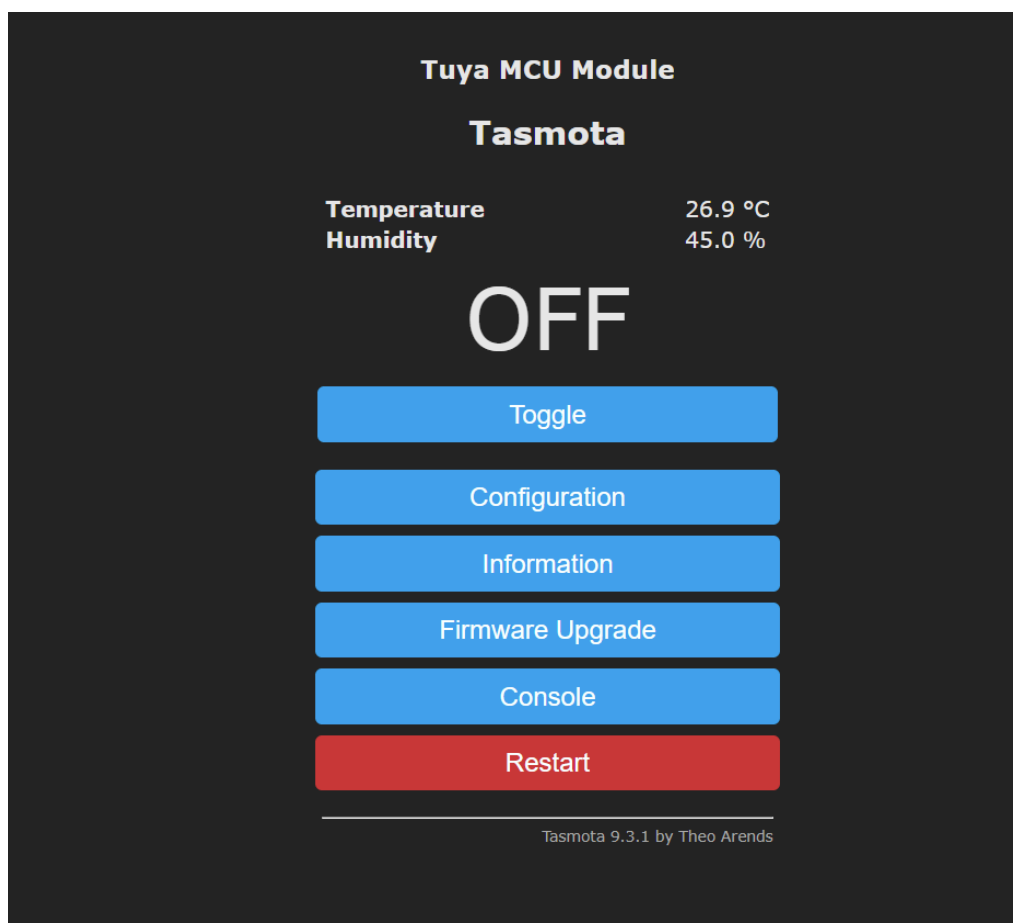
Sau khi đã nạp thành công Tasmota và kết nối thiết bị vào mạng Wi-Fi ở Module 5, Giao diện Người dùng Web (WebUI) chính là cổng chính để bạn tương tác, cấu hình và điều khiển thiết bị Tasmota của mình một cách trực quan. Module này sẽ hướng dẫn bạn cách làm chủ giao diện này.

6.1 Truy cập Thiết bị Tasmota của bạn qua địa chỉ IP

- **Điều kiện cần:**
 - Thiết bị Tasmota đã hoạt động và kết nối vào mạng Wi-Fi nhà bạn.
 - Bạn đã biết địa chỉ IP của thiết bị (tìm được ở cuối Module 5).
 - Máy tính hoặc điện thoại bạn dùng để truy cập phải đang kết nối vào **cùng một mạng Wi-Fi** với thiết bị Tasmota.
- **Các bước truy cập:**
 - Mở một trình duyệt web bất kỳ trên máy tính hoặc điện thoại (ví dụ: Chrome, Firefox, Safari).
 - Nhập **chính xác địa chỉ IP** của thiết bị Tasmota vào thanh địa chỉ của trình duyệt (ví dụ: **192.168.1.105**).
 - Nhấn phím **Enter**.
 - Nếu mọi thứ đúng, giao diện WebUI chính của Tasmota sẽ hiện ra.

6.2 Khám phá Menu chính và các thông tin hiển thị

Giao diện chính (Main Menu) cung cấp cái nhìn tổng quan và các điều khiển nhanh:



- **Tiêu đề:** Thường hiển thị tên thiết bị (bạn có thể đổi tên trong Configure Other) và phiên bản firmware Tasmota đang chạy.
- **Thông tin Trạng thái:** Một khu vực hiển thị các thông số quan trọng theo thời gian thực:
 - **Dữ liệu Cảm biến:** Nếu bạn đã cấu hình và kết nối các cảm biến (nhiệt độ, độ ẩm, ánh sáng...), giá trị đọc được sẽ hiển thị ở đây.
 - **Trạng thái Nguồn:** Hiển thị trạng thái ON/OFF của tất cả các relay/đèn được cấu hình (POWER1, POWER2...).

- **Nút Điều khiển Chính (Ví dụ: "Toggle"):** Nếu thiết bị của bạn được cấu hình với một hoặc nhiều Relay hoặc Đèn (Light), sẽ có các nút bật/tắt lớn ở đây (ví dụ: "Toggle" cho POWER1, "Toggle 2" cho POWER2...). Nhấn vào nút này sẽ thay đổi trạng thái bật/tắt của relay tương ứng ngay lập tức. Trạng thái hiện tại (ON/OFF) cũng được hiển thị rõ ràng.
- **Các Nút Menu Chính:** Thông thường nằm ở phía dưới hoặc thanh bên cạnh, cho phép truy cập các chức năng khác:
 - **Main Menu:** Nút để quay lại màn hình chính này từ các menu khác.
 - **Configuration:** Điểm truy cập vào tất cả các trang cài đặt chi tiết.
 - **Information:** Hiển thị thông tin chi tiết về phần cứng và phần mềm của thiết bị.
 - **Firmware Upgrade:** Thực hiện cập nhật firmware qua mạng (OTA).
 - **Console:** Mở bảng điều khiển dòng lệnh mạnh mẽ của Tasmota.
 - **Restart Device:** Khởi động lại thiết bị Tasmota.
- **Trang "Information":**
 - Nhấn vào nút **"Information"**.
 - Trang này cung cấp một bản tóm tắt cực kỳ chi tiết về thiết bị của bạn: Phiên bản Tasmota chính xác, thời gian biên dịch firmware, thông tin về bộ nhớ (Flash size, Free memory), địa chỉ IP, địa chỉ MAC (rất quan trọng để đặt DHCP Reservation), Subnet Mask, Gateway, DNS, trạng thái kết nối MQTT, tên Topic, trạng thái các chân GPIO... Đây là nơi rất hữu ích để kiểm tra thông số kỹ thuật và tìm thông tin khi cần gỡ lỗi.

ESP32C3

Tasmota

Information

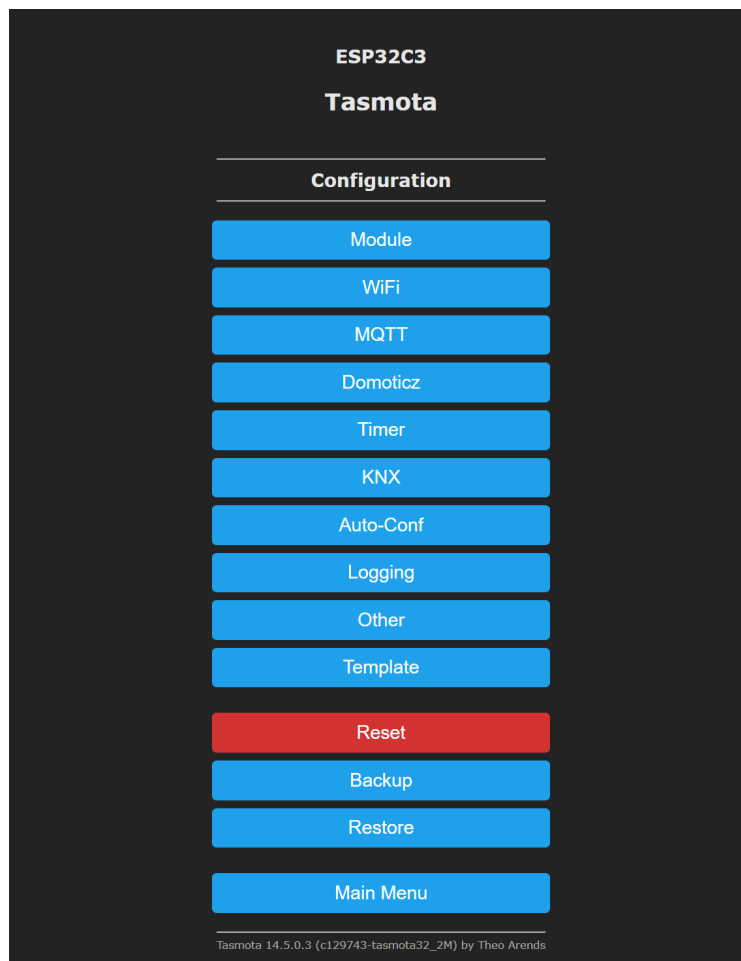
Program Version	14.5.0.3 (c129743-tasmota32_2M)
Build Date & Time	2025-04-11T13:39:58
Core/SDK Version	3_1_3/5.3.2.250403
Uptime	0T00:13:33
Flash Write Count	10
Boot Count	3
Restart Reason	Software reset CPU
AP1 Information	SSID test RSSI 74%, -63 dBm Mode HT40 Channel 6 BSSID 9A:62:06:8F:5E:D4
Hostname	tasmota-4B83BC-0956
IPv6 Local (WiFi)	fe80::2a37:2fff:fe4b:83bc%st8
MAC Address	28:37:2F:4B:83:BC
IP Address (WiFi)	192.168.213.24
Gateway	192.168.213.60
Subnet Mask	255.255.255.0
DNS Server1	192.168.213.60
DNS Server2	0.0.0.0
HTTP API	Enabled
MQTT Host	
MQTT Port	1883
MQTT TLS	Disabled
MQTT User	DVES_USER
MQTT Client	DVES_4B83BC
MQTT Topic	tasmota_%06X
MQTT Group Topic 1	cmnd/tasmtotas/
MQTT Full Topic	cmnd/tasmota_4B83BC/
MQTT Fallback Topic	cmnd/DVES_4B83BC_fb/
MQTT No Retain	Disabled
Emulation	None
ESP Chip Id	4948924 (ESP32-C3 v0.4)
Flash Chip Id	0x164020 (DIO)
Flash Size	4096 KB
Program Flash Size	4096 KB
Program Size	1573 KB
Free Program Space	218 KB
Free Memory	192.1 KB (frag. 39%)
Partition app0*	1792 KB (used 87%)
Partition fs	2240 KB

Main Menu

Tasmota 14.5.0.3 (c129743-tasmota32_2M) by Theo Arends

6.3 Tìm hiểu Menu Cấu hình

Đây là "trái tim" của việc tùy chỉnh Tasmota. Nhấn nút **"Configuration"** từ Main Menu để truy cập.



- Các mục cấu hình quan trọng nhất:
 - **Module:**
 - Đây là bước cấu hình phần cứng đầu tiên và quan trọng nhất sau khi nạp Tasmota. Bạn phải cho Tasmota biết nó đang chạy trên loại phần cứng nào.
 - **Wi-Fi:**

- Cho phép bạn thay đổi hoặc cấu hình lại thông tin kết nối Wi-Fi (SSID, Mật khẩu) cho tối đa 2 mạng.
- **MQTT:**
 - **Cực kỳ quan trọng nếu bạn muốn tích hợp Tasmota vào các nền tảng IoT như Core IoT hoặc hệ thống Home Assistant.** Đây là nơi bạn cung cấp thông tin để Tasmota kết nối đến máy chủ MQTT (MQTT Broker) của bạn.
- **Logging:**
 - Cài đặt cách Tasmota ghi lại nhật ký hoạt động.
- **Other:**
 - Chứa các cài đặt linh tinh nhưng quan trọng như đặt tên cho thiết bị, mật khẩu để login vào Web UI...
- **Timer:** Cho phép bạn thiết lập tối đa 16 bộ hẹn giờ đơn giản để tự động bật hoặc tắt các relay theo thời gian thực hoặc theo thời gian mặt trời mọc/lặn (cần cấu hình vĩ độ/kinh độ).
- **Nguyên tắc chung:** Sau khi thực hiện thay đổi trong các mục **Module**, **Wi-Fi**, **MQTT**, bạn cần nhấn **"Save"** và thiết bị sẽ **tự khởi động lại** để áp dụng cài đặt mới.

6.4 Sử dụng công cụ Console để gửi lệnh & xem Log

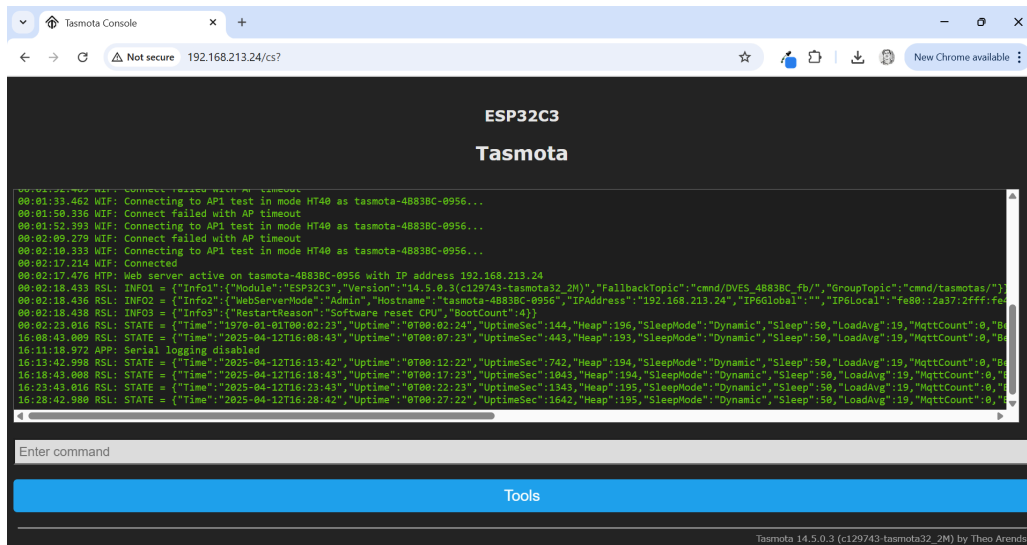
Console là một trong những công cụ mạnh mẽ và được sử dụng thường xuyên nhất khi làm việc với Tasmota.

Để truy cập vào công cụ Console, bạn nhấn vào menu **Tools > Console** trên WebUI.

Công cụ Console có 2 chức năng chính bạn cần biết:

1. **Hiển thị Log (Nhật ký hoạt động):**

Khung hiển thị chính của Console liên tục cập nhật các thông điệp log từ Tasmota theo thời gian thực.



Bạn có thể xem được:

- Quá trình khởi động của thiết bị.
- Trạng thái kết nối Wi-Fi.
- Trạng thái kết nối MQTT.
- Dữ liệu từ các cảm biến được đọc.
- Kết quả thực thi các lệnh.
- Các thông báo lỗi hoặc cảnh báo.
- Mức độ chi tiết của log được điều khiển bởi cài đặt "Web Log Level". Đây là công cụ **gỡ lỗi (debug)** số một khi bạn gặp vấn đề không rõ nguyên nhân.

2. Nhập và Gửi Lệnh (Command Input):

Phía dưới cùng của Console có một ô nhập liệu với nhãn "Enter command". Tại đây, bạn có thể gõ trực tiếp các **lệnh Tasmota (Tasmota Commands)** và nhấn Enter để thực thi.

- **Cú pháp:** Thường là **TênLệnh <ThamSố1> <ThamSố2>...** (Ví dụ: **Power1 ON, Status 8, TelePeriod 60, Rule1 ON**). Tên lệnh không phân biệt chữ hoa/thường.
- **Phản hồi:** Kết quả của lệnh (thường là một đoạn text hoặc JSON) hoặc thông báo lỗi sẽ được hiển thị ngay lập tức trên Console.
- **Ví dụ một số lệnh hữu ích:**

- **Power** hoặc **Power1**: Xem trạng thái Relay 1.
- **Power1 ON**: Bật Relay 1.
- **Power1 OFF**: Tắt Relay 1.
- **Power1 TOGGLE**: Đảo trạng thái Relay 1.
- **Status 0**: Xem trạng thái tổng quát.
- **Status 8**: Xem dữ liệu cảm biến.
- **Status 10**: Xem trạng thái MQTT.
- **Status 11**: Xem trạng thái các chân GPIO.
- **Restart 1**: Khởi động lại thiết bị.
- **Reset 1**: Đặt lại cấu hình về mặc định (Cẩn thận!).
- **WebLog 4**: Bật chế độ log chi tiết nhất để debug.
- Có hàng trăm lệnh khác nhau. Bạn có thể tham khảo danh sách đầy đủ trên trang tài liệu chính thức của Tasmota (Tasmota Commands).

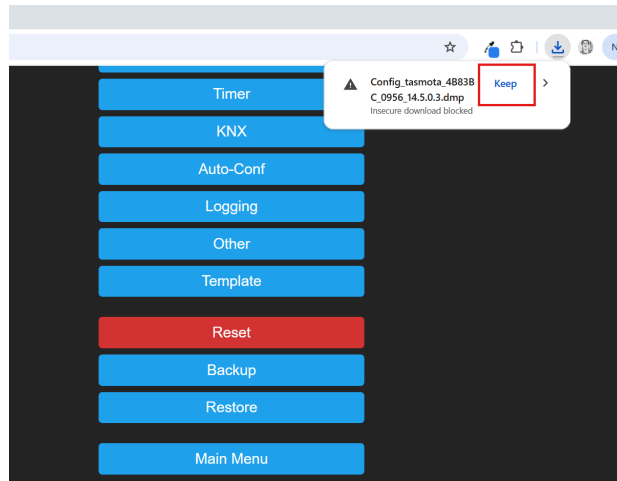
Tại sao công cụ Console lại quan trọng?

Nó cho phép bạn kiểm soát mọi thứ, xem mọi hoạt động bên trong và là công cụ không thể thiếu để khắc phục sự cố.

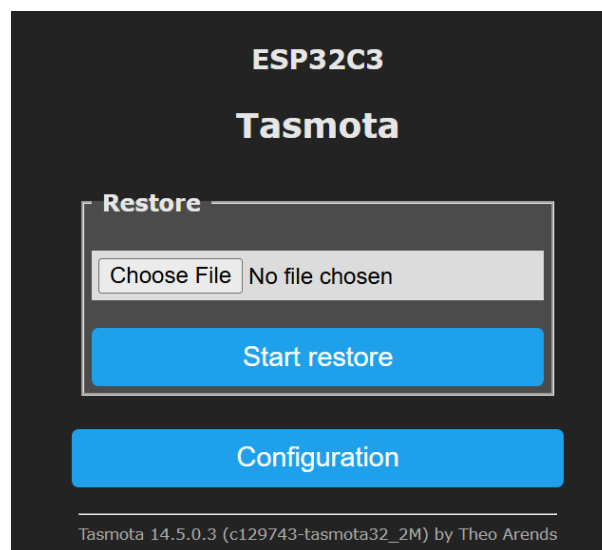
6.5 Sao lưu và khôi phục cấu hình Tasmota

Sau khi bạn đã dành thời gian để cấu hình mọi thứ (Module, Wi-Fi, MQTT, tên thiết bị, Rules...), việc tạo một bản sao lưu (backup) cho cấu hình đó là rất thông minh.

- **Cách Sao lưu (Backup):**
 - Từ WebUI, truy cập **Configuration -> Backup**.
 - Trình duyệt sẽ tự động tải về một file có đuôi **.dmp** (ví dụ: **tasmota_C8A75B_config.dmp**).
 - Nhiều trình duyệt web sẽ yêu cầu bạn xác nhận download file. Bạn cần xác nhận bằng cách nhấn vào nút Keep



- **Hãy lưu file .dmp này vào một nơi an toàn** trên máy tính của bạn. File này chứa tất cả các cài đặt bạn đã thực hiện.
- **Cách Khôi phục (Restore):**
 - Nếu bạn cần đưa thiết bị về lại cấu hình đã sao lưu (ví dụ sau khi reset hoặc nạp lại firmware), truy cập **Configuration -> Restore**.



- Nhấn nút **"Choose File"** (hoặc "Chọn file"). Tìm đến và **chọn file .dmp** mà bạn đã lưu trước đó.
- Nhấn nút **"Start Restore"**.

- **Cảnh báo:** Hành động này sẽ **xóa sạch cấu hình hiện tại** và thay thế bằng cấu hình trong file backup. Hãy chắc chắn bạn chọn đúng file!
- Thiết bị sẽ tự động khởi động lại sau khi quá trình khôi phục hoàn tất.

- **Lợi ích của việc Sao lưu:**

- Giúp bạn nhanh chóng phục hồi trạng thái hoạt động mong muốn nếu có sự cố.
- Giúp bạn dễ dàng "nhân bản" cấu hình cho nhiều thiết bị giống hệt nhau.

Kết thúc Module 6, bạn đã nắm vững cách làm việc với Giao diện WebUI của Tasmota. Từ việc truy cập, xem thông tin, cấu hình các thành phần quan trọng như Module, Wi-Fi, MQTT, sử dụng Console để điều khiển và gỡ lỗi, cho đến việc sao lưu/khôi phục cấu hình. Đây là những kỹ năng nền tảng để bạn tiếp tục tinh chỉnh và khai thác sức mạnh của Tasmota trong các module tiếp theo, bắt đầu với việc cấu hình phần cứng chi tiết ở Module 7.

Module 7: Cấu hình Phần cứng trong Tasmota

Mục tiêu

Sau khi đã nạp thành công firmware Tasmota lên thiết bị ESP, bước tiếp theo và cực kỳ quan trọng là phải "chỉ" cho Tasmota biết cách các thành phần phần cứng như Relay (Relay), Nút nhấn (Button), Đèn LED, Cảm biến... được kết nối với các chân GPIO (General Purpose Input/Output) của chip ESP như thế nào. Module này sẽ hướng dẫn bạn các cách thực hiện việc này.

7.1 Templates và Module trong Tasmota

Tại sao cần cấu hình phần cứng?

Firmware Tasmota bạn vừa nạp (ví dụ: [tasmota.bin](#)) là một phiên bản chung, nó có thể chạy trên nhiều loại thiết bị khác nhau. Tuy nhiên, bản thân Tasmota không tự động biết được rằng trên cái bo mạch cụ thể bạn đang dùng, thì Relay được nối vào chân GPIO số mấy, Nút nhấn nối vào chân nào, Đèn LED trạng thái nối vào đâu... Chúng ta cần cung cấp thông tin này cho Tasmota.

Có 3 cách chính để Tasmota hiểu về phần cứng của bạn:

- **Sử dụng Loại Module (Module Type) định nghĩa sẵn:** Tasmota đã tích hợp sẵn cấu hình cho rất nhiều thiết bị thương mại phổ biến (ví dụ: Sonoff Basic, Wemos D1 Mini...). Bạn chỉ cần chọn đúng tên thiết bị trong danh sách.
- **Sử dụng Mẫu (Template):** Template là một đoạn mã cấu hình (dạng JSON) mô tả chi tiết cách các linh kiện được nối với chân GPIO. Đây là cách linh hoạt hơn vì cộng đồng có thể tạo và chia sẻ template cho các thiết bị mới hoặc tùy chỉnh mà không cần chờ cập nhật firmware Tasmota.

- **Cấu hình GPIO thủ công (Manual GPIO Configuration):** Nếu không có Module Type hoặc Template nào phù hợp (ví dụ: khi bạn tự xây dựng mạch hoàn toàn từ đầu), bạn có thể chọn Module Type là "Generic (18)" và tự mình gán chức năng cho từng chân GPIO.

Trong hướng dẫn này, chúng ta sẽ chủ yếu sử dụng cách thứ 3 là cấu hình GPIO thủ công để có thể hiểu rõ hơn từng loại phần cứng.

7.2 Cấu hình GPIO thủ công

Phương pháp này phù hợp cho các phần cứng chưa nằm trong danh sách hỗ trợ chính thức của firmware Tasmota hoặc khi bạn không thể tìm thấy Module Type hay Template phù hợp.

Bo mạch Yolo Node hay Yolo UNO chúng ta sử dụng rơi vào trường hợp này. Để cấu hình đúng, chúng ta cần xem sơ đồ kết nối chân của các bo mạch này.

Yolo Node có các GPIO được kết nối với các ngoại vi tích hợp sẵn trên bo như sau:

Chức năng	GPIO
Nút nhấn (Boot)	GPIO9
Đèn LED	GPIO8
Chân SCL giao tiếp I2C (Cảm biến AHT20)	GPIO6
Chân SDA giao tiếp I2C (Cảm biến AHT20)	GPIO5
Cảm biến ánh sáng Analog	GPIO2

Chúng ta sẽ thực hiện các bước sau để cấu hình thông báo cho Tasmota hiểu:

- Truy cập WebUI, vào **Configuration -> Module**.
- Trong danh sách thả xuống "**Module type**", chọn giá trị "**ESP32C3 (0)**". Đây là module "trống", cho phép bạn tự định nghĩa mọi thứ.
- Bạn sẽ thấy một giao diện liệt kê tất cả các chân **GPIO** khả dụng của chip ESP-C3 (từ GPIO0 đến GPIO21 không liên tục do 1 số GPIO không khả dụng). Bên cạnh mỗi tên GPIO là một danh sách thả xuống.

- **Gán chức năng cho từng chân:** Đây là lúc bạn cần đối chiếu với sơ đồ kết nối trong bảng ở trên. Với mỗi chân GPIO bạn đã sử dụng để nối với một linh kiện, hãy chọn chức năng tương ứng từ danh sách thả xuống:

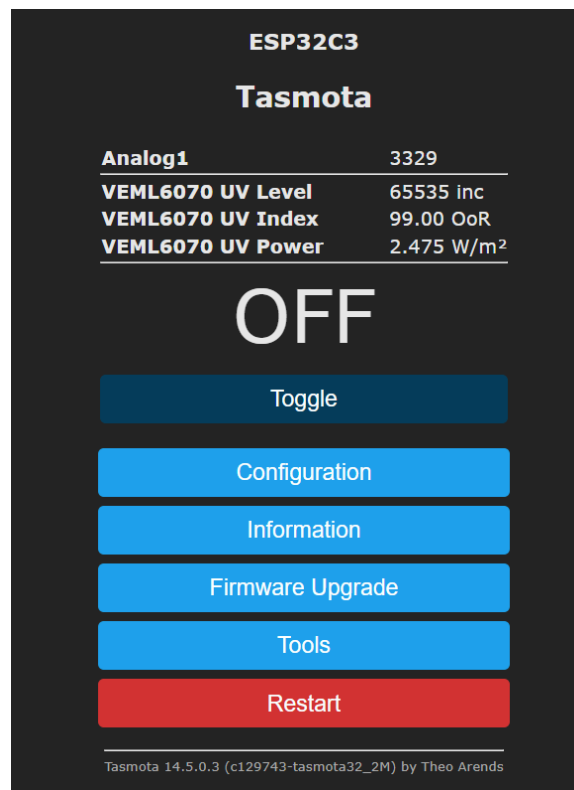
Module parameters	
Module type (ESP32C3)	
ESP32C3 (0)	
AO GPIO0	None
AO GPIO1	None
AO GPIO2	ADC Input
AO GPIO3	None
AO GPIO4	None
AO GPIO5	I2C SDA
IO GPIO6	I2C SCL
IO GPIO7	None
IO GPIO8	Relay
IO GPIO9	Button
IO GPIO10	None
IO GPIO18	None
IO GPIO19	None
RX GPIO20	None
TX GPIO21	None

Save

- Do đèn LED là một thiết bị có chung kiểu đóng ngắt nên chúng ta chọn là loại Relay. Nếu khi đèn LED bạn muốn dùng là để báo trạng thái, bạn có thể dùng loại chức năng là LED hoặc LED_i (nếu trên mạch thiết kế theo kiểu đảo ngược chân). Tasmota đều sẽ quản lý và làm chúng chạy như nhau.
- Đối với các GPIO không sử dụng: Hãy để giá trị là **None**.
- **Kiểm tra lại cẩn thận:** Đảm bảo bạn đã gán đúng chức năng cho đúng chân GPIO theo sơ đồ mạch của mình. Việc gán sai có thể khiến thiết bị không hoạt động, hoạt động sai.
- Sau khi đã gán xong tất cả các chân cần thiết, cuộn xuống cuối trang và nhấn **"Save"**.
- Thiết bị sẽ lại tự khởi động lại để áp dụng cấu hình GPIO bạn vừa thiết lập.

7.4 Kiểm tra cấu hình phần cứng của bạn

Sau khi Tasmota khởi động lại với cấu hình mới, bạn cần kiểm tra xem mọi thứ có hoạt động đúng như mong đợi không. Bạn sẽ thấy một giao diện như sau:



- **Kiểm tra Relay (Relay):**
 - Bạn sẽ thấy một nút "**Toggle**" tương ứng với số lượng Relay bạn đã cấu hình.
 - Nhấn vào nút "Toggle". Trạng thái hiển thị (ON/OFF) trên WebUI sẽ thay đổi và đèn LED trên Yolo Node sẽ bật/tắt tương ứng.
- **Kiểm tra Nút nhấn (Button):**
 - Nhấn nút và quan sát trên WebUI xem trạng thái của Relay tương ứng sẽ thay đổi theo.
 - Mở công cụ **Tools** > **Console** và quan sát log. Khi nhấn nút, bạn thường sẽ thấy các thông điệp như **POWER = ON** và thông báo trạng thái **POWER** thay đổi.

```

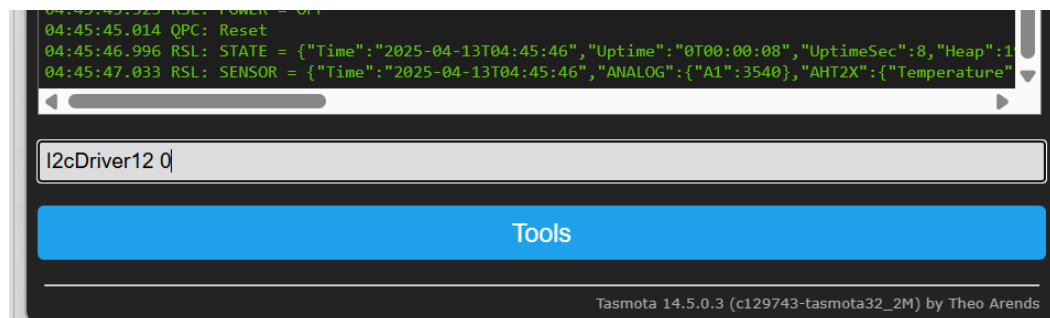
04:38:34.960 RSL: POWER = ON
04:38:36.194 RSL: RESULT = {"POWER":"OFF"}
04:38:36.194 RSL: POWER = OFF
04:38:40.222 RSL: RESULT = {"POWER":"ON"}
04:38:40.222 RSL: POWER = ON
04:38:43.496 RSL: RESULT = {"POWER":"OFF"}
04:38:43.497 RSL: POWER = OFF
04:39:54.870 RSL: RESULT = {"POWER":"ON"}
04:39:54.870 RSL: POWER = ON
04:40:05.292 RSL: RESULT = {"POWER":"OFF"}
04:40:05.292 RSL: POWER = OFF

```

- **Kiểm tra Cảm biến (Sensor):**

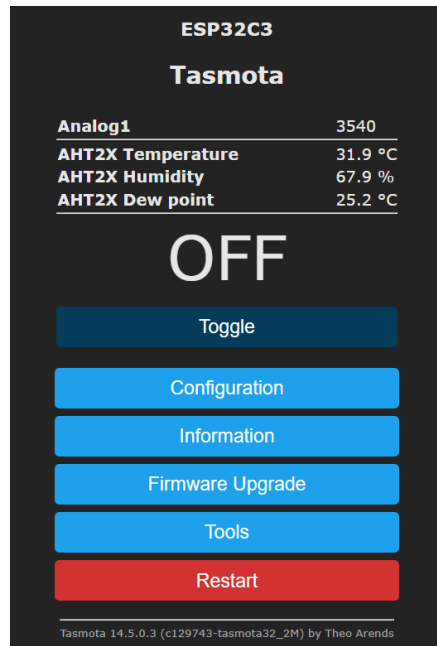
- Ở đây chúng ta có 1 vấn đề phải giải quyết đó là cảm biến AHT20 tích hợp sẵn trên Yolo Node bị Tasmota nhận diện sai thành cảm biến VEML6070 do 2 loại cảm biến này có chung địa chỉ I2C.
- Chúng ta cần thông báo lại cho Tasmota biết là bo mạch chúng ta dùng không có cảm biến VEML6070, khi đó Tasmota sẽ nhận diện đúng loại còn lại là AHT20 bằng câu lệnh trong cửa sổ Console (từ Home bạn vào Tools > Console)

I2cDriver12 0

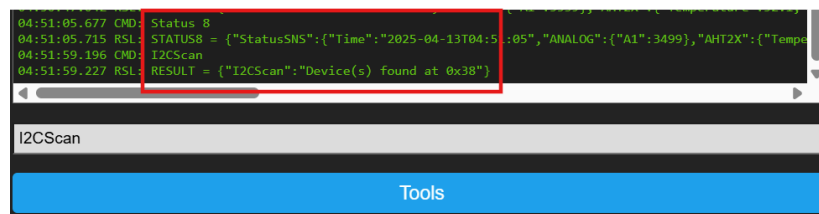


Trong câu lệnh trên, I2cDriver12 là mã của cảm biến VEML6070, 0 là yêu cầu tắt không nhận diện cảm biến này. Bạn có thể tra danh sách các cảm biến I2C được Tasmota hỗ trợ và mã của chúng [ở đây](#).

- Sau khi chạy lệnh này xong, Tasmota sẽ reset lại và nhận diện đúng AHT20 và các chỉ số đọc được sau vài giây.
- Các giá trị đọc được (Nhiệt độ, Độ ẩm, Ánh sáng...) sẽ hiển thị trực tiếp trên Main Menu.



- Bạn cũng có thể vào **Console** và gõ lệnh **Status 8** để yêu cầu Tasmota cập nhật và hiển thị dữ liệu cảm biến ngay lập tức dưới dạng JSON.
- **Kiểm tra thiết bị I2C:**
 - Cách tốt nhất là xem log trên **Console** khi thiết bị khởi động. Tasmota sẽ cố gắng quét các bus giao tiếp này và nếu tìm thấy thiết bị tương thích, nó sẽ báo trong log.
 - Đối với I2C, bạn có thể dùng lệnh **I2CScan** trên Console để xem địa chỉ của các thiết bị đang được kết nối.



Hoàn thành Module 7 là bạn đã thực sự làm chủ được việc cấu hình phần cứng cho thiết bị Tasmota của mình. Bạn đã biết cách sử dụng Module Type và tự tay gán từng chân GPIO. Bạn cũng biết cách kiểm tra để đảm bảo mọi thứ hoạt động chính xác. Bước tiếp theo, Module 8, sẽ đưa bạn đến với MQTT - chìa khóa để kết nối thiết bị Tasmota vào một hệ thống nhà thông minh lớn hơn.

Module 8: Giao thức MQTT trong IoT

Mục tiêu

Ở các module trước, chúng ta đã nạp firmware Tasmota, cấu hình Wi-Fi và phần cứng. Giờ là lúc tìm hiểu về MQTT - một giao thức cực kỳ quan trọng giúp thiết bị Tasmota của bạn "nói chuyện" được với các hệ thống nhà thông minh khác một cách hiệu quả và linh hoạt. Nếu không có MQTT, việc tích hợp Tasmota vào một hệ sinh thái nhà thông minh lớn hơn sẽ khó khăn hơn rất nhiều.

8.1 MQTT là gì?

- **MQTT là gì?**
 - Viết tắt của **Message Queuing Telemetry Transport**.
 - Là một **giao thức nhắn tin (messaging protocol)** hoạt động theo mô hình **Publish/Subscribe (Xuất bản/Đăng ký)**.
 - Được thiết kế đặc biệt cho môi trường IoT: **nhẹ (lightweight)**, **hiệu quả về băng thông, đáng tin cậy** ngay cả trên các mạng không ổn định.
 - Nó không phải là một hệ thống nhà thông minh hoàn chỉnh, mà là **giao thức truyền nhận thông tin** giữa các thành phần trong hệ thống đó.
- **Các thành phần Chính của Hệ thống MQTT:**
 - **MQTT Broker (Máy chủ trung gian / "Bưu cục"):**
 1. Là thành phần trung tâm, hoạt động như một người đưa thư hoặc một bưu cục.
 2. **Tất cả** các thiết bị và ứng dụng (gọi là Clients) đều **kết nối đến Broker**.

3. Nhiệm vụ: Nhận tin nhắn từ Client gửi đến (Publishers) và chuyển tiếp những tin nhắn đó đến các Client khác đã đăng ký nhận chúng (Subscribers). Broker **không** xử lý nội dung tin nhắn, chỉ điều phối chúng.
 4. *Ví dụ về phần mềm Broker: Mosquitto* (mã nguồn mở, rất phổ biến, có thể tự cài đặt), HiveMQ, EMQX, các dịch vụ Cloud MQTT Broker (AWS IoT, Google Cloud IoT...).
- **MQTT Clients (Máy khách / "Người gửi/nhận thư"):**
 1. Là bất kỳ thiết bị, ứng dụng, hoặc dịch vụ nào kết nối đến MQTT Broker.
 2. Client có thể đóng vai trò là **Publisher** (gửi tin nhắn), **Subscriber** (nhận tin nhắn), hoặc cả hai.
 3. *Ví dụ:* Thiết bị Tasmota của bạn, nền tảng Home Assistant, ứng dụng điều khiển nhà thông minh trên điện thoại...
 4. Mỗi Client khi kết nối đến Broker phải có một **Client ID duy nhất** để Broker nhận diện.
 - **Topics (Chủ đề / "Địa chỉ thư"):**
 1. Là các chuỗi ký tự dùng để **phân loại và định tuyến tin nhắn**. Hoạt động giống như địa chỉ trên phong bì thư hoặc tên các kênh chat.
 2. Topics được tổ chức theo cấu trúc phân cấp, sử dụng dấu gạch chéo / để phân tách các cấp. Ví dụ:
nha/phong_khach/den_tran/trang_thai,
van_phong/khu_a/cam_bien/nhiet_do.
 3. Clients **publish (xuất bản)** tin nhắn **LÊN** một Topic cụ thể.
 4. Clients **subscribe (đăng ký)** để nhận tin nhắn **TỪ** một hoặc nhiều Topic cụ thể. Có thể sử dụng ký tự đại diện (wildcards như **+** và **#**) khi đăng ký để nhận tin nhắn từ nhiều topic cùng lúc.
 - **Messages (Tin nhắn / "Nội dung thư"):**
 1. Là dữ liệu thực tế được truyền đi.
 2. Nội dung tin nhắn (payload) có thể là bất kỳ định dạng nào (văn bản thuần túy, số, JSON, XML, ảnh, binary...).
 3. Trong ngữ cảnh nhà thông minh và Tasmota, payload thường là **văn bản đơn giản** (ví dụ: **ON**, **OFF**, **TOGGLE**) hoặc định dạng **JSON** (ví dụ: `{"Temperature": 28.5, "Humidity": 75.2}`).

- **Mô hình Publish/Subscribe (Pub/Sub):**

- Đây là điểm khác biệt chính so với mô hình Client-Server truyền thống (như HTTP).
- **Hoạt động:**
 1. Một Client (Subscriber) kết nối đến Broker và nói: "Tôi muốn nhận tất cả tin nhắn được gửi đến topic `nha/phong_khach/den_tran/lenh`".
 2. Một Client khác (Publisher) kết nối đến Broker và gửi (publish) tin nhắn `ON` đến topic `nha/phong_khach/den_tran/lenh`. Publisher không cần biết ai sẽ nhận tin nhắn này.
 3. Broker nhận tin nhắn `ON` từ Publisher, kiểm tra xem có Subscriber nào đang lắng nghe topic `nha/phong_khach/den_tran/lenh` không.
 4. Broker chuyển tiếp tin nhắn `ON` đến tất cả các Subscriber đó.
- **Lợi ích:** Tạo ra sự **tách biệt (Decoupling)** hoàn toàn giữa người gửi và người nhận. Họ không cần biết về sự tồn tại của nhau, chỉ cần thống nhất về Broker và cấu trúc Topic. Điều này giúp hệ thống trở nên cực kỳ **linh hoạt, dễ mở rộng và thay đổi**.

8.2 Tại sao MQTT cần thiết cho IoT?

MQTT đã trở thành một chuẩn de-facto (tiêu chuẩn không chính thức nhưng được chấp nhận rộng rãi) trong lĩnh vực IoT và nhà thông minh vì những lý do sau:

- **Nhẹ và hiệu quả (Lightweight & Efficient):** Giao thức MQTT có phần header (thông tin điều khiển) rất nhỏ, giúp giảm thiểu lượng dữ liệu truyền trên mạng. Điều này cực kỳ quan trọng đối với các thiết bị có cấu hình yếu như ESP8266/ESP32 và tiết kiệm băng thông mạng gia đình. Nó cũng tiêu thụ ít năng lượng hơn so với HTTP.
- **Đáng tin cậy (Reliable):**
 - Hỗ trợ 3 mức **Chất lượng Dịch vụ (Quality of Service - QoS)** để đảm bảo việc gửi tin nhắn:

- **QoS 0:** Gửi tối đa 1 lần (At most once) - Nhanh nhất nhưng có thể mất tin nhắn nếu mạng lỗi.
 - **QoS 1:** Gửi ít nhất 1 lần (At least once) - Đảm bảo tin nhắn đến nơi, nhưng có thể bị lặp lại nếu có lỗi xác nhận.
 - **QoS 2:** Gửi đúng 1 lần (Exactly once) - Đảm bảo nhất nhưng tốn nhiều tài nguyên và độ trễ cao hơn.
 - (Tasmota thường dùng QoS 0 hoặc 1).
 - Hỗ trợ tính năng **Last Will and Testament (LWT)**: Client có thể đăng ký trước một tin nhắn "di chúc" với Broker. Nếu Client bị mất kết nối đột ngột (ví dụ: mất điện, treo), Broker sẽ tự động gửi tin nhắn LWT này đến một topic đã định, giúp các hệ thống khác biết được thiết bị đã offline.
- **Mô hình Pub/Sub lý tưởng cho Nhà thông minh:** Như đã giải thích ở trên, mô hình này cho phép các thiết bị (cảm biến, công tắc) và hệ thống điều khiển (Home Assistant, Node-RED) giao tiếp với nhau một cách linh hoạt mà không cần kết nối trực tiếp.
 - **Giao tiếp hai chiều (Bi-directional):** Thiết bị Tasmota có thể vừa publish trạng thái và dữ liệu cảm biến lên Broker, vừa subscribe để nhận lệnh điều khiển từ Broker.
 - **Cập nhật trạng thái tức thời (Real-time):** Khi một sự kiện xảy ra (nhấn nút, cảm biến thay đổi), tin nhắn MQTT có thể được gửi đi ngay lập tức, giúp hệ thống nhà thông minh phản ứng nhanh chóng, không cần phải chờ đợi để "hỏi thăm" trạng thái liên tục (gọi là cơ chế polling).
 - **Chuẩn mở và phổ biến:** MQTT là một tiêu chuẩn mở được định nghĩa rõ ràng. Nó được hỗ trợ bởi hầu hết các nền tảng nhà thông minh mã nguồn mở và thương mại, rất nhiều thư viện lập trình cho các ngôn ngữ khác nhau, và các nhà cung cấp dịch vụ đám mây lớn.

8.3 Thiết lập một MQTT Broker

Để sử dụng MQTT, bạn **bắt buộc phải có một MQTT Broker** đang chạy. Có hai lựa chọn Broker chính:

1. Tự cài đặt Broker cục bộ (Local Broker):

- **Ưu điểm:** Kiểm soát hoàn toàn dữ liệu trong mạng nhà bạn, không phụ thuộc Internet, tốc độ phản hồi nhanh nhất, miễn phí (phần mềm). Đây là **lựa chọn được khuyến nghị mạnh mẽ** khi dùng với Tasmota để đảm bảo tính năng Local Control.
- **Nhược điểm:** Cần một thiết bị phần cứng (như Raspberry Pi, máy tính mini, NAS, máy tính cũ...) để chạy Broker 24/7. Cần tự thực hiện việc cài đặt, cấu hình và bảo trì.
- **Phần mềm phổ biến nhất: Mosquitto.** Miễn phí, mã nguồn mở, rất nhẹ, ổn định và có sẵn cho nhiều hệ điều hành (Linux, Windows, macOS). Nếu bạn dùng Home Assistant OS hoặc Supervised, có sẵn Add-on "Mosquitto broker" cực kỳ tiện lợi.

2. Sử dụng MQTT Broker có sẵn trên Internet:

- **Ưu điểm:** Không cần lo lắng về phần cứng hay cài đặt. Nhà cung cấp dịch vụ sẽ quản lý Broker cho bạn. Thường dễ dàng truy cập từ xa.
- **Nhược điểm:** **Phụ thuộc vào kết nối Internet** và nhà cung cấp dịch vụ. Có thể phát sinh chi phí (nhiều dịch vụ có gói miễn phí với giới hạn). Dữ liệu của bạn sẽ đi qua máy chủ bên ngoài, cần cân nhắc về quyền riêng tư.
- **Ví dụ:** HiveMQ Cloud, Flespi, CloudMQTT (ít miễn phí hơn trước), các dịch vụ tích hợp của AWS/Google Cloud/Azure...

Để đơn giản hóa, tài liệu này sẽ hướng dẫn bạn sử dụng một MQTT broker có sẵn trên Internet do OhStem setup ở địa chỉ **mqtt.ohstem.vn** và port **1883**.

Bạn có thể tham khảo rất nhiều hướng dẫn cách tự cài đặt một MQTT broker của riêng mình trên Internet, ví dụ như [ở đây](#).

8.4 Cấu hình cài đặt MQTT của Tasmota

Bây giờ, bạn cần "chỉ" cho thiết bị Tasmota biết cách tìm và kết nối đến MQTT Broker bạn vừa thiết lập.

1. Truy cập WebUI, vào **Configuration -> MQTT**.

2. Điền thông tin kết nối Broker:

- **Host:** Nhập địa chỉ của MQTT Broker là **mqtt.ohstem.vn**.
- **Port:** Nhập port MQTT là **1883**. Nếu bạn dùng một broker khác thì cần điền đúng giá trị port.
- **Client:** Để trống (Tasmota sẽ tự tạo dạng **DVES_XXXXXX**) hoặc đặt một tên duy nhất cho thiết bị này (ví dụ: **Tasmota_DenHoc**). **Không được trùng** với Client ID của thiết bị khác.
- **User:** Để trống vì OhStem Broker không yêu cầu xác thực. Bạn có thể nhập **tên user** trên Broker khác nếu có yêu cầu.
- **Password:** Để trống vì OhStem Broker không có yêu cầu xác thực.
- **Topic:** Đặt một **tên định danh duy nhất** cho thiết bị. Quy tắc đặt tên tốt: **ngắn gọn, không dấu, không khoảng trắng, dùng chữ thường, có thể dùng gạch dưới _**. Ví dụ: **den_ban_hoc**, **cam_bien_ban_cong**, **o_cam_quat**. Tên này sẽ được dùng làm giá trị cho biến **%topic%**. **Nếu bạn mới bắt đầu, bạn nên để giá trị mặc định** của Tasmota chính là tên thiết bị có dạng **tasmota_%06X**.
- **Full Topic:** Cấu trúc đầy đủ của các topic. **Nên để mặc định là %prefix%/ %topic%/** nếu bạn mới bắt đầu.

MQTT parameters

Host ()

mqtt.ohstem.vn

Port (1883)

1883

■ MQTT TLS

Client (DVES_4B83BC)

DVES_%06X

User (DVES_USER)

DVES_USER

Password ■

....

Topic = %topic% (tasmota_4B83BC)

tasmota_%06X

Full Topic (%prefix%/topic%/)

%prefix%/topic%/

Save

- Cuộn xuống và nhấn **"Save"**.
- Thiết bị Tasmota sẽ tự động **khởi động lại**.
- Kiểm tra kết nối**: Sau khi khởi động lại, vào **Console** để xem log sẽ thấy được Tasmota kết nối MQTT thành công hay chưa.

```

00:00:00.000 UART: Berry initialized, RAM used 3568 bytes
00:00:00.061 Project tasmota - Tasmota Version 14.5.0.3(c129743-tasmota32_2M)-3_1_3(2025-04-11T13:39:58)
00:00:00.111 I2C: AHT2X found at 0x38
00:00:01.002 WiFi: Connecting to API Gốc C3 Coffee Channel 11 BSSID 74:83:C2:1:29:8F in mode HT40 as tasmota-4B83BC-0956...
00:00:01.362 WiFi: Connected
00:00:02.568 HTTP: Web server active on tasmota-4B83BC-0956 with IP address 192.168.1.55
07:07:49.531 MQTT: Attempting connection...
07:07:49.673 MQTT: Connected
07:07:49.676 MQTT: tele/tasmota_4B83BC/LWT = Online (retained)
07:07:49.678 MQTT: cmd/tasmota_4B83BC/POWER =
07:07:49.684 MQTT: tele/tasmota_4B83BC/INFO1 = {"Info1":{"Module":"ESP32C3","Version":"14.5.0.3(c129743-tasmota32_2M)","FallbackT
07:07:49.689 MQTT: tele/tasmota_4B83BC/INFO2 = {"Info2":{"WebServerMode":"Admin","Hostname":"tasmota-4B83BC-0956","IPAddress":"19
07:07:49.692 MQTT: tele/tasmota_4B83BC/INFO3 = {"Info3":{"RestartReason":"Software reset CPU","BootCount":5}}
07:07:49.695 MQTT: stat/tasmota_4B83BC/RESULT = {"POWER":"OFF"}
07:07:49.697 MQTT: stat/tasmota_4B83BC/POWER = OFF
07:07:52.016 QPC: Reset
07:07:54.023 MQTT: tele/tasmota_4B83BC/STATE = {"Time":"2025-04-13T07:07:54","Uptime":"0T00:00:10","UptimeSec":10,"Heap":191,"Sla
07:07:54.063 MQTT: tele/tasmota_4B83BC/SENSOR = {"Time":"2025-04-13T07:07:54","ANALOG":{"A1":2702},"AHT2X":{"Temperature":null,"H

```

- Nếu không thành công, hãy kiểm tra lại thông tin đã nhập, tình trạng mạng, và xem log trên **Console** để biết chi tiết lỗi.

8.5 Hiểu cấu trúc các topic MQTT của Tasmota

Tasmota sử dụng một hệ thống topic rất có tổ chức, giúp việc quản lý và tích hợp trở nên dễ dàng. Với cấu hình **FullTopic** mặc định là `%prefix%/%topic%/`, các topic chính sẽ có dạng:

- **cmnd/%topic%/<Command> (Command Topic):**
 - **Mục đích:** Dùng để **GỬI LỆNH** điều khiển đến thiết bị Tasmota.
 - **Ví dụ:** Để bật đèn có `%topic%` là `tasmota_4B83BC` (tên thiết bị của người viết), bạn publish tin nhắn `ON` đến topic `cmnd/tasmota_4B83BC/Power`. Để yêu cầu cập nhật trạng thái, bạn publish tin nhắn trống đến `cmnd/tasmota_4B83BC/Status 0`.
- **stat/%topic%/<Result> (Status Topic):**
 - **Mục đích:** Tasmota dùng topic này để **GỬI PHẢN HỒI** trạng thái hoặc kết quả sau khi thực hiện một lệnh `cmnd`.
 - **Ví dụ:** Sau khi nhận lệnh `cmnd/tasmota_4B83BC/Power ON`, Tasmota sẽ publish tin nhắn `{ "POWER" : "ON" }` đến topic `stat/tasmota_4B83BC/RESULT`. Nếu bạn gửi `cmnd/tasmota_4B83BC/Status 8`, nó sẽ publish dữ liệu cảm biến đến `stat/tasmota_4B83BC/STATUS8`.
- **tele/%topic%/<Sensor/State> (Telemetry Topic):**
 - **Mục đích:** Tasmota **TỰ ĐỘNG GỬI ĐI** dữ liệu trạng thái (uptime, wifi signal...) và dữ liệu cảm biến (nhiệt độ, độ ẩm...) theo thời gian định kỳ (**TelePeriod**).
 - **Ví dụ:** Cứ mỗi 300 giây (mặc định), Tasmota sẽ publish tin nhắn JSON chứa thông tin trạng thái đến `tele/tasmota_4B83BC/STATE`. Nếu có cảm biến, nó sẽ publish dữ liệu đến `tele/tasmota_4B83BC/SENSOR`.
 - Topic `tele/%topic%/LWT` được dùng cho tin nhắn Last Will and Testament, thường có giá trị `Online` hoặc `Offline`.

Việc hiểu rõ ba tiền tố **cmd**, **stat**, **tele** và biến **%topic%** là nền tảng để bạn giao tiếp hiệu quả với thiết bị Tasmota qua MQTT.

8.6 Kiểm tra giao tiếp MQTT

Cách tốt nhất để đảm bảo mọi thứ hoạt động đúng là sử dụng một MQTT Client độc lập để theo dõi và tương tác với Broker. Có nhiều phần mềm MQTT client miễn phí bạn có thể download và cài đặt trên máy tính.

Tuy nhiên để đơn giản, bạn có thể sử dụng một công cụ MQTT client phiên bản web để có thể sử dụng ngay mà không cần cài đặt. Một trong số đó là HiveMQ MQTT Websocket Client ở URL <https://www.hivemq.com/demos/websocket-client/>. Công cụ này cho phép bạn gửi tin nhắn MQTT cũng như đăng ký để nhận được dễ dàng.

Các bước thao tác như sau:

1. Truy cập vào đường link của HiveMQ trên
2. **Khai báo và kết nối:**
 - Host: **mqtt.ohstem.vn**
 - Port: **8084** (do web sử dụng MQTT port khác với port mà thiết bị sử dụng)
 - Username và Password: Bỏ trống vì Broker OhStem không đòi hỏi xác thực
 - Nhấn "**Connect**". Nếu thông tin đúng thì công cụ sẽ chuyển sang trạng thái là Connected

MQTT Websocket Client

hivemq.com/demos/websocket-client/?

HIVEMQ CLOUD Need a fully managed MQTT broker? Get your own Cloud broker and connect up to 100 devices for free. [Get your free account](#)

Connection disconnected

Host: mqtt.ohstem.vn Port: 8084 ClientID: clientid-W1pWoJOJLE [Connect](#)

Username: Password: Keep Alive: 60 SSL: Clean Session: ☒

Last-Will Topic: Last-Will QoS: 0 Last-Will Retain: ☐

Last-Will Message:

Publish **Subscriptions** **Messages**

HIVEMQ CLOUD Need a fully managed MQTT broker? Get your own Cloud broker and connect up to 100 devices for free. [Get your free account](#)

Connection connected

Publish

Topic: testtopic/1 QoS: 0 Retain: ☐ [Publish](#)

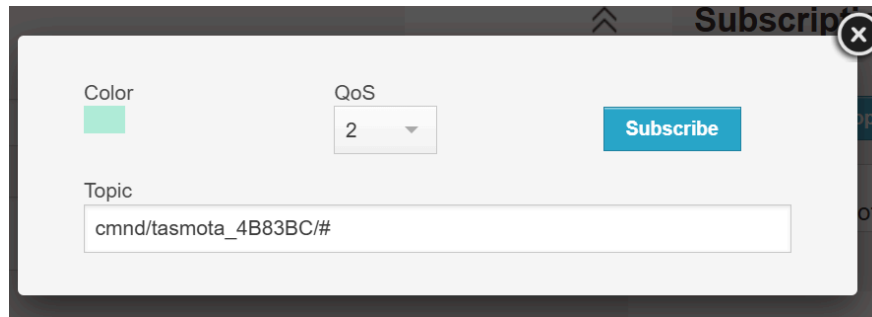
Message:

Subscriptions [Add New Topic Subscription](#)

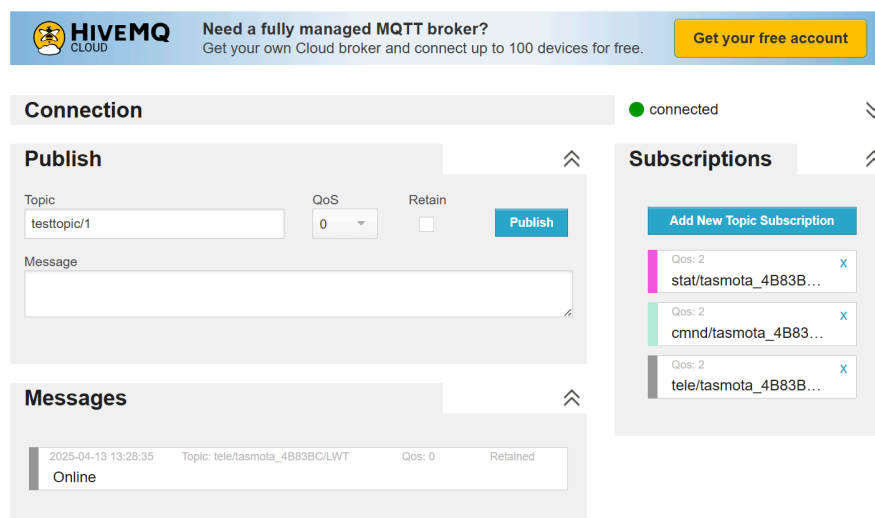
Messages

3. Đăng ký nhận tin thông qua các Topic:

- Để có thể nhận được các thông tin gửi qua MQTT, bạn cần khai báo đăng ký nhận thông qua các Topic mà Tasmota sử dụng.
- Nhấn vào nút Add New Topic Subscription và thêm vào lần lượt 3 topic sau:
 - cmnd/tasmota_4B83BC/#
 - stat/tasmota_4B83BC/#
 - tele/tasmota_4B83BC/#



- Chúng ta cần thêm ký tự đại diện (wild card) # sau mỗi topic để có thể nhận được toàn bộ các tin nhắn bắt đầu bằng 3 loại topic khai báo. Bạn cần sửa tên topic **tasmota_xxxxxx** cho đúng với thiết bị của bạn hoặc giá trị topic nếu bạn đã thay đổi khác với giá trị mặc định của Tasmota.
- Đây là giao diện sau khi bạn đã đăng ký các topic:



4. Thử gửi lệnh qua MQTT:

- Trong khu vực "Publish" của HiveMQ:
 - **Topic:** Nhập topic lệnh, ví dụ: **cmdn/tasmota_4B83BC/Power**
 - **Message:** Nhập nội dung lệnh, ví dụ: **ON** (dạng text).
 - Nhấn nút **"Publish"**.

5. Xác minh Kết quả:

- **Trên bo mạch Yolo Node:** Đèn LED sẽ được bật
- **Trên WebUI Tasmota:** Nút Toggle có chuyển sang trạng thái ON

- Trong phần **Message của HiveMQ**: sẽ nhận được các tin nhắn cập nhật trạng thái

Publish

Topic

cmnd/tasmota_4B83BC/Power

QoS

0

Retain

☐

Publish

Message

ON

Messages

2025-04-13 13:33:13	Topic: stat/tasmota_4B83BC/POWER	Qos: 0	ON
2025-04-13 13:33:13	Topic: stat/tasmota_4B83BC/RESULT	Qos: 0	{"POWER": "ON"}
2025-04-13 13:33:13	Topic: cmnd/tasmota_4B83BC/Power	Qos: 0	ON

- Thử gửi lệnh **OFF** và các lệnh khác để kiểm tra tương tự.
- Sau mỗi 300 giây, các thông tin trạng thái và giá trị cảm biến cũng được gửi vào topic **tele**.

```

2025-04-13 13:32:51 Topic: tele/tasmota_4B83BC/SENSOR Qos: 0
{"Time": "2025-04-13T07:32:56", "ANALOG": {"A1": 220}, "AHT2X":
{"Temperature": 32.2, "Humidity": 66.1, "DewPoint": 25.0}, "TempUnit": "C"}

2025-04-13 13:32:51 Topic: tele/tasmota_4B83BC/STATE Qos: 0
{"Time": "2025-04-13T07:32:56", "Uptime": "0T00:25:12", "UptimeSec": 1512, "Heap": 189, "Sleep
Mode": "Dynamic", "Sleep": 50, "LoadAvg": 19, "MqttCount": 1, "Berry":
{"HeapUsed": 4, "Objects": 37}, "POWER": "OFF", "Wifi": {"AP": 1, "SSId": "Góc
C3
Coffee", "BSSId": "74:83:C2:C1:29:8F", "Channel": 11, "Mode": "HT40", "RSSI":
100, "Signal": -50, "LinkCount": 1, "Downtime": "0T00:00:04"}}

```

Nếu bạn có thể thấy Tasmota gửi các dữ liệu trên và thực hiện lệnh bạn gửi xuống (**cmnd**) qua HiveMQ, thì xin chúc mừng, bạn đã thiết lập thành công giao tiếp MQTT cho thiết bị Tasmota của mình!

Kết thúc Module 8, bạn đã nắm vững một trong những phần quan trọng nhất để tích hợp Tasmota: giao thức MQTT. Bạn hiểu cách nó hoạt động, cách cài đặt Broker, cách cấu hình Tasmota và cách kiểm tra giao tiếp. Với nền tảng này, bạn đã sẵn sàng để thực hiện các thao tác điều khiển và đọc dữ liệu một cách có hệ thống hơn trong Module 9.

Module 9: Điều khiển thiết bị và đọc Cảm biến

Mục tiêu

Sau khi đã thiết lập kết nối MQTT và cấu hình phần cứng cơ bản, Module này tập trung vào cách bạn có thể thực sự **tương tác** với thiết bị Tasmota: gửi lệnh để điều khiển các bộ phận như Relay và đọc dữ liệu từ các cảm biến (Sensor) đã kết nối. Chúng ta sẽ xem xét các phương pháp khác nhau và đi sâu vào các lệnh cụ thể.

9.1 Điều khiển thiết bị Cơ bản qua WebUI, MQTT và HTTP

Tasmota cung cấp nhiều cách để bạn "ra lệnh" cho thiết bị:

1. WebUI (Giao diện Web):

- **Cách làm:** Truy cập địa chỉ IP của thiết bị trên trình duyệt, nhấn các nút điều khiển (như "Toggle") trên Main Menu.
- **Ưu điểm:** Trực quan, dễ sử dụng nhất, không cần cài đặt gì thêm. Phản hồi trạng thái ngay lập tức trên giao diện.
- **Nhược điểm:** Chỉ hoạt động khi bạn đang ở trong cùng mạng Wi-Fi với thiết bị. Không phù hợp cho tự động hóa phức tạp.
- **Khi nào dùng:** Điều khiển thủ công nhanh chóng, kiểm tra trạng thái, thực hiện cấu hình.

2. MQTT:

- **Cách làm:** Sử dụng một MQTT Client (như HiveMQ, MQTT Explorer, Home Assistant...) để publish (xuất bản) các tin nhắn lệnh đến các **cmnd** topic cụ thể của thiết bị trên MQTT Broker.

- **Ưu điểm:** Phương thức mạnh mẽ và linh hoạt nhất cho tự động hóa và tích hợp hệ thống. Hoạt động dựa trên mô hình publish/subscribe, tách biệt thiết bị và bộ điều khiển. Cho phép giao tiếp hai chiều (gửi lệnh và nhận trạng thái/dữ liệu). Hoạt động được cả trong mạng cục bộ và từ xa (nếu Broker được cấu hình đúng).
- **Nhược điểm:** Yêu cầu phải có một MQTT Broker đang chạy. Cần hiểu về cấu trúc topic và định dạng tin nhắn.
- **Khi nào dùng:** Tích hợp vào các nền tảng nhà thông minh, xây dựng kịch bản tự động hóa, điều khiển từ xa (qua nền tảng trung gian), nhận dữ liệu cảm biến liên tục.

3. Lệnh HTTP (HTTP Commands):

- **Cách làm:** Gửi yêu cầu HTTP GET trực tiếp đến địa chỉ IP của thiết bị Tasmota với lệnh được nhúng vào URL.
- **Cấu trúc URL cơ bản:** `http://<địa_chỉ_IP_Tasmota>/cm?cmd=<Lệnh Tasmota đã mã hóa URL>`
- **Ví dụ:** Để bật Relay 1 của thiết bị có IP `192.168.1.105`:
`http://192.168.1.105/cm?cmd=Power%20ON` (Lưu ý: %20 là mã hóa URL cho dấu cách)
- **Ví dụ yêu cầu trạng thái:** `http://192.168.1.105/cm?cmd=Status%20` (Kết quả sẽ trả về dưới dạng JSON trong phản hồi HTTP).
- **Xác thực:** Nếu bạn đã đặt mật khẩu cho Web Admin (**Configure Other**), bạn cần thêm thông tin user/password vào URL:
`http://<IP>/cm?user=<tên_user>&password=<mật_khẩu>&cmd=<Lệnh>`
- **Ưu điểm:** Đơn giản, không yêu cầu MQTT Broker. Dễ dàng gọi từ các script đơn giản (Bash, Python...), trình duyệt web, hoặc các hệ thống tự động hóa cũ hơn không hỗ trợ MQTT.
- **Nhược điểm:** Kém hiệu quả hơn MQTT (mỗi lệnh là một kết nối HTTP mới). Khó nhận trạng thái cập nhật tự động (phải chủ động hỏi). Bảo mật kém hơn nếu không dùng mật khẩu và HTTPS (Tasmota có hỗ trợ HTTPS nhưng cấu hình phức tạp hơn).

- **Khi nào dùng:** Tích hợp đơn giản, điều khiển nhanh từ bookmark trình duyệt, không có MQTT Broker.

9.2 Điều khiển Relay bằng Power command

Lệnh **Power** là lệnh cơ bản nhất để điều khiển các Relay (Relay) hoặc các thiết bị bật/tắt (như đèn) được cấu hình trong Tasmota.

- **Cú pháp và Tham số:**

- **Power** (hoặc **Power1**) không có tham số: Gửi lệnh này sẽ yêu cầu Tasmota trả về trạng thái hiện tại của Relay 1 (ví dụ: qua MQTT `stat/.../RESULT` sẽ là `{"POWER": "ON"}`).
- **Power ON** (hoặc **Power 1**): **Bật** Relay 1.
- **Power OFF** (hoặc **Power 0**): **Tắt** Relay 1.
- **Power TOGGLE** (hoặc **Power 2**): **Đảo ngược** trạng thái hiện tại của Relay 1 (Nếu đang Bật thì thành Tắt, và ngược lại).

- **Điều khiển nhiều Relay:** Nếu thiết bị của bạn có nhiều hơn một Relay (được cấu hình là Relay1, Relay2, Relay3...), bạn sử dụng chỉ số đi kèm lệnh **Power**:

- **Power2 ON**: Bật Relay 2.
- **Power3 OFF**: Tắt Relay 3.
- **Power4 TOGGLE**: Đảo trạng thái Relay 4.
- ...
- **Power0** (hoặc đôi khi chỉ **Power**): Có thể dùng để điều khiển **tất cả** các relay cùng lúc (hành vi cụ thể có thể thay đổi tùy phiên bản, nên thử nghiệm).

- **Cách gửi Lệnh:**

- **WebUI:** Nhấn các nút "Toggle", "Toggle 2"... tương ứng.

- **Console:** Gõ lệnh trực tiếp, ví dụ: `Power ON`, `Power2 OFF`, `Power3 TOGGLE`.
- **MQTT:** Publish payload (`ON`, `OFF`, hoặc `TOGGLE`) đến các topic lệnh tương ứng:
 - `cmnd/%topic%/Power` (cho Relay1)
 - `cmnd/%topic%/Power2` (cho Relay2)
 - ...
- **HTTP:** Sử dụng URL đã mã hóa:
 - `http://<IP>/cm?cmnd=Power%20ON`
 - `http://<IP>/cm?cmnd=Power2%20OFF`
 - ...
- **Nhận phản hồi trạng thái (qua MQTT):**
 - Khi trạng thái của một Relay thay đổi (do lệnh MQTT, HTTP, WebUI, hoặc nút nhấn vật lý), Tasmota sẽ **publish** trạng thái mới lên MQTT Broker:
 - Topic `stat/%topic%/RESULT`: Message nhận được có dạng JSON, ví dụ `{"POWER1": "ON"}`.
 - Topic `stat/%topic%/POWER<x>`: Message nhận được là trạng thái hiện tại, ví dụ `ON` hoặc `OFF`.
 - Trạng thái cũng sẽ được bao gồm trong gói tin **Telemetry** định kỳ (`tele/%topic%/STATE`).

9.3 Đọc dữ liệu cảm biến

Nếu bạn đã kết nối và cấu hình cảm biến (nhiệt độ, độ ẩm, ánh sáng...) cho thiết bị Tasmota (ở Module 7), có hai cách chính để lấy dữ liệu từ chúng:

1. Tự động qua Telemetry (gửi định kỳ):

- **Cách hoạt động:** Tasmota sẽ tự động đọc giá trị từ tất cả các cảm biến đã được cấu hình và gửi chúng đi qua MQTT theo một chu kỳ thời gian nhất định.
- **Topic MQTT:** `tele/%topic%/SENSOR`

- **Chu kỳ gửi (Tần suất):** Được xác định bởi cài đặt **TelePeriod** (truy cập qua **Configuration -> Logging -> Telemetry Period**, hoặc dùng lệnh **TelePeriod** trên Console). Giá trị mặc định là **300** giây (5 phút). Bạn có thể thay đổi nó, ví dụ: **TelePeriod 60** để gửi mỗi phút.
- **Nội dung tin nhắn (Payload):** Là một chuỗi **JSON** chứa dữ liệu từ các cảm biến. Cấu trúc JSON có thể thay đổi tùy thuộc vào loại và số lượng cảm biến.

Ví dụ (khi có cảm biến AHT20):

```
{
  "Time": "2025-04-12T10:55:00",
  "AHT2X": {"Temperature": 29.5, "Humidity": 71.8},
  "TempUnit": "C"
}
```

- **Ưu điểm:** Tự động, không cần làm gì thêm. Phù hợp để theo dõi dữ liệu theo thời gian.
- **Nhược điểm:** Có độ trễ giữa các lần cập nhật (bằng **TelePeriod**).

2. Yêu cầu Thủ công (Manual Status Request):

- **Cách hoạt động:** Bạn chủ động gửi một lệnh đến Tasmota để yêu cầu nó đọc và trả về dữ liệu cảm biến ngay lập tức.
- **Lệnh sử dụng:** **Status 8**
- **Cách gửi lệnh:**
 - **Console:** Gõ **Status 8** rồi Enter.
 - **MQTT:** Publish một tin nhắn **trống** (empty payload) đến topic **cmnd/%topic%/Status8**.
 - **HTTP:** Truy cập URL **http://<IP>/cm?cmnd=Status%208**.
- **Cách nhận phản hồi:**
 - **Console:** Kết quả JSON sẽ hiển thị trực tiếp trên Console.
 - **MQTT:** Tasmota sẽ publish kết quả JSON đến topic **stat/%topic%/STATUS8**. Nội dung JSON này tương tự như payload của **tele/.../SENSOR**.

- **HTTP:** Kết quả JSON sẽ được trả về trong nội dung phản hồi của yêu cầu HTTP.
 - **Ưu điểm:** Lấy được dữ liệu mới nhất ngay khi cần.
 - **Nhược điểm:** Phải chủ động gửi lệnh mỗi khi muốn cập nhật.
- **Xem trên WebUI:** Dữ liệu cảm biến được cập nhật gần nhất (thường là từ lần gửi Telemetry cuối cùng) cũng được hiển thị trực tiếp trên **Main Menu** của WebUI.
- **Đơn vị Nhiệt độ:**
 - Mặc định Tasmota hiển thị và gửi dữ liệu nhiệt độ theo độ C (Celsius).
 - Để đổi sang độ F (Fahrenheit), dùng lệnh Console: `SetOption8 1`.
 - Để quay lại độ C: `SetOption8 0`.

9.4 Làm việc với Nút nhấn và Công tắc

Cách mà nút nhấn hoặc công tắc vật lý được kết nối với Tasmota hoạt động có thể được tùy chỉnh rất linh hoạt bằng lệnh `SwitchMode`.

- **Hành vi mặc định:**
 - Khi bạn cấu hình một chân GPIO là `Button1` (hoặc `Button<x>`), Tasmota thường tự động liên kết nó với `Relay1` (hoặc `Relay<x>`) tương ứng. Mỗi lần nhấn nút (`Button`) sẽ **đảo trạng thái (TOGGLE)** của Relay đó.
 - Khi bạn cấu hình là `Switch1` (hoặc `Switch<x>`), Tasmota thường liên kết nó với `Relay1` (hoặc `Relay<x>`) và Relay sẽ **theo trạng thái (FOLLOW)** của công tắc.
- **Lệnh `SwitchMode<x>`:**
 - Dùng để thay đổi hành vi của Nút nhấn/Công tắc thứ `x` (ví dụ: `SwitchMode1` cho `Button1/Switch1`, `SwitchMode2` cho `Button2/Switch2...`).

- Gửi lệnh qua Console, ví dụ: `SwitchMode1 7`.
- **Các Giá trị `SwitchMode` Thông dụng:**
 - **0 (TOGGLE):** Nhấn nút -> Đảo trạng thái Relay. (Mặc định cho Button)
 - **1 (FOLLOW):** Trạng thái Relay = Trạng thái Công tắc. (Mặc định cho Switch)
 - **2 (INV FOLLOW):** Trạng thái Relay = Ngược trạng thái Công tắc.
 - **3 (PUSHBUTTON):** Relay chỉ BẬT khi nút được nhấn và giữ. Nhả ra -> Relay TẮT.
 - **4 (INV PUSHBUTTON):** Relay chỉ TẮT khi nút được nhấn và giữ. Nhả ra -> Relay BẬT.
 - **7 (PUSHBUTTON_TOGGLE MULTIPRESS - Rất hữu ích):** Tách rời hoàn toàn nút nhấn khỏi Relay. Khi nhấn nút, Tasmota sẽ **chỉ gửi thông tin về hành động nhấn** (ví dụ: nhấn đơn - SINGLE, nhấn đúp - DOUBLE, nhấn giữ - HOLD) qua MQTT đến topic `stat/%topic%/RESULT` (dạng JSON: `{"Button1":{"Action":"SINGLE"}}`). Tín hiệu này **KHÔNG** trực tiếp điều khiển Relay nữa. Điều này cho phép bạn:
 - Xử lý logic phức tạp hơn trong hệ thống. Ví dụ: Nhấn đơn bật đèn A, nhấn đúp bật đèn B, nhấn giữ điều khiển kịch bản khác.
 - Sử dụng nút nhấn vật lý để điều khiển các thiết bị khác trong mạng mà không cần nối dây trực tiếp.
- **Tách rời Công tắc (Switch) khỏi Relay:**
 - Nếu bạn muốn công tắc vật lý chỉ gửi trạng thái ON/OFF qua MQTT mà không trực tiếp điều khiển Relay, bạn cần kết hợp `SwitchMode` (thường là 1 hoặc 2) với lệnh `SwitchTopic`.
 - Dùng lệnh `SwitchTopic 0` để ngăn Tasmota tự động gửi lệnh điều khiển dựa trên trạng thái công tắc.
 - Sau đó, bạn phải dùng **Rules** (sẽ học ở Module 10) để bắt sự kiện thay đổi `Switch<x>#State` và quyết định hành động tiếp theo (ví dụ: gửi tin nhắn MQTT tùy chỉnh).

- **Gửi trạng thái Nút/Công tắc qua MQTT:**

- Sử dụng **SwitchMode 7** sẽ tự động gửi hành động nhấn qua **stat/.../RESULT**.
- Để trạng thái của Công tắc (**Switch<x>**) được gửi đi với cờ **Retain=true** (hữu ích cho Home Assistant biết trạng thái công tắc vật lý), dùng lệnh **SwitchRetain 1**. Trạng thái sẽ được gửi đến topic **stat/%topic%/SWITCH<x>** (payload là **{"STATE": "ON"}** hoặc **{"STATE": "OFF"}**).

Việc làm chủ **SwitchMode** cho phép bạn tạo ra các tương tác vật lý thông minh và linh hoạt hơn nhiều so với hành vi bật/tắt đơn thuần.

Kết thúc Module 9, bạn đã biết cách thực hành điều khiển Relay, đọc dữ liệu cảm biến bằng nhiều phương pháp (WebUI, MQTT, HTTP), và quan trọng là tùy chỉnh được hành vi của các nút nhấn/công tắc vật lý để phù hợp hơn với nhu cầu tự động hóa nhà thông minh. Module tiếp theo, Module 10, sẽ giới thiệu về Tasmota Rules, cho phép bạn tạo ra các tự động hóa đơn giản chạy trực tiếp ngay trên chính thiết bị Tasmota.

Module 10: Tự động hóa với Tasmota Rules

Mục tiêu

Sau khi đã điều khiển và đọc dữ liệu từ thiết bị Tasmota qua WebUI, MQTT và HTTP, Module này sẽ giới thiệu một tính năng cực kỳ mạnh mẽ khác: **Tasmota Rules**. Rules cho phép bạn tạo ra các kịch bản tự động hóa đơn giản chạy **ngay trên chính thiết bị ESP**, giúp nó hoạt động thông minh hơn mà không cần đến sự can thiệp liên tục từ bên ngoài.

10.1 Giới thiệu về Tasmota Rules

- **Tasmota Rules là gì?** Hiểu đơn giản, Rules là một hệ thống cho phép bạn định nghĩa các quy tắc dạng "NEU [sự kiện này xảy ra] THÌ [thực hiện hành động này]". Điểm đặc biệt là các quy tắc này được lưu trữ và thực thi **hoàn toàn trên bộ nhớ của chip ESP**, độc lập với máy chủ MQTT hay hệ thống nhà thông minh trung tâm.
- **Tại sao nên sử dụng Rules?**
 - **Tự động hóa cục bộ & ổn định:** Tạo ra các phản ứng tức thời giữa các thành phần trên cùng một thiết bị (ví dụ: nhấn nút vật lý bật/tắt ngay lập tức relay được nối cùng bo mạch). Các tự động hóa này **vẫn hoạt động bình thường ngay cả khi mất kết nối Wi-Fi hoặc MQTT Broker**.
 - **Giảm độ trễ:** Xử lý logic ngay tại chỗ giúp giảm độ trễ so với việc gửi tín hiệu lên trung tâm rồi chờ lệnh gửi về.
 - **Giảm tải cho hệ thống trung tâm:** Các tác vụ đơn giản (như liên kết nút bấm với relay) có thể được xử lý bởi Rules, không cần làm phiền đến server xử lý.
 - **Tùy biến nâng cao:** Cho phép tạo ra các hành vi phức tạp hơn mà cấu hình cơ bản không đáp ứng được (ví dụ: tạo hiệu ứng nhấp nháy đèn, xử lý nhiều kiểu nhấn nút...).

- **Hoạt động độc lập:** Thiết bị có thể tự thực hiện các tác vụ cơ bản mà không cần kết nối mạng (ví dụ: cảm biến nhiệt độ điều khiển quạt làm mát được nối cùng thiết bị).
- **Giới hạn của Rules:**
 - Rules được thiết kế cho các logic **tương đối đơn giản**. Các kịch bản quá phức tạp, liên quan đến nhiều thiết bị khác nhau hoặc yêu cầu lưu trữ trạng thái phức tạp nên được thực hiện ở hệ thống điều khiển trung tâm.
 - Bộ nhớ dành cho việc lưu trữ Rules trên chip ESP là **có hạn** (đặc biệt trên ESP8266).

10.2 Cú pháp Rule

Việc tạo và quản lý Rules được thực hiện hoàn toàn thông qua giao diện **Console** của WebUI.

Cấu trúc cơ bản

Tasmota hỗ trợ tối đa 3 bộ Rule độc lập: **Rule1**, **Rule2**, **Rule3**. Mỗi Rule được định nghĩa bằng lệnh **Rule<x> <Nội dung Rule>**. Cú pháp chung của nội dung Rule bao gồm các khối **ON...ENDON**:

```
ON <Trigger> DO <Command> ENDON
```

Hoặc có thể phức tạp hơn với điều kiện:

```
ON <Trigger> IF <Condition> DO <Command1> ELSE DO <Command2> ENDIF ENDON
```

Một **Rule<x>** có thể chứa nhiều khối **ON...ENDON** khác nhau.

Giải thích các thành phần

- **Rule<x>**: Lệnh dùng để bắt đầu định nghĩa hoặc xem nội dung của Rule thứ x (1, 2, hoặc 3).

- **ON <Trigger>**: Phần quan trọng nhất, xác định **Sự kiện (Event)** nào sẽ **kích hoạt (trigger)** khối lệnh này. Trigger thường là sự thay đổi trạng thái của một thành phần phần cứng hoặc phần mềm trên Tasmota.
 - **Cú pháp Trigger**: <ComponentName>#<StateName> hoặc <EventName>
 - **Ví dụ các Trigger phổ biến**:
 - **Button1#State**: Trạng thái Nút nhấn 1 thay đổi.
 - **Switch1#State**: Trạng thái Công tắc 1 thay đổi (ON/OFF).
 - **AHT2X#Temperature**: Giá trị nhiệt độ từ cảm biến AHT20 thay đổi.
 - **Analog#A0**: Giá trị đọc từ chân Analog A0 thay đổi.
 - **Power1#State**: Trạng thái Relay 1 thay đổi.
 - **System#Boot**: Thiết bị vừa khởi động xong.
 - **Time#Minute**: Kích hoạt mỗi phút một lần (khi đồng hồ thay đổi phút).
 - **Time#Initialized**: Khi thời gian hệ thống được đồng bộ lần đầu (qua NTP hoặc lệnh **Time**).
 - **Mqtt#Connected**: Khi kết nối đến MQTT Broker thành công.
 - **Rules#Timer=1**: Khi bộ đếm thời gian RuleTimer1 kết thúc.
 - **Event#MyCustomEvent**: Kích hoạt bởi một sự kiện tùy chỉnh do bạn tạo ra.
 - Trigger cũng có thể đi kèm **điều kiện giá trị**: **ON AHT2X#Temperature > 30 DO...** (Chỉ kích hoạt khi nhiệt độ > 30).
- **IF <Condition> ... ENDIF (Tùy chọn)**: Cho phép bạn kiểm tra một **điều kiện logic** trước khi thực hiện lệnh. Chỉ khi điều kiện là đúng (true) thì lệnh trong **DO** mới được thực thi.
 - **Condition**: Thường là phép so sánh giá trị của một biến với một giá trị khác.
 - **Toán tử so sánh**: **==** (bằng), **!=** (không bằng), **>** (lớn hơn), **<** (nhỏ hơn), **>=** (lớn hơn hoặc bằng), **<=** (nhỏ hơn hoặc bằng).
 - **Biến (Variables)**:
 - **%value%**: Chứa giá trị của sự kiện vừa kích hoạt Trigger (ví dụ: giá trị nhiệt độ, trạng thái ON/OFF là 1/0).
 - **%mem<x>%** (x từ 1 đến 5): Các biến tạm thời, chỉ tồn tại trong một lần Rule được kích hoạt và thực thi. Dùng để lưu trữ giá trị trung gian.

- **Var<x>** (x từ 1 đến 16): Các biến lưu trữ giá trị lâu dài hơn (giữ nguyên giữa các lần kích hoạt Rule, nhưng sẽ mất khi thiết bị khởi động lại). Dùng lệnh **Var<x> <giá_trị>** để gán.
 - Các biến hệ thống khác: **%topic%**, **%ipaddress%**, **Wifi#Status**, **Mqtt#Status...**
- **Ví dụ Condition:** **IF %value% < 25, IF %mem1% == 1, IF Mqtt#Status == 1.**
- **ELSEDO <Command> (Tùy chọn):** Đi kèm với **IF**, lệnh này sẽ được thực thi nếu điều kiện trong **IF** là sai (false).
- **DO <Command>: Hành động** chính cần thực hiện khi Trigger xảy ra (và thỏa mãn điều kiện IF, nếu có).
 - **Command:** Có thể là **bất kỳ lệnh Tasmota hợp lệ** nào bạn có thể gõ trên Console.
 - Có thể thực thi **nhiều lệnh** bằng cách dùng lệnh **Backlog** bên trong Rule. Ví dụ: **DO Backlog Power1 ON; Delay 50; Power1 OFF ENDON**. Dấu **;** dùng để ngăn cách các lệnh trong Backlog.
- **ENDON:** Đánh dấu kết thúc một khối xử lý sự kiện **ON...DO...**

Các lệnh để quản lý Rule trên Console

Bạn có thể cấu hình các Rule bằng cách vào WebUI -> **Tools** -> **Console** và gõ lệnh khai báo trực tiếp.

- **Định nghĩa Rule:** **Rule<x> <Nội dung Rule>**

Ví dụ: **Rule1 ON Button1#State DO Power1 TOGGLE ENDON**

- **Kích hoạt Rule:** **Rule<x> 1**

Ví dụ: **Rule1 1**

Các Rule chỉ chạy khi được kích hoạt.

- **Tắt Rule:** **Rule<x> 0**

Ví dụ: Rule1 0

- **Xem nội dung Rule:** Rule<x> (không có tham số)
- **Xóa nội dung Rule:** Rule<x> "" (Gán một chuỗi rỗng)
- **Bật/tắt chế độ One-Shot:** Rule<x> 5 (Rule chỉ chạy một lần sau khi trigger, rồi tự tắt).

10.3 Ví dụ một số Rule trong thực tế

Dưới đây là một số ví dụ cụ thể để bạn hình dung rõ hơn:

Ví dụ 1: Nút nhấn 1 (Button1) luôn đảo trạng thái Relay 1 (Relay1) (Đây là hành vi mặc định nếu SwitchMode=0, nhưng ví dụ cách viết bằng Rule)

```
Rule1 ON Button1#State DO Power1 TOGGLE ENDON
Rule1 1
```

Ví dụ 2: Nếu nhiệt độ từ cảm biến AHT20 lớn hơn 32 độ C thì bật Relay 1, ngược lại thì tắt Relay 1.

```
Rule1
  ON AHT2X#Temperature > 32 DO Power1 1 ENDON
  ON AHT2X#Temperature < 32 DO Power1 0 ENDON
Rule1 1
```

Ví dụ 3: Tách nút nhấn 1 khỏi Relay 1. Nhấn đơn (Single) thì bật Relay 1, nhấn kép (Double) thì bật Relay 2.

Để chạy được bạn cần chạy lệnh SwitchMode1 7 trước và cũng cần cấu hình thêm 1 Relay để có thể thử nghiệm. Ở đây chúng ta tạm dùng GPIO0 để điều khiển Relay 2.

ESP32C3
Tasmota

Module parameters

Module type (ESP32C3)
ESP32C3 (1) ▼

AO GPIO0	Relay ▼	2 ▼
AO GPIO1	None ▼	
AO GPIO2	ADC Input ▼	1 ▼
AO GPIO3	None ▼	
AO GPIO4	None ▼	

```
Rule1
  ON Button1#State=10 DO Power1 TOGGLE ENDON
  ON Button1#State=11 DO Delay 1; Power2 TOGGLE ENDON
Rule1 1
```

Lưu ý: Các mã State cho Button khi SwitchMode=7: 10=SINGLE, 11=DOUBLE, 12=HOLD, 13=RELEASE (sau HOLD)...

Chúng ta bổ sung thêm câu lệnh Delay 1 để tránh Tasmota nhận liên tục 2 lần sự kiện khi nhấn đúp nút.

10.4 Gỡ lỗi Rule qua giao diện Console

Việc viết Rule không phải lúc nào cũng đúng ngay lần đầu. Console là trợ thủ đắc lực để bạn tìm và sửa lỗi:

- **Kiểm tra Lỗi Cú pháp:** Khi bạn nhập lệnh Rule<x> <Nội dung Rule>, Tasmota sẽ phân tích cú pháp. Nếu sai, nó sẽ báo lỗi ngay trên Console, thường chỉ ra vị trí gần chỗ sai (Syntax Error near...). Hãy đọc kỹ thông báo lỗi.
- **Xem Log Thực thi Rule:** Để xem chi tiết Rule đang chạy như thế nào, hãy tăng mức độ log trên Console:
 - Gõ lệnh WebLog 3 (Debug) hoặc WebLog 4 (Debug More).

- Bây giờ, khi một sự kiện kích hoạt Rule, bạn sẽ thấy các dòng log bắt đầu bằng **RUL**: cho biết:
 - Rule nào được kích hoạt (**RULE 1**).
 - Trigger nào gây ra (**trig**).
 - Điều kiện IF được đánh giá là đúng hay sai (**cond true / cond false**).
 - Lệnh nào đang được thực thi (**cmd ...**).
 - Giá trị của các biến như **%value%**.
- Quan sát log này giúp bạn hiểu tại sao Rule không chạy như mong đợi.
- **Kiểm tra Giá trị Biến:** Dùng các lệnh trên Console như **Var1**, **Mem1**, **Status 0**, **Status 8...** để kiểm tra giá trị hiện tại của các biến mà Rule của bạn sử dụng trong điều kiện IF hoặc lệnh DO.
- **Kiểm tra Trigger:** Hãy chắc chắn rằng sự kiện bạn dùng làm Trigger thực sự xảy ra.
 - Với **Button#State** hoặc **Switch#State**, hãy nhấn nút/gạt công tắc và xem log có ghi nhận không.
 - Với **Sensor#Value**, hãy dùng **Status 8** để xem cảm biến có trả về giá trị không.
 - Với **Time#Minute**, hãy đợi qua phút mới xem Rule có chạy không.
- **Thử nghiệm Từng Phần Nhỏ:** Nếu Rule của bạn dài và phức tạp, hãy chia nhỏ nó ra, viết và thử nghiệm từng khối **ON...ENDON** hoặc từng lệnh đơn giản trong **DO** trước khi kết hợp chúng lại.
- **Sử dụng RuleTimer để Debug:** Đôi khi bạn muốn xem giá trị một biến sau một hành động nào đó. Bạn có thể dùng **RuleTimer** để tạo độ trễ rồi dùng lệnh **Publish** để gửi giá trị biến đó qua MQTT hoặc hiển thị trên Console.

Thực hành nhiều và tham khảo tài liệu Tasmota Rules chính thức [ở đây](#) là cách tốt nhất để bạn làm chủ tính năng này.

Kết thúc Module 10, bạn đã khám phá sức mạnh của Tasmota Rules, cho phép tạo ra các tự động hóa thông minh chạy độc lập ngay trên thiết bị. Với kiến thức này, thiết bị Tasmota của bạn không chỉ là một thiết bị được điều khiển từ xa, mà còn có thể tự động phản ứng với môi trường xung quanh. Module tiếp theo, Module 11, sẽ quay lại việc kết nối Tasmota với thế giới bên ngoài, tập trung vào cách tích hợp nó vào các nền tảng IoT phổ biến.

Module 11: Tích hợp Tasmota với Core IoT

Mục tiêu

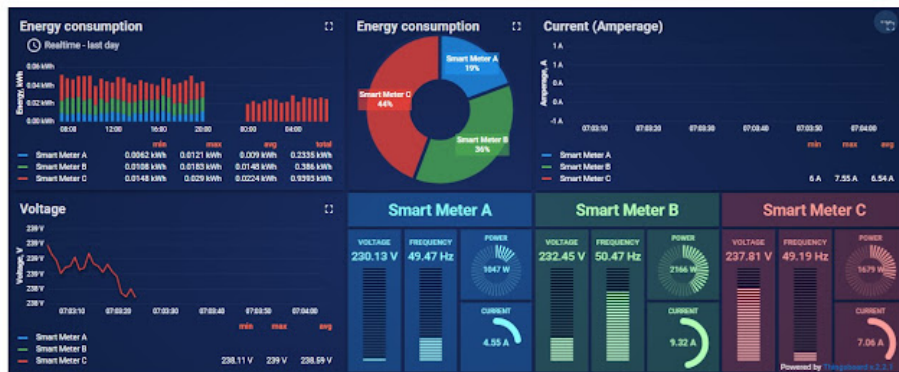
Sau khi đã làm chủ việc cấu hình và điều khiển thiết bị Tasmota độc lập, bước tiếp theo là đưa nó vào một "bộ não" trung tâm để quản lý và phối hợp hoạt động với các thiết bị thông minh khác trong nhà bạn. Module này sẽ tập trung vào cách tích hợp Tasmota với các nền tảng IoT phổ biến, tiêu biểu là Core IoT.

11.1 Giới thiệu về Core IoT

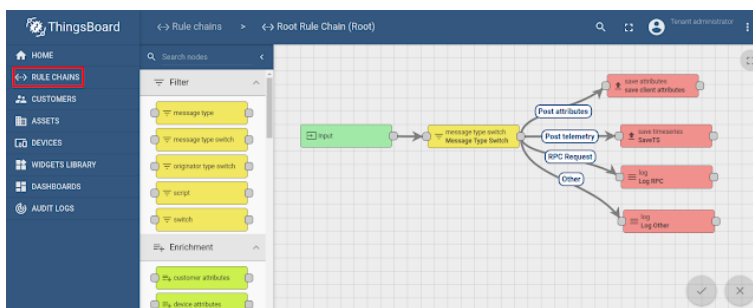
Core IoT là một nền tảng IoT mã nguồn mở, được phát triển dựa trên nền tảng Thingsboard nhưng được tùy chỉnh và triển khai miễn phí không giới hạn, phù hợp cho giáo dục và nghiên cứu cũng như nhu cầu làm các hệ thống thử nghiệm (prototyping). Nó cho phép phát triển nhanh chóng, quản lý và mở rộng các dự án IoT. Với nền tảng Core IoT, bạn có thể thu thập, xử lý, hiển thị trực quan và quản lý thiết bị.

Core IoT cho phép kết nối thiết bị thông qua các giao thức IoT tiêu chuẩn công nghiệp trong đó có MQTT, một giao thức phổ biến để thu thập dữ liệu và điều khiển thiết bị thông qua mạng Internet.

Một trong những ưu điểm của Core IoT, là nó hỗ trợ nhiều dịch vụ tiện ích mạnh mẽ, theo khuynh hướng của một ứng dụng công nghiệp. Chẳng hạn như màn hình điều hành của Core IoT, hỗ trợ rất nhiều giao diện phức tạp và hiện đại, như ví dụ sau đây:

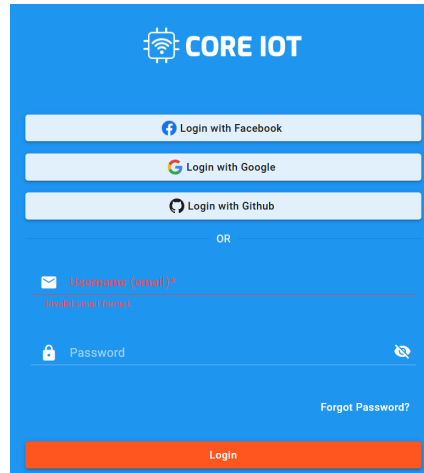


Một dịch vụ khác, khiến người dùng chọn Core IoT, là khả năng xây dựng một chuỗi các tương tác để hệ thống có thể vận hành tự động. Khi một điều kiện xảy ra, nó sẽ tự động kích hoạt các tác vụ tiếp theo. Khái niệm này khá tương đồng với IFTTT (IF THIS THEN THAT). Trên Core IoT, dịch vụ này gọi là Rule Chains, như minh họa ở hình sau đây:

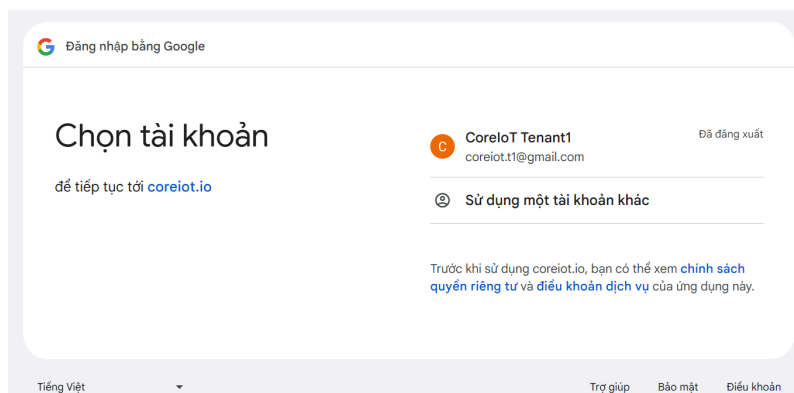


11.2 Thao tác cơ bản với Core IoT Platform

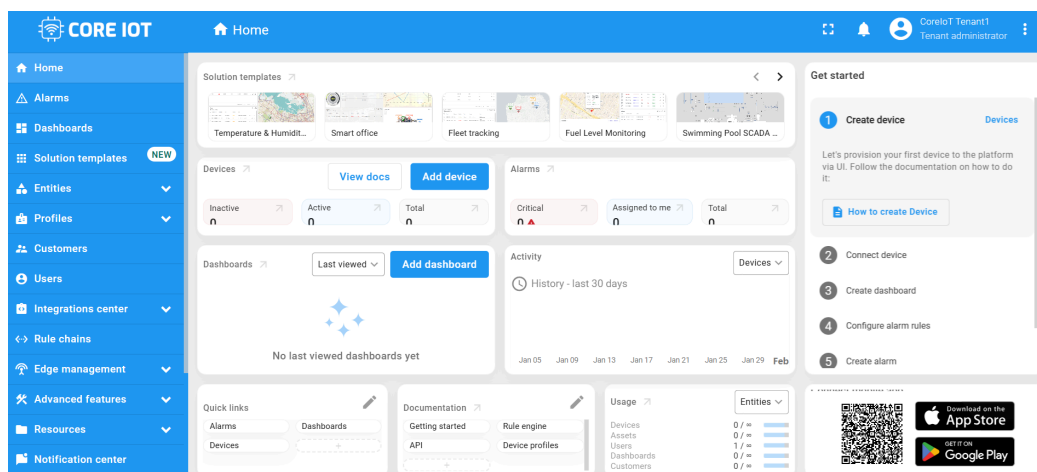
Bạn vào link <https://app.coreiot.io/> và chọn Login bằng Google hoặc Facebook hoặc Github để đăng ký một tài khoản mới.



Ví dụ nếu bạn sử dụng tài khoản Google để login thì bạn cần xác nhận tài khoản để sử dụng ở màn hình tiếp theo sau đó:

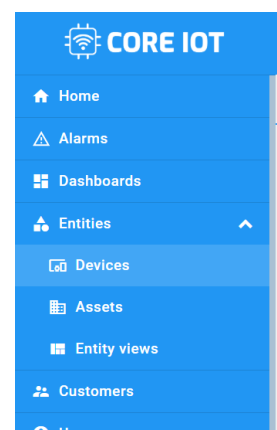


Tương tự cho Facebook và Github, sau khi xác nhận chọn tài khoản sử dụng xong, một tài khoản mới sẽ được tạo trên Core IoT và sau bước này bạn đã có thể đăng nhập vào hệ thống với tài khoản của mình. Giao diện sau khi login sẽ như sau:

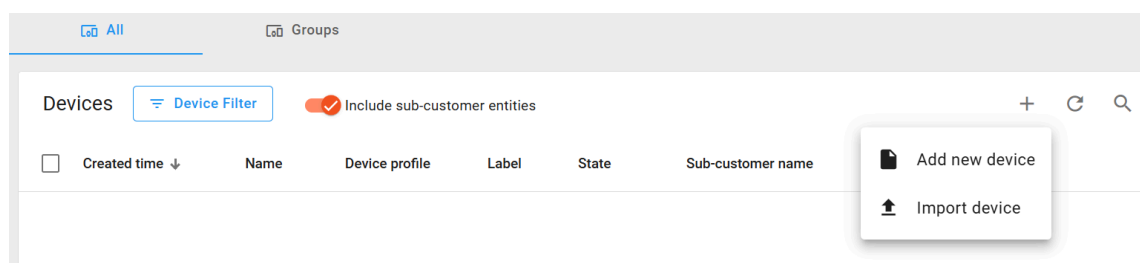


Tạo Device trên Core IoT

Device là một tài nguyên sẽ được tạo ra trên Core IoT để lưu trữ các thông tin. Thông thường, chúng ta sẽ dùng dịch vụ Device này để lưu trữ thông tin cảm biến từ thiết bị. Từ danh sách menu bên trái, chọn vào **Entities** và chọn tiếp vào **Devices**.



Tiếp theo, từ kí hiệu dấu cộng (+) ở góc bên phải, nhấn vào đó và chọn tiếp vào **Add new Device**



Điền các thông tin tương ứng, như minh họa ở hình bên dưới. Bạn có thể đặt tên thiết bị tùy ý.

Add new device

1 Device details — 2 Credentials Optional

Name*
Tasmota_Device1

Label
Test device used for document

Device profile*
default

☐ Is gateway

Owner and groups

Next: Credentials

Cancel Add

Nhấn tiếp vào nút **Next Credentials**, để đến giao diện sau đây:

Add new device

✓ Device details — 2 Credentials Optional

Credentials type

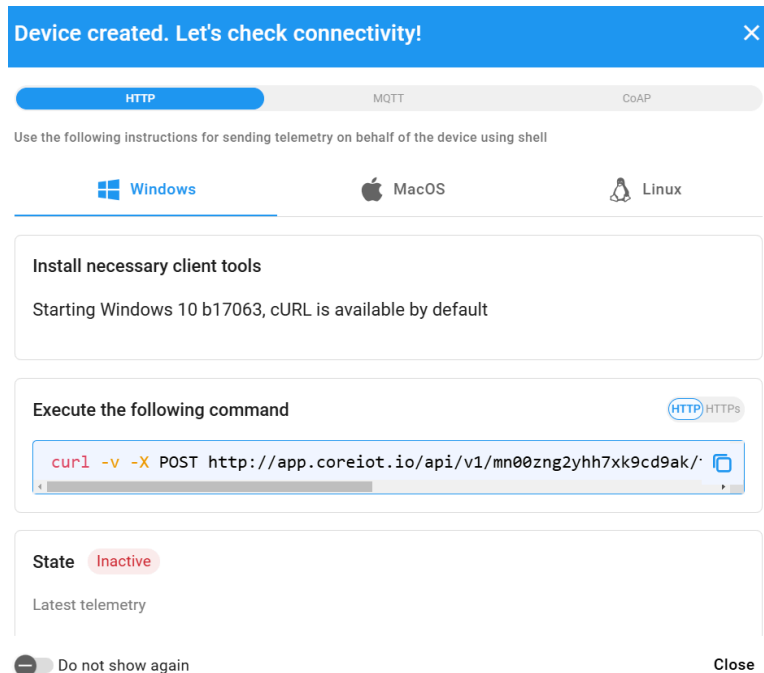
Access token X.509 MQTT Basic

Access token*
s2h...fa5ois07

Back

Cancel Add

Lưu ra file Nofilead thông tin của token, bạn sẽ cần nó để gửi dữ liệu lên server Core IoT ở phần tiếp theo. Nhấn nút **Add** để kết thúc quá trình này.



Nhấn **Close** để kết thúc việc tạo một Device trên Core IoT.

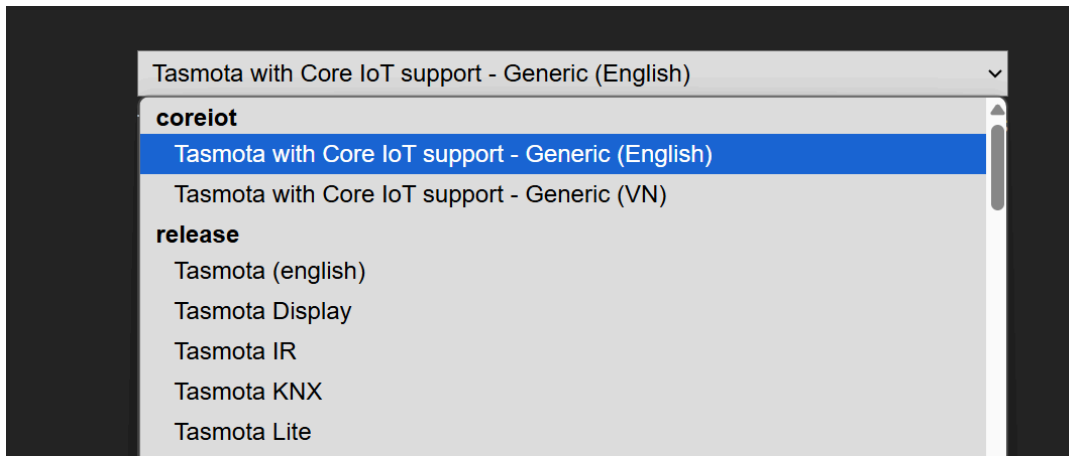
11.3 Flash Firmware Tasmota hỗ trợ Core IoT

Phiên bản firmware Tasmota gốc chưa hỗ trợ kết nối đến Core IoT vì các nền tảng IoT cao cấp như Core IoT đòi hỏi các quy tắc gửi nhận thông tin qua topic MQTT theo các cách chặt chẽ hơn như cách thức mặc định mà Tasmota làm việc.

Bạn sẽ cần flash phiên bản Tasmota có hỗ trợ Core IoT ở URL này:

<https://ohstem-public.github.io/tasmota-install/>

Cách thức flash firmware tương tự như bạn đã từng làm trong hướng dẫn của Module 5. Bạn cần chọn đúng phiên bản có hỗ trợ Core IoT như trong hình dưới đây.



11.4 Thiết lập MQTT kết nối với Core IoT

Như đã học ở Module 8, MQTT là cầu nối giao tiếp chính giữa Tasmota và các hệ thống IoT như Core IoT. Để chúng "nói chuyện" được với nhau, bạn cần cấu hình lại MQTT cho đúng với server của Core IoT.

Sau khi flash lại Tasmota phiên bản hỗ trợ Core IoT, bạn cần tiến hành cấu hình lại mọi thứ từ Wi-Fi cho đến các GPIO như trong các bài trước.

Và tiếp theo bạn cần cấu hình MQTT lại như sau:

MQTT parameters

Host ()
app.coreiot.io

Port (1883)
1883

☒ **MQTT TLS**

Client (DVES_4B83BC)
DVES_%06X

User (DVES_USER)
sgbw...f9a1vr|

Password ☒
....

Topic = %topic% (tasmota_4B83BC)
tasmota_%06X

Full Topic (%prefix%/%topic%/)
%prefix%/%topic%/

Save

- **Broker:** Nhập địa chỉ MQTT Broker của Core IoT là app.coreiot.io.
- **Port:** Nhập cổng MQTT [1883](#).
- **Username:** Nhập Access token của thiết bị đã tạo trên Core IoT ở phần 11.2.
- **Password:** Để giá trị mặc định hoặc để trống vì Core IoT không dùng đến.
- *Các tùy chọn khác có thể để mặc định.*
- Nhấn **SAVE**. Tasmota sẽ lưu và reset thiết bị.

Nếu thông tin MQTT đúng và kết nối thành công, bạn sẽ thấy thông báo kết nối thành công trong cửa sổ Logs của Tasmota. Bạn sẽ thấy xuất hiện các topic sẽ theo chuẩn của Core IoT như [v1/devices/me/attributes](#) thay vì topic mặc định của Tasmota.

Logs

```
00:00:01.001 WiFi: Connecting to AP: GOC C3 Coffee Channel: 11 BSSID: 74:83:C2:C1:29:8F in mode HT40-40+ TasMota-4883BC-0956...
00:00:03.387 WiFi: Connected
00:00:03.593 HTTP: Web server active on tasmota-4883BC-0956 with IP address 192.168.1.55
16:03:11.564 MQTT: Attempting connection...
16:03:11.787 MQTT: Connected
16:03:11.788 MQTT: tele/tasmota_4883BC/LWT = Online (retained)
16:03:11.789 MQTT: cmdnd/tasmota_4883BC/POWER =
16:03:11.798 MQTT: v1/devices/me/attributes = {"Info1":
{"Module": "ESP32C3", "Version": "14.5.0.1(tasmota32)", "FallbackTopic": "cmdnd/DVES_4883BC_fb/", "GroupTopic": "cmdnd/tasmtotas/"}}
16:03:11.802 MQTT: v1/devices/me/attributes = {"Info2": {"WebServerMode": "Admin", "Hostname": "tasmota-4883BC-
0956", "IPAddress": "192.168.1.55", "IP6Global": "", "IP6Local": "fe80::2a37:2fff:fe4b:83bc%st1"}}
16:03:11.805 MQTT: v1/devices/me/attributes = {"Info3": {"RestartReason": "Software reset CPU", "BootCount": 6}}
16:03:11.808 MQTT: v1/devices/me/attributes = {"POWER": "OFF"}
16:03:11.810 MQTT: stat/tasmota_4883BC/POWER = OFF
16:03:13.993 QPC: Reset
16:03:16.001 MQTT: v1/devices/me/attributes = {"Time": "2025-04-
13T16:03:15", "Uptime": "0T00:00:09", "UptimeSec": 9, "Heap": 180, "SleepMode": "Dynamic", "Sleep": 50, "LoadAvg": 19, "MqttCount": 1, "Berry":
{"HeapUsed": 4, "Objects": 45}, "POWER": "OFF", "Wifi": {"AP": 1, "SSID": "GOC C3
Coffee", "BSSID": "74:83:C2:C1:29:8F", "Channel": 11, "Mode": "HT40", "RSSI": 92, "Signal": -54, "LinkCount": 1, "Downtime": "0T00:00:04"}}
16:03:16.041 MQTT: Sensor AHT2X info Temperature is null.
16:03:16.042 MQTT: Sensor AHT2X info Humidity is null.
16:03:16.043 MQTT: Sensor AHT2X info DewPoint is null.
```

Trên web quản trị Core IoT, bạn cũng sẽ thấy trạng thái thiết bị sẽ từ Inactive chuyển sang Active.

Devices							Device Filter	Include customer entities	+	↺	🔍
☐	Created time ↓	Name	Device profile	Label	State	Customer name	🔒	🗑️			
☐	2025-02-14 10:11:22	Test Device	default	Test Device	Inactive		🔒	🗑️			
☐	2025-02-06 20:43:45	Sensor C1	Temperature Sensor		Inactive	Customer D	🔒	🗑️			
☐	2025-02-06 20:43:45	Sensor T1	Temperature Sensor		Inactive		🔒	🗑️			
☐	2025-02-05 13:28:41	Tasmota_Device1	default	Tasmota_Device1	Active		🔒	🗑️			
☐	2025-02-01 21:10:29	Device D1	default	Device D1	Inactive		🔒	🗑️			

Nhấn vào thiết bị, chuyển qua tab Telemetry, bạn cũng sẽ thấy được Tasmota gửi các thông tin từ cảm biến lên Core IoT với các key có format như Tên cảm biến-Thông số, ví dụ như AHT20X-Temperature.

Devices

Device Filter

Inc

☐	Created time ↓	Name
☐	2025-02-14 10:11:22	Test Device
☐	2025-02-06 20:43:45	Sensor C1
☐	2025-02-06 20:43:45	Sensor T1
☐	2025-02-05 13:28:41	Tasmota_Device1
☐	2025-02-01 21:10:29	Device D1

Tasmota_Device1

Device details

<

Details

Attributes

Latest telemetry

Alarms

Events

>

Telemetry

+

🔍

☐	Last update time	Key ↑	Value	
☐	2025-04-11 20:47:13	AHT2X-DewPoint	26.8	🗑️
☐	2025-04-11 20:47:13	AHT2X-Humidity	80.9	🗑️

Items per page:

10

1 – 10 of 38

<<

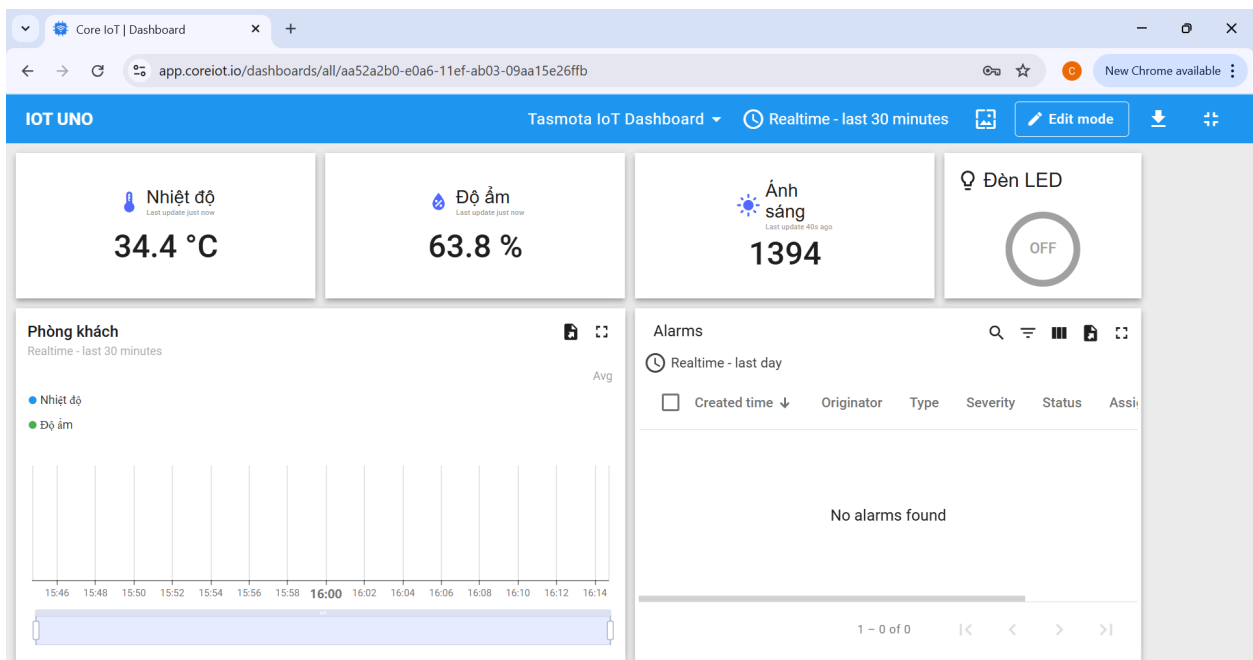
<

>

>>

11.5 Tạo bảng điều khiển (Dashboard) trong Core IoT

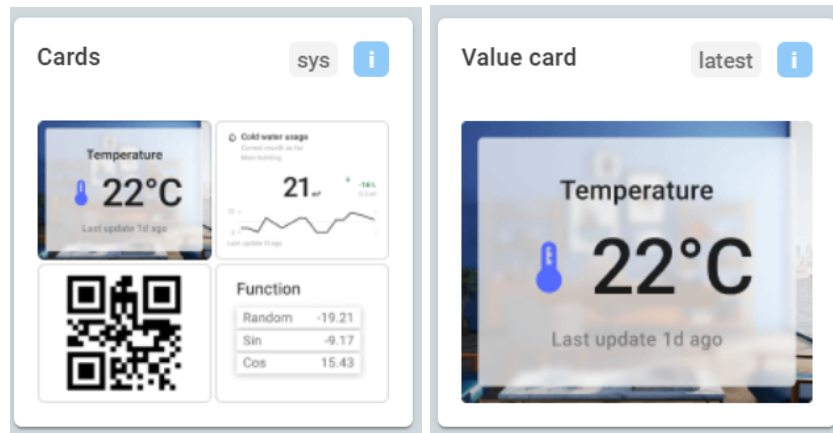
Để hiển thị trực quan các trạng thái bật tắt và giá trị các cảm biến của thiết bị, chúng ta sẽ tạo một Dashboard để hiển thị và điều khiển có các widget như sau:



Dashboard trên hiển thị được nhiệt độ, độ ẩm cùng đồ thị của 2 thông tin này do thiết bị chạy Tasmota gửi lên. Ngoài ra Dashboard cũng hỗ trợ bật tắt đèn qua nút nhấn.

Hiển thị thông tin cảm biến nhiệt độ và độ ẩm

Chúng ta cần 2 widget dạng Card value để hiển thị thông tin Nhiệt độ và độ ẩm. Cấu hình như hình sau:



Value card

Value card

Basic Advanced X

Preview Decline Apply

Datasource

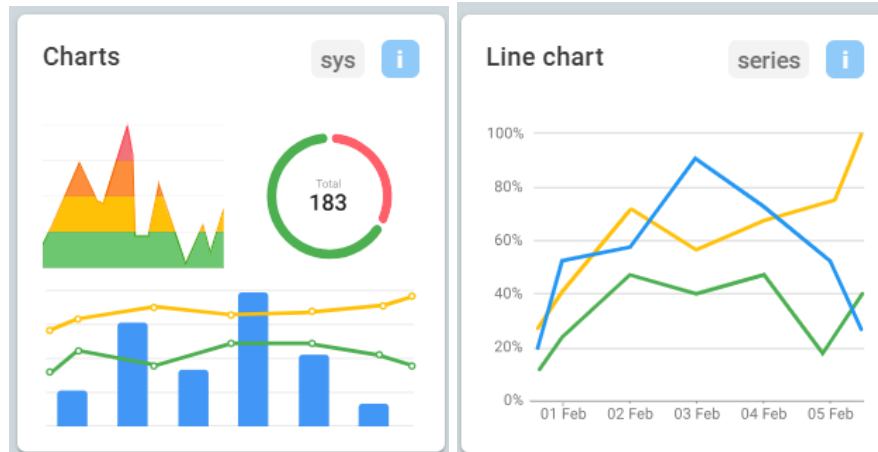
Device Entity alias

Device*
Tasmota_Device1 X

Data key*
AHT2X-Temperature X

Bạn cần đảm bảo chọn đúng tên thiết bị cần lấy dữ liệu để hiển thị và Telemetry key (ví dụ **AHT2X-Temperature** và **AHT2X-Humidity**).

Thêm một Chart widget để vẽ biểu đồ nhiệt độ và độ ẩm do thiết bị gửi lên Core IoT.



Device*
Tasmota_Device1

Series

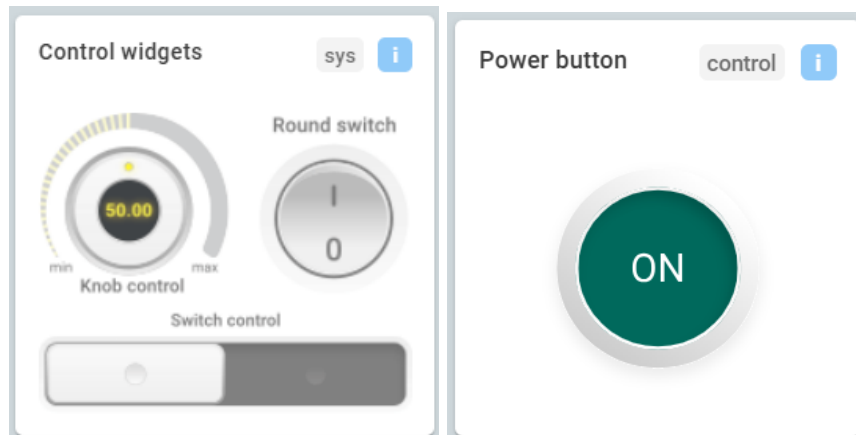
Key	Label	Type	Y axis	Color	Units	Decimals
~ AHT2X-Te... ✎ ✕	Nhiệt độ	📈	default	🟦	°C	0
~ AHT2X-Hu... ✎ ✕	Độ ẩm	📈	default	🟩	%	0

Sau khi tiến hành thêm và cấu hình các widget trong Dashboard như trên, bạn sẽ thấy thông tin từ Tasmota gửi lên định kỳ 300 giây (bạn có thể dùng lệnh **TelePeriod** để thay đổi khoảng thời gian này cho ngắn hơn để bớt phải chờ lâu)

Điều khiển relay và đồng bộ trạng thái

Để thể hiện trạng thái của Relay trên thiết bị, chúng ta cần widget loại điều khiển và sẽ gắn vào đúng attribute là POWER. Trường hợp thiết bị của bạn có nhiều relay điều khiển hơn 1 thì sẽ là POWER1, POWER2...

Bạn thêm widget và cấu hình như sau:



Đèn LED
Power button
Basic
Advanced
X

Preview
Decline
Apply

Target device
Device
Entity alias

Device*
Tasmota_Device1
X

Behavior

Initial state ⓘ
Use attribute 'POWER'

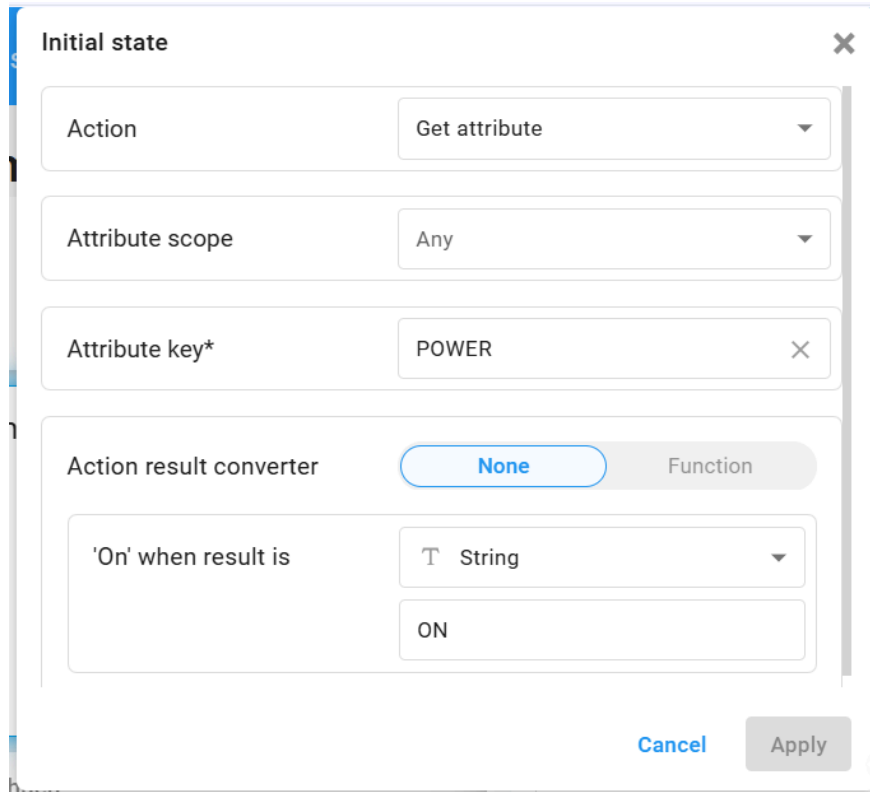
Power 'On' ⓘ
Execute RPC method 'POWER(1)'

Power 'Off' ⓘ
Execute RPC method 'POWER(0)'

Với một widget loại điều khiển, bạn sẽ thường cần có 3 cấu hình quan trọng:

1. Initial state

Cấu hình cho widget biết lấy trạng thái ban đầu như thế nào khi Dashboard mới được user mở ra. Ở đây chúng ta sẽ cho nút nhấn này luôn lấy trạng thái bật/tắt của Relay được gửi trong attribute có tên là POWER do Tasmota gửi lên.



Initial state

Action: Get attribute

Attribute scope: Any

Attribute key*: POWER

Action result converter: **None** (Function)

'On' when result is: String

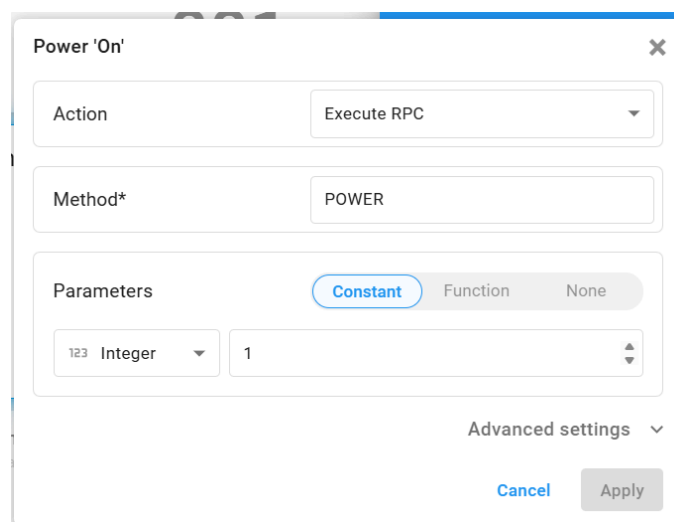
ON

Cancel Apply

Tasmota luôn gửi giá trị ON khi Relay được bật. Do đó ở đây ta khai báo nút nhấn này sẽ được chuyển sang trạng thái bật khi giá trị của attribute là ON.

2. Power 'On'

Cấu hình cho Core IoT sẽ làm gì khi user nhấn nút bật trên Dashboard. Ta sẽ cho gửi 1 lệnh xuống Tasmota, tương tự lệnh sử dụng để bật Relay là **Power 1**



Power 'On'

Action: Execute RPC

Method*: POWER

Parameters: **Constant** (Function, None)

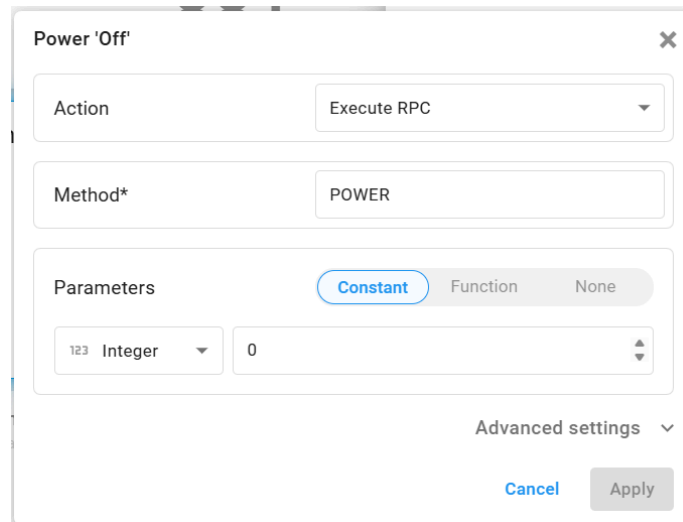
Integer: 1

Advanced settings

Cancel Apply

3. Power 'Off'

Cấu hình cho Core IoT sẽ làm gì khi user nhấn nút tắt trên Dashboard. Ta sẽ cho gửi 1 lệnh xuống Tasmota, tương tự lệnh sử dụng để bật Relay là **Power 0**



Để gửi một lệnh bất kỳ đến thiết bị chạy Tasmota, chúng ta sẽ dùng cơ chế RPC call của Core IoT, tên Method của RPC sẽ là tên command trong Tasmota, còn Parameters của RPC sẽ là tham số.

Ví dụ, trong Tasmota chúng ta có thể sử dụng command POWER 1 để bật relay 1. Như vậy trên widget, chúng ta sẽ chọn Execute RPC với Method là POWER, còn Parameters sẽ là kiểu Integer và giá trị là 1.

Sau khi bạn đã cấu hình trên Core IoT như các bước trên, bạn sẽ có một Dashboard có thể quan sát được các giá trị cảm biến của thiết bị Tasmota, đồng thời cũng có thể bật/tắt đèn LED từ Dashboard này.

Bạn có thể thử nhấn nút để bật/tắt LED và sẽ thấy trạng thái được đồng bộ trên WebUI của Tasmota lẫn Core IoT dashboard.

Kết thúc Module 11, bạn đã biết cách kết nối và tích hợp thiết bị Tasmota vào một nền tảng IoT cao cấp tiêu biểu là Core IoT. Giờ đây, thiết bị Tasmota của bạn đã thực sự trở thành một phần của một hệ sinh thái thông minh lớn hơn, sẵn sàng cho các kịch bản tự động hóa phong phú. Module cuối cùng sẽ tổng kết lại và giới thiệu một số tính năng nâng cao khác của Tasmota cũng như các tài nguyên để bạn tiếp tục học hỏi.

Module 12: Các tính năng nâng cao

Mục tiêu

Ngoài các chức năng cơ bản về điều khiển, đọc cảm biến và kết nối MQTT, Tasmota còn tích hợp nhiều tính năng nâng cao giúp bạn xây dựng các ứng dụng IoT linh hoạt và mạnh mẽ hơn. Module này sẽ giới thiệu một số tính năng đáng chú ý.

12.1 Hẹn giờ các tác vụ với Timers

- **Giới thiệu:** Tasmota cho phép bạn thiết lập tối đa **16 bộ hẹn giờ (Timers)** độc lập ngay trên thiết bị. Tính năng này giúp bạn tự động hóa việc bật/tắt các Relay (Relay/Power) hoặc kích hoạt các hành động khác theo một lịch trình định sẵn mà **không cần** đến sự can thiệp của hệ thống nhà thông minh trung tâm hay kết nối Internet.
- **Các chế độ hoạt động của Timer:**
 1. **Once (Một lần):** Kích hoạt vào một ngày và giờ cụ thể trong tương lai.
 2. **Repeat Daily (Lặp lại hàng ngày):** Kích hoạt vào một giờ cố định mỗi ngày.
 3. **Repeat Weekly (Lặp lại hàng tuần):** Kích hoạt vào một giờ cố định vào những ngày được chọn trong tuần (Thứ Hai, Thứ Ba...).
- **Cách cấu hình Timers (Sử dụng WebUI là dễ nhất):**

Tasmota

Timer parameters

☒ **Enable Timers**

12345678910111213141516

Output 1 ▾
Action On ▾

☒ **Enable** ☐ **Repeat**

☒ **Time**
☐ **Sunrise** (12:02)
☐ **Sunset** (01:38)

16 ▾ : 40 ▾ +/- 00 ▾

☒ **Sun**
☒ **Mon**
☐ **Tue**
☐ **Wed**
☐ **Thu**
☐ **Fri**
☐ **Sat**

Save

1. Truy cập WebUI Tasmota, vào menu **Configuration -> Timers**.
2. Bạn sẽ thấy giao diện quản lý 16 Timers. Chọn một Timer bạn muốn cài đặt (ví dụ: Timer 1).
3. **Enable:** Đánh dấu vào ô này để bật Timer.
4. **Mode:** Chọn chế độ hoạt động (Once, Repeat daily, hoặc chọn các ngày trong tuần).
5. **Time:** Nhập thời gian muốn kích hoạt theo định dạng HH:MM (ví dụ: **18:00** cho 6 giờ tối).
6. **Output:** Chọn Relay (Power1, Power2...) mà Timer này sẽ tác động đến.
7. **Action:** Chọn hành động sẽ thực hiện khi đến giờ: **ON** (Bật), **OFF** (Tắt), **TOGGLE** (Đảo trạng thái), hoặc **RULE** (Kích hoạt trigger **Rules#Timer=x** để xử lý bằng Tasmota Rules).
8. **(Tùy chọn) Random:** Có thể thêm một khoảng thời gian ngẫu nhiên (tối đa +/- 60 phút) vào thời gian kích hoạt, hữu ích để mô phỏng sự có mặt ở nhà.

9. Nhấn "Save".

- **Yêu cầu về thời gian:** Để Timers chạy chính xác, thiết bị Tasmota cần phải biết thời gian hiện tại. Nó thường tự động đồng bộ thời gian qua Internet (NTP) sau khi kết nối Wi-Fi. Bạn cần đảm bảo đã cấu hình đúng **Múi giờ (Timezone)** cho khu vực của mình (ví dụ: **Timezone +07:00** hoặc **Timezone 99** cho Việt Nam) thông qua Console hoặc menu Configure Time.
- **Ứng dụng:** Tự động bật đèn sân thượng lúc 18:00 và tắt lúc 22:00 hàng ngày; Hẹn giờ bật máy pha cà phê vào buổi sáng; Tắt các thiết bị không cần thiết vào nửa đêm để tiết kiệm điện...

12.2 Device Groups: Đồng bộ Nhóm Thiết bị

- **Giới thiệu:** Tính năng này cho phép bạn liên kết nhiều thiết bị Tasmota lại thành một nhóm để chúng **tự động đồng bộ trạng thái bật/tắt** với nhau. Khi bạn thay đổi trạng thái của một thiết bị trong nhóm (ví dụ: nhấn nút bật đèn A), các thiết bị khác trong cùng nhóm (đèn B, đèn C) cũng sẽ tự động bật theo.
- **Cách hoạt động:** Giao tiếp diễn ra **trực tiếp giữa các thiết bị** trong cùng mạng Wi-Fi bằng cách sử dụng các gói tin UDP multicast, **không cần thông qua MQTT Broker**.
- **Cách cấu hình:** (Phải thực hiện trên **TẤT CẢ** các thiết bị bạn muốn đưa vào cùng một nhóm)
 1. Truy cập **Console** của từng thiết bị.
 2. **Đặt tên nhóm (GroupTopic):** Dùng lệnh **GroupTopic <tên_nhóm_chung>**.
Ví dụ, nếu muốn nhóm các đèn tầng 1, bạn đặt lệnh **GroupTopic den_tang_1** trên tất cả các thiết bị đèn tầng 1. Tên nhóm nên là duy nhất, không dấu, không khoảng trắng. (Bạn có thể tạo tối đa 4 nhóm khác nhau bằng **GroupTopic1** đến **GroupTopic4**).
 3. **(Quan trọng) Kích hoạt gửi trạng thái:** Mặc định, thiết bị chỉ "lắng nghe" tin nhắn nhóm. Để nó cũng "nói" (gửi đi trạng thái thay đổi của mình cho các thiết bị khác trong nhóm), bạn cần dùng lệnh: **SetOption20 1**. Phải đặt lệnh này trên tất cả các thiết bị trong nhóm.
 4. **(Tùy chọn) Liên kết Relay với Nhóm:** Mặc định, **GroupTopic** (hoặc **GroupTopic1**) sẽ đồng bộ trạng thái của **Power1**. Nếu bạn muốn đồng bộ

Relay khác (ví dụ Power2) hoặc đồng bộ với nhóm khác (GroupTopic2), bạn cần dùng các lệnh `SetOption` tương ứng (ví dụ: `SetOption38 1` để Power2 đồng bộ với GroupTopic1, `SetOption37 2` để Power1 đồng bộ với GroupTopic2...).

Tham khảo tài liệu Tasmota SetOptions.

- **Ưu điểm:** Đồng bộ trạng thái bật/tắt rất nhanh chóng, hoạt động cục bộ không cần Broker.
- **Nhược điểm:** Chỉ hoạt động trong cùng một mạng LAN. Giao thức UDP multicast có thể không ổn định hoặc bị chặn trên một số cấu hình mạng Wi-Fi phức tạp. Chỉ đồng bộ trạng thái ON/OFF cơ bản, không đồng bộ độ sáng hay màu sắc.

12.3 Energy Monitoring: Giám sát năng lượng & Hiệu chỉnh

- **Giới thiệu:** Nhiều thiết bị thông minh (ổ cắm, công tắc) tương thích Tasmota được trang bị chip phần cứng chuyên dụng để đo lường các thông số về điện năng tiêu thụ (ví dụ: chip HLW8012, CSE7766, BL0937...). Tasmota có thể giao tiếp với các chip này để cung cấp cho bạn thông tin chi tiết.
- **Thiết bị hỗ trợ:** Bạn cần kiểm tra xem thiết bị của mình có phần cứng đo năng lượng hay không và Template/Module Type bạn chọn có kích hoạt tính năng này không. Ví dụ: Sonoff POW R2, Sonoff S31, Shelly 1PM, nhiều ổ cắm Tuya có chức năng đo điện.
- **Dữ liệu cung cấp:**
 - **WebUI:** Thường hiển thị trên Main Menu các thông số như: Điện áp (V), Dòng điện (A), Công suất tức thời (W), Năng lượng tiêu thụ Hôm nay (kWh), Hôm qua (kWh), Tổng cộng (kWh).
 - **MQTT:** Dữ liệu chi tiết hơn được gửi định kỳ qua topic `tele/%topic%/SENSOR` trong một đối tượng JSON có tên là `Energy`. Bao gồm cả Công suất biểu kiến (VA), Công suất phản kháng (VAr), Hệ số công suất (Power Factor)...
- **Hiệu chỉnh (Calibration):**
 - Do sai số của linh kiện phần cứng, các giá trị đo lường ban đầu có thể không hoàn toàn chính xác. Tasmota cho phép bạn **hiệu chỉnh (calibrate)** lại các thông số này nếu bạn có thiết bị đo lường chuẩn (ví dụ: đồng hồ vạn năng True RMS chất lượng tốt, thiết bị đo công suất chuyên dụng).

- **Quy trình:** Yêu cầu bạn sử dụng một tải điện ổn định (như bóng đèn sợi đốt), đo chính xác Điện áp, Dòng điện, Công suất bằng thiết bị chuẩn, sau đó dùng các lệnh trên Console Tasmota như `VoltageSet <giá_trị_chuẩn>`, `CurrentSet <giá_trị_chuẩn_tính_bằng_mA>`, `PowerSet <giá_trị_chuẩn_W>` để Tasmota tự điều chỉnh lại hệ số đo lường.
- **Lưu ý:** Đây là quy trình nâng cao, đòi hỏi thiết bị đo chuẩn và sự cẩn thận. Nếu không có thiết bị chuẩn, bạn có thể bỏ qua bước này vì sai số thường ở mức chấp nhận được cho việc theo dõi thông thường. Hãy đọc kỹ tài liệu "Power Monitoring Calibration" của Tasmota nếu muốn thực hiện.

12.4 Over-the-Air (OTA): Cập nhật Firmware qua mạng

- **OTA là gì?** Là khả năng cập nhật phiên bản firmware mới cho thiết bị Tasmota của bạn **qua mạng Wi-Fi**, mà **không cần** phải tháo thiết bị ra và kết nối lại với máy tính bằng cáp USB. Đây là cách cực kỳ tiện lợi để nâng cấp lên các phiên bản Tasmota mới hơn với nhiều tính năng, sửa lỗi và cải tiến bảo mật.
- **Cách thực hiện:**
 - **Qua WebUI (Dễ nhất):**
 1. Vào WebUI -> **Firmware Upgrade**.
 2. **"Upgrade by web server"**: Nhấn nút **"Start upgrade"**. Tasmota sẽ tự động kết nối đến máy chủ OTA chính thức (ota.tasmota.com), tải về phiên bản ổn định mới nhất phù hợp với thiết bị của bạn và tự động cài đặt. Đây là cách **khuyến nghị**.
 3. **"Upgrade by file upload"**: Nếu bạn muốn cài một phiên bản cụ thể hoặc một bản firmware tùy chỉnh đã tải về máy tính (file `.bin` hoặc `.bin.gz`), nhấn "Choose File", chọn file đó, rồi nhấn "Start upgrade".
 - **Qua Console:** Dùng lệnh `Upgrade 1` (nâng cấp bản ổn định) hoặc `Upgrade 0` (nâng cấp bản phát triển) hoặc `Upgrade <URL_cua_file_bin>`.
 - **Qua MQTT:** Publish `1, 0`, hoặc `<URL>` đến topic `cmnd/%topic%/Upgrade`.
- **Lưu ý quan trọng với thiết bị 1MB Flash (ESP8266):**
 - Do bộ nhớ hạn chế, quá trình OTA trên các thiết bị này diễn ra theo 2 bước:
 1. Tasmota tải và nạp một phiên bản **tasmota-minimal.bin** siêu nhẹ.

2. Thiết bị khởi động lại vào bản minimal, sau đó từ bản minimal này mới tải và nạp phiên bản đầy đủ bạn muốn (ví dụ `tasmota.bin`).
 3. Thiết bị khởi động lại lần nữa vào phiên bản đầy đủ mới.
 - Do đó, OTA trên thiết bị 1MB sẽ **lâu hơn** và **khởi động lại 2 lần**. Đừng ngắt nguồn giữa chừng!
- **Trước khi OTA:**
 - Đảm bảo kết nối Wi-Fi ổn định.
 - Nên **sao lưu cấu hình** (Backup Configuration) để phòng bất trắc. (Thường thì cấu hình sẽ được giữ nguyên).
 - Đọc "Release Notes" trên GitHub của Tasmota nếu nâng cấp giữa các phiên bản lớn để biết các thay đổi quan trọng.

12.5 Displays: Cấu hình màn hình hiển thị cơ bản

- **Khả năng:** Tasmota có thể điều khiển và hiển thị thông tin lên các loại màn hình LCD hoặc OLED nhỏ được kết nối qua giao tiếp I2C hoặc SPI.
- **Yêu cầu Firmware:** Tính năng này **không có trong các bản build tiêu chuẩn**. Bạn cần sử dụng một bản firmware chuyên dụng như `tasmota-displays.bin` (hoặc `tasmota32-displays.bin`) hoặc **tự biên dịch firmware** với các tùy chọn màn hình được kích hoạt.
- **Các loại màn hình phổ biến:** OLED I2C (SSD1306, SH1106), LCD I2C (1602, 2004 với module I2C), TFT SPI (ILI9341, ST77xx)...
- **Cấu hình cơ bản (Ví dụ OLED SSD1306 I2C):**
 1. Nạp firmware `tasmota-displays.bin`.
 2. Kết nối màn hình: VCC-3.3V, GND-GND, SDA-GPIOx, SCL-GPIOy.
 3. Vào **Configure Module**, gán chức năng **I2C SDA** và **I2C SCL** cho các chân GPIOx, GPIOy tương ứng. Lưu và khởi động lại.
 4. Vào **Console**, dùng lệnh `DisplayModel <số_model>` để chọn loại màn hình (ví dụ: `DisplayModel 2` cho SSD1306). Dùng `I2CScan` để tìm địa chỉ I2C nếu cần.
 5. Màn hình sẽ hiển thị thông tin mặc định. Dùng lệnh `DisplayMode` để chọn kiểu hiển thị có sẵn, hoặc `DisplayText` để hiển thị nội dung tùy chỉnh.

- **Ứng dụng:** Làm đồng hồ, hiển thị nhiệt độ/độ ẩm, trạng thái thiết bị, thông báo ngắn...

12.6 Deep Sleep Modes: Chế độ Ngủ cho thiết bị dùng Pin

- **Deep Sleep là gì?** Là trạng thái **siêu tiết kiệm năng lượng** của chip ESP. Khi vào Deep Sleep, hầu hết các thành phần (CPU, Wi-Fi...) đều bị tắt, chỉ còn bộ đếm thời gian RTC hoạt động với mức tiêu thụ năng lượng cực thấp (vài microAmpe).
- **Tại sao quan trọng?** Đây là chìa khóa để chế tạo các thiết bị Tasmota **chạy bằng pin** có thể hoạt động trong thời gian dài (hàng tuần, hàng tháng).
- **Hoạt động:**
 - Thiết bị thức dậy, kết nối Wi-Fi, thực hiện công việc (đọc cảm biến, gửi MQTT...).
 - Sau khi xong việc, nó tự đưa mình vào Deep Sleep trong một khoảng thời gian định trước.
 - Hết thời gian ngủ, bộ đếm RTC sẽ kích hoạt tín hiệu Reset, làm chip **khởi động lại hoàn toàn**.
 - Chu trình lặp lại.
- **Cấu hình:**
 - **Lệnh kích hoạt:** `DeepSleepTime <số_giây_ngủ>` (Ví dụ: `DeepSleepTime 600` để ngủ 10 phút). Đặt là `0` để tắt Deep Sleep.
 - **Kết nối Wake-up (Chỉ cho ESP8266): BẮT BUỘC** phải nối chân **GPIO16** (thường là D0) với chân **RST** trên bo mạch ESP8266. Nếu không có kết nối này, nó sẽ ngủ luôn không dậy được! (ESP32 thường không cần).
 - **Tối ưu:** Cần giảm thiểu thời gian "thức" bằng cách tối ưu kết nối Wi-Fi/MQTT, tắt tính năng không cần thiết...
- **Hạn chế:** Thiết bị **không thể truy cập/điều khiển** được khi đang ngủ. Nó chỉ online trong khoảng thời gian ngắn khi thức dậy.
- **Ứng dụng:** Cảm biến môi trường chạy pin, cảm biến cửa/chuyển động gửi thông báo khi có sự kiện...

Kết thúc Module 12, bạn đã khám phá thêm nhiều công cụ và kỹ thuật mạnh mẽ trong Tasmota, từ việc tự động hóa theo lịch trình, đồng bộ thiết bị, giám sát năng lượng, cập nhật OTA, hiển thị thông tin, cho đến việc tối ưu hóa cho thiết bị dùng pin. Những kiến thức này giúp bạn xây dựng các giải pháp IoT phức tạp và hiệu quả hơn.

Module 13: Tùy chỉnh Firmware Tasmota

Mục tiêu

Trong các module trước, chúng ta đã làm quen với việc sử dụng các phiên bản firmware Tasmota được biên dịch sẵn. Tuy nhiên, để khai thác tối đa tiềm năng của Tasmota và đáp ứng các nhu cầu cụ thể của dự án, việc tự biên dịch firmware là một kỹ năng quan trọng. Module này sẽ hướng dẫn bạn cách thiết lập môi trường phát triển và tùy chỉnh firmware Tasmota theo ý muốn.

13.1 Tại sao nên tự biên dịch Firmware?

Việc tự biên dịch firmware Tasmota mang lại nhiều lợi ích đáng kể, đặc biệt khi bạn muốn thực hiện các dự án phức tạp hoặc có yêu cầu đặc biệt:

- **Thêm Tính năng:** Tasmota có một cộng đồng phát triển lớn mạnh và hỗ trợ vô số tính năng. Tuy nhiên, không phải tất cả các tính năng này đều được hỗ trợ trong các phiên bản firmware tiêu chuẩn. Khi tự biên dịch, bạn có toàn quyền lựa chọn và kích hoạt các tính năng bạn cần, chẳng hạn như:
 - Hỗ trợ các loại cảm biến hoặc thiết bị cụ thể không được hỗ trợ mặc định.
 - Kích hoạt các giao thức truyền thông đặc biệt (ví dụ: KNX, EnOcean).
 - Thêm các lệnh hoặc chức năng tùy chỉnh.
- **Tối ưu kích thước:** Các phiên bản firmware được biên dịch sẵn thường bao gồm nhiều tính năng để đáp ứng nhu cầu đa dạng của người dùng. Điều này có thể dẫn đến kích thước firmware lớn hơn cần thiết cho dự án cụ thể của bạn. Khi tự biên dịch, bạn có thể loại bỏ các tính năng không cần thiết, giúp giảm đáng kể kích thước firmware. Điều này đặc biệt hữu ích cho các thiết bị có bộ nhớ flash hạn chế.
- **Tùy chỉnh sâu hơn:** Bạn có thể thực hiện các tùy chỉnh sâu hơn vào mã nguồn của Tasmota nếu cần thiết (mặc dù điều này đòi hỏi kiến thức lập trình C++).

- **Cập nhật các bản vá và tính năng mới nhất:** Bạn có thể biên dịch firmware từ mã nguồn mới nhất trên GitHub, đảm bảo bạn luôn có các bản vá lỗi và tính năng mới nhất.
- **Học hỏi và hiểu rõ hơn về Tasmota:** Quá trình tự biên dịch giúp bạn hiểu rõ hơn về cấu trúc và cách hoạt động bên trong của Tasmota.

13.2 Thiết lập Môi trường phát triển PlatformIO (VS Code)

Để tự biên dịch firmware Tasmota, chúng ta sẽ sử dụng PlatformIO IDE, một môi trường phát triển mã nguồn mở mạnh mẽ, thường được tích hợp với Visual Studio Code (VS Code).

Các bước thiết lập:

1. Cài đặt Visual Studio Code (VS Code):

- Truy cập trang web chính thức của VS Code (<https://code.visualstudio.com/>) và tải xuống phiên bản phù hợp với hệ điều hành của bạn.
- Thực hiện cài đặt theo hướng dẫn.

2. Cài đặt PlatformIO IDE Extension:

- Mở VS Code.
- Nhấp vào biểu tượng **Extensions** (hình vuông nhỏ ở thanh bên trái) hoặc nhấn **Ctrl+Shift+X** (Windows/Linux) hoặc **Cmd+Shift+X** (macOS).
- Tìm kiếm "PlatformIO IDE" trong thanh tìm kiếm.
- Nhấp vào nút **Install** bên cạnh kết quả "PlatformIO IDE".
- Đợi quá trình cài đặt hoàn tất. Sau khi cài đặt xong, bạn có thể cần khởi động lại VS Code.

3. Mở dự án Tasmota:

- Tải xuống mã nguồn Tasmota từ GitHub. Bạn có thể truy cập trang repository chính thức tại <https://github.com/arendst/Tasmota> hoặc phiên bản có hỗ trợ Core IoT tại <https://github.com/ohstem-public/Tasmota-CoreIoT> và nhấp vào nút **Code** sau đó chọn **Download ZIP**. Giải nén file ZIP vào một thư mục trên máy tính của bạn.
- Trong VS Code, nhấp vào **File > Open Folder...** và chọn thư mục mã nguồn Tasmota bạn vừa giải nén.

- PlatformIO sẽ tự động nhận diện dự án và hiển thị các tùy chọn liên quan ở thanh bên trái (biểu tượng PlatformIO).

13.3 Tùy chỉnh Build Flags và user_config_override.h

Đây là hai phương pháp chính để tùy chỉnh firmware Tasmota mà không cần trực tiếp chỉnh sửa mã nguồn C++.

13.3.1 Tùy chỉnh Cờ biên dịch (Build Flags) trong `platformio.ini`:

Tệp `platformio.ini` nằm ở thư mục gốc của dự án Tasmota và chứa các cấu hình cho quá trình biên dịch. Chúng ta có thể sử dụng các "build flags" để bật hoặc tắt các tính năng.

- Mở file `platformio.ini`:** Tìm và mở file này trong VS Code.
- Tìm đến phần `[env:...]`:** Phần này định nghĩa các môi trường biên dịch khác nhau cho các loại thiết bị khác nhau. Bạn sẽ cần chọn môi trường phù hợp với thiết bị của mình (ví dụ: `[env:tasmota]`, `[env:tasmota-minimal]`, `[env:tasmota-display]`, v.v.). Nếu bạn không chắc chắn, `[env:tasmota]` thường là một lựa chọn tốt cho hầu hết các thiết bị ESP8266/ESP32.

Thêm hoặc sửa đổi dòng `build_flags`: Trong phần môi trường bạn đã chọn, thêm hoặc chỉnh sửa dòng sau:

```
build_flags =
    ${env.build_flags}
    -D <TÊN_TÍNH_NĂNG>
    -U <TÊN_TÍNH_NĂNG>
```

`-D <TÊN_TÍNH_NĂNG>`: Được sử dụng để **kích hoạt** một tính năng. Bạn cần thay thế `<TÊN_TÍNH_NĂNG>` bằng tên của tính năng bạn muốn bật (thường được định nghĩa trong các file cấu hình của Tasmota).

Ví dụ: để bật hỗ trợ cho cảm biến DHT, bạn có thể thêm `-D USE_DHT`.

-U <TÊN_TÍNH_NĂNG>: Được sử dụng để **vô hiệu hóa** một tính năng mặc định. Thay thế **<TÊN_TÍNH_NĂNG>** bằng tên của tính năng bạn muốn tắt.

Ví dụ: Để bật hỗ trợ DHT và vô hiệu hóa hỗ trợ cho màn hình LCD (giả sử bạn không sử dụng), bạn có thể thêm vào phần `[env:tasmota]`:

```
build_flags =  
    ${env.build_flags}  
    -D USE_DHT  
    -U USE_LCD
```

Bạn có thể tìm thấy danh sách các build flags có sẵn trong tài liệu chính thức của Tasmota hoặc bằng cách xem các file cấu hình trong thư mục `tasmota/user_config.h` và các file tương tự.

Tùy chỉnh bằng file `user_config_override.h`:

Tệp `user_config_override.h` cho phép bạn ghi đè các cấu hình mặc định của Tasmota mà không cần chỉnh sửa trực tiếp các file cấu hình gốc. Đây là cách được khuyến nghị để thực hiện các tùy chỉnh của bạn.

- **Tạo file `user_config_override.h`:**
 - Trong thư mục gốc của dự án Tasmota, tạo một thư mục mới có tên là `user_config_override`.
 - Bên trong thư mục `user_config_override`, tạo một file mới có tên là `user_config_override.h`.

Thêm các định nghĩa tùy chỉnh: Mở file `user_config_override.h` và thêm các dòng `#define` để kích hoạt hoặc cấu hình các tính năng. Cú pháp tương tự như khi sử dụng build flags với `-D`.

```

#ifndef USER_CONFIG_OVERRIDE_H_
#define USER_CONFIG_OVERRIDE_H_

// Kích hoạt hỗ trợ cảm biến DHT
#define USE_DHT

// Đặt tên WiFi mặc định
#define WIFI_SSID "YourWiFiNetwork"
#define WIFI_PASSWORD "YourWiFiPassword"

// Kích hoạt giao thức MQTT
#define USE_MQTT

// Đặt máy chủ MQTT
#define MQTT_HOST "your_mqtt_broker_ip"
#define MQTT_PORT 1883

// ... Thêm các tùy chỉnh khác tại đây ...

#endif

```

Lưu ý quan trọng:

- Đảm bảo bạn bọc nội dung của file bằng `#ifndef USER_CONFIG_OVERRIDE_H_`, `#define USER_CONFIG_OVERRIDE_H_`, và `#endif` để tránh lỗi định nghĩa lại.
- Bạn có thể tìm thấy các tùy chọn cấu hình có sẵn trong các file `*.h` khác nhau trong thư mục `tasmota/`. Đặc biệt, hãy tham khảo kỹ file `tasmota/user_config.h`.

Khi nào nên sử dụng Build Flags và khi nào nên sử dụng `user_config_override.h`?

- **Build Flags** (trong `platformio.ini`) thường được sử dụng để bật hoặc tắt các tính năng lớn hoặc các thành phần cốt lõi của Tasmota.
- `user_config_override.h` thường được sử dụng để cấu hình các tham số cụ thể, đặt các giá trị mặc định, hoặc kích hoạt các tùy chọn chi tiết hơn cho các tính năng đã được bật.

Trong nhiều trường hợp, bạn sẽ sử dụng kết hợp cả hai phương pháp. Ví dụ, bạn có thể sử dụng build flag để bật hỗ trợ một loại cảm biến (`USE_SENSOR_XYZ`) và sau đó sử dụng

`user_config_override.h` để đặt các thông số cấu hình cụ thể cho cảm biến đó (ví dụ: chân GPIO kết nối).

13.4 Biên dịch và nạp Firmware tùy chỉnh của bạn

Sau khi bạn đã thực hiện các tùy chỉnh mong muốn bằng cách sử dụng build flags hoặc `user_config_override.h`, bạn đã sẵn sàng để biên dịch và nạp firmware tùy chỉnh lên thiết bị Tasmota của mình.

Các bước biên dịch:

1. **Mở PlatformIO:** Trong VS Code, nhấp vào biểu tượng PlatformIO ở thanh bên trái.
2. **Chọn môi trường biên dịch:** Trong phần **Project Tasks**, mở rộng môi trường biên dịch bạn đã chọn trong `platformio.ini` (ví dụ: `tasmota`).
3. **Nhấp vào "Build":** PlatformIO sẽ bắt đầu quá trình biên dịch firmware dựa trên các cấu hình bạn đã thiết lập.
4. **Theo dõi quá trình biên dịch:** Bạn sẽ thấy thông tin chi tiết về quá trình biên dịch trong cửa sổ "Terminal" ở phía dưới. Nếu có lỗi, hãy kiểm tra lại các cấu hình của bạn và đảm bảo không có lỗi chính tả hoặc cú pháp.
5. **Tìm file firmware đã biên dịch:** Sau khi biên dịch thành công, file firmware (thường có tên `env_name.bin` với `env_name` là tên môi trường mà bạn sử dụng để build và được lưu trong thư mục `build_output/firmware/`).

Các bước nạp Firmware:

Quá trình nạp firmware tùy chỉnh tương tự như nạp các phiên bản firmware được biên dịch sẵn. Bạn có thể sử dụng nhiều phương pháp khác nhau, tùy thuộc vào thiết bị của bạn:

- **Sử dụng Tasmota Web Installer:** Truy cập trang web <https://tasmota.github.io/install/> và làm theo hướng dẫn để nạp firmware qua trình duyệt web (phương pháp này thường áp dụng cho các thiết bị mới chưa được flash Tasmota). Bạn sẽ cần chọn file `firmware.bin` bạn vừa biên dịch.

- **Sử dụng công cụ ESP Flasher:** Các công cụ như `esptool.py` (dành cho ESP8266/ESP32) hoặc các ứng dụng GUI như ESP Flash Download Tool có thể được sử dụng để nạp firmware qua kết nối nối tiếp (UART). Bạn sẽ cần kết nối thiết bị của mình với máy tính qua cáp USB-to-serial adapter.
 - **Ví dụ sử dụng `esptool.py`:**
 - Đảm bảo bạn đã cài đặt `esptool.py` (`pip install esptool`).
 - Xác định cổng nối tiếp của thiết bị (ví dụ: `/dev/ttyUSB0` trên Linux/macOS hoặc `COM3` trên Windows).
 - Đưa thiết bị về chế độ flash (thường bằng cách giữ nút GPIO0 khi cấp nguồn).

Chạy lệnh sau trong terminal (thay thế các giá trị phù hợp):

```
esptool.py --port <cổng_nối_tiếp> erase_flash
esptool.py --port <cổng_nối_tiếp> --baud 115200 write_flash 0x0
<đường_dẫn_đến_firmware.bin>
```

- **Sử dụng OTA (Over-The-Air) từ firmware Tasmota hiện tại:** Nếu thiết bị của bạn đã chạy một phiên bản Tasmota, bạn có thể nạp firmware tùy chỉnh qua giao diện web bằng cách truy cập trang **Firmware Upgrade** và chọn file `firmware.bin` của bạn.

Quan trọng: Luôn sao lưu cấu hình hiện tại của thiết bị trước khi nạp firmware mới.

13.5 Giới thiệu về Berry Scripting cho các tùy chỉnh Nâng cao

Berry là một ngôn ngữ kịch bản nhẹ và nhanh chóng được tích hợp vào Tasmota. Nó cung cấp một cách mạnh mẽ và linh hoạt để tùy chỉnh hành vi của thiết bị mà không cần phải viết mã C++ và biên dịch lại toàn bộ firmware.

Tại sao nên sử dụng Berry?

- **Dễ học và sử dụng:** Berry có cú pháp tương đối đơn giản, dễ tiếp cận đối với người dùng không có nhiều kinh nghiệm lập trình C++.
- **Thực thi nhanh chóng:** Berry được thiết kế để có hiệu suất tốt trên các thiết bị nhúng.
- **Không cần biên dịch lại firmware:** Bạn có thể tải lên và chạy các script Berry trực tiếp trên thiết bị thông qua giao diện web hoặc MQTT. Điều này giúp quá trình phát triển và thử nghiệm trở nên nhanh chóng hơn.
- **Truy cập vào các chức năng và API của Tasmota:** Berry script có thể tương tác với nhiều khía cạnh của Tasmota, bao gồm:
 - Đọc và ghi trạng thái của các thiết bị và cảm biến.
 - Điều khiển các chân GPIO.
 - Gửi và nhận dữ liệu qua MQTT.
 - Tương tác với các giao thức web.
 - Lập lịch các tác vụ.

Các ứng dụng phổ biến của Berry Scripting:

- **Tạo các Rule và script tự động hóa phức tạp:** Bạn có thể viết các script để thực hiện các hành động dựa trên các sự kiện hoặc điều kiện cụ thể (ví dụ: bật đèn khi trời tối và có người).
- **Tùy chỉnh giao diện web:** Berry cho phép bạn tạo các trang web tùy chỉnh để điều khiển và giám sát thiết bị của mình.
- **Xử lý dữ liệu cảm biến:** Bạn có thể viết script để xử lý và lọc dữ liệu từ các cảm biến trước khi gửi chúng qua MQTT hoặc hiển thị trên giao diện web.
- **Tích hợp với các dịch vụ và API bên ngoài:** Berry có thể được sử dụng để tương tác với các dịch vụ web khác nhau (ví dụ: gửi thông báo Telegram, điều khiển các thiết bị thông minh khác qua API).

Bắt đầu với Berry

- **Truy cập giao diện Berry:** Trong giao diện web của Tasmota, bạn có thể tìm thấy một mục "Berry" hoặc tương tự. Tại đây, bạn có thể tạo, chỉnh sửa và chạy các script Berry.
- **Tìm hiểu cú pháp Berry:** Bạn có thể tìm thấy tài liệu và ví dụ về cú pháp Berry trên trang web chính thức của Tasmota Wiki.

- **Thử nghiệm với các ví dụ:** Bắt đầu với các ví dụ đơn giản để làm quen với cách Berry tương tác với Tasmota.

Berry scripting mở ra một thế giới khả năng tùy chỉnh nâng cao cho Tasmota. Mặc dù nó có thể đòi hỏi một chút thời gian để làm quen, nhưng nó là một công cụ vô cùng mạnh mẽ cho những người muốn kiểm soát sâu hơn các thiết bị thông minh của mình.

Module 14: Xây dựng Smart Home thực tế

Mục tiêu

Trong module này, chúng ta sẽ áp dụng những kiến thức đã học để xây dựng một dự án Smart Home thực tế, có thể triển khai trong một ngôi nhà thật chứ không chỉ dừng lại ở các mô phỏng cơ bản. Thông qua dự án thực tế này, bạn sẽ hiểu rõ hơn về cách Tasmota có thể được sử dụng để tự động hóa ngôi nhà và tạo ra các giải pháp thông minh.

Hệ thống nhà thông minh này sẽ bao gồm:

14.1 Tính năng 1: Ổ cắm Thông minh Wi-Fi

- **Mục tiêu:** Xây dựng một ổ cắm điện có thể điều khiển từ xa qua mạng Wi-Fi bằng Tasmota.

14.2 Tính năng 2: Thiết bị giám sát môi trường

- **Mục tiêu:** Xây dựng một thiết bị có khả năng đo nhiệt độ, độ ẩm và cường độ ánh sáng, sau đó gửi dữ liệu này đến Core IoT để bạn có thể theo dõi và để ý chất lượng không khí trong phòng làm việc hoặc phòng ở của mình.

14.3 Tính năng 3: Công tắc tường thông minh

- **Mục tiêu:** Thay thế công tắc tường bật tắt đèn truyền thống bằng công tắc thông minh cảm ứng chạy Tasmota để có thể điều khiển đèn từ xa qua Wi-Fi.

14.4 Tính năng 4: Điều khiển dây LED màu RGB

- **Mục tiêu:** Điều khiển một dải đèn LED địa chỉ (ví dụ: WS2812B, NeoPixel) bằng Tasmota để tạo ra các hiệu ứng chiếu sáng đẹp mắt và phù hợp trong nhiều ngữ cảnh.

14.5 Tính năng 5: Bộ điều khiển hồng ngoại cho TV và máy lạnh

- **Mục tiêu:** Xây dựng một thiết bị có thể bật tắt hoặc điều chỉnh nhiệt độ cho các thiết bị điều khiển qua remote IR như TV, máy lạnh.

14.5 Tính năng 6: Bộ điều khiển trung tâm

- **Mục tiêu:** Xây dựng một thiết bị điều khiển trung tâm với màn hình cảm ứng 4 inch, có thể bật tắt các thiết bị trong nhà, đồng thời quan sát được các thông số khác cũng như bật các ngữ cảnh đã được thiết lập sẵn.

Do tính phức tạp và thường xuyên cập nhật của các tính năng này, phần hướng dẫn sẽ được chúng tôi mô tả chi tiết ở đường dẫn này https://github.com/ohstem-public/smart_home_project

Các hướng dẫn sẽ bao gồm đầy đủ cấu hình, script, mã nguồn và cả các thiết kế 3D nếu có trong đường dẫn trên và sẽ liên tục được cập nhật trong thời gian tới.

Module 15: Bảo mật trong Tasmota

Mục tiêu

Khi bạn tích hợp các thiết bị Tasmota vào mạng gia đình hoặc văn phòng của mình, việc đảm bảo an ninh cho chúng và mạng của bạn là điều tối quan trọng. Module này sẽ giới thiệu các phương pháp bảo mật tốt nhất mà bạn nên áp dụng để bảo vệ hệ thống của mình khỏi các truy cập trái phép và các mối đe dọa tiềm ẩn.

15.1 Bảo mật Mạng Wi-Fi của bạn (WPA2/WPA3)

Nền tảng cho mọi kết nối của thiết bị Tasmota là mạng Wi-Fi của bạn. Một mạng Wi-Fi không an toàn có thể tạo điều kiện cho kẻ xấu xâm nhập vào các thiết bị Tasmota và các thiết bị khác trên mạng của bạn.

- **Sử dụng giao thức mã hóa mạnh mẽ:**
 - **WPA2 (Wi-Fi Protected Access 2):** Đây là giao thức mã hóa Wi-Fi tiêu chuẩn và nên được sử dụng nếu có thể. Đảm bảo router Wi-Fi của bạn được cấu hình để sử dụng WPA2 với mã hóa AES (thường là tùy chọn mặc định). Tránh sử dụng các giao thức cũ hơn như WEP hoặc WPA1 vì chúng có nhiều lỗ hổng bảo mật đã được biết đến.
 - **WPA3 (Wi-Fi Protected Access 3):** Đây là giao thức mã hóa mới nhất và an toàn nhất hiện có. Nếu router và các thiết bị của bạn (bao gồm cả ESP8266/ESP32 chạy Tasmota) hỗ trợ WPA3, bạn nên ưu tiên sử dụng giao thức này để có mức bảo mật cao nhất. Tuy nhiên, hãy lưu ý rằng không phải tất cả các thiết bị cũ hơn đều tương thích với WPA3.
- **Chọn mật khẩu mạnh:**

- Mật khẩu Wi-Fi của bạn nên **dài ít nhất 12 ký tự** và bao gồm sự kết hợp của chữ hoa, chữ thường, số và ký tự đặc biệt.
- Tránh sử dụng các mật khẩu dễ đoán như ngày sinh, tên, hoặc các từ thông dụng.
- Cân nhắc sử dụng một trình quản lý mật khẩu để tạo và lưu trữ các mật khẩu mạnh một cách an toàn.
- Thay đổi mật khẩu Wi-Fi của bạn định kỳ, đặc biệt nếu bạn nghi ngờ có ai đó đã truy cập trái phép vào mạng của bạn.

- **Kiểm tra cấu hình router:**

- Đăng nhập vào trang quản trị của router Wi-Fi (thường có địa chỉ IP như **192.168.1.1** hoặc **192.168.0.1**).
- Tìm đến phần cài đặt Wi-Fi hoặc Wireless.
- Đảm bảo rằng giao thức bảo mật được đặt thành WPA2-PSK (AES) hoặc WPA3-SAE (nếu hỗ trợ).
- Kiểm tra và thay đổi mật khẩu Wi-Fi nếu cần.

15.2 Đặt mật khẩu giao diện Web quản trị của Tasmota

Tasmota cung cấp một giao diện web để cấu hình và điều khiển thiết bị. Nếu bạn không đặt mật khẩu cho giao diện này, bất kỳ ai có quyền truy cập vào mạng của bạn đều có thể thay đổi cài đặt của thiết bị, điều khiển nó hoặc thậm chí nạp firmware độc hại.

- **Cách đặt mật khẩu:**

- Truy cập địa chỉ IP của thiết bị Tasmota trên trình duyệt web.
- Nhấp vào nút **Configuration**.
- Chọn **Configure Password**.
- Nhập mật khẩu bạn muốn đặt vào cả hai trường **New password** và **Confirm password**.
- Nhấp vào nút **Save**.

- Lần sau khi bạn truy cập giao diện web, bạn sẽ được yêu cầu nhập tên người dùng (mặc định là `admin`) và mật khẩu bạn vừa đặt.

- **Lưu ý quan trọng:**

- Chọn một mật khẩu mạnh và duy nhất cho Tasmota, khác với mật khẩu Wi-Fi của bạn.
- Ghi nhớ mật khẩu này hoặc lưu trữ nó một cách an toàn. Nếu bạn quên mật khẩu, bạn có thể cần phải flash lại firmware.

15.3 Bảo mật cho giao thức MQTT

Nếu bạn sử dụng MQTT để giao tiếp với các thiết bị Tasmota của mình (ví dụ: thông qua một hệ thống nhà thông minh như Home Assistant), việc bảo mật MQTT broker là rất quan trọng.

- **Sử dụng tên người dùng và mật khẩu:**

- Hầu hết các MQTT broker đều hỗ trợ việc thiết lập tên người dùng và mật khẩu để xác thực các client kết nối.
- Đảm bảo bạn đã cấu hình tên người dùng và mật khẩu mạnh cho MQTT broker của mình.
- Trong cấu hình Tasmota (trong phần **Configure MQTT**), hãy nhập đúng tên người dùng và mật khẩu bạn đã thiết lập trên broker.

- **Thiết lập Danh sách Kiểm soát Truy cập (ACLs):**

- ACLs cho phép bạn kiểm soát chi tiết những client nào có quyền publish (gửi) hoặc subscribe (nhận) dữ liệu trên các topic MQTT cụ thể.
- Việc thiết lập ACLs giúp hạn chế những hành động mà một client có thể thực hiện, ngay cả khi họ đã được xác thực.
- Cách thiết lập ACLs sẽ khác nhau tùy thuộc vào MQTT broker bạn đang sử dụng (ví dụ: Mosquitto, EMQX). Tham khảo tài liệu của broker bạn để biết hướng dẫn chi tiết.

- **Ví dụ với Mosquitto:** Bạn có thể cấu hình file `mosquitto.conf` để định nghĩa quyền truy cập cho từng người dùng đối với các topic cụ thể. Ví dụ: bạn có thể cho phép thiết bị Tasmota của mình chỉ publish lên topic liên quan đến trạng thái của nó và subscribe vào topic lệnh điều khiển dành riêng cho nó.
- **Sử dụng TLS/SSL (Tùy chọn nhưng được khuyến nghị):**
 - TLS/SSL mã hóa lưu lượng truy cập MQTT giữa client (Tasmota) và broker, ngăn chặn việc nghe lén dữ liệu.
 - Việc thiết lập TLS/SSL phức tạp hơn một chút và đòi hỏi bạn phải tạo và quản lý chứng chỉ. Tuy nhiên, nó cung cấp một lớp bảo mật bổ sung quan trọng, đặc biệt nếu MQTT broker của bạn có thể truy cập từ bên ngoài mạng cục bộ.

15.4 Network Segmentation (tùy chọn)

Network Segmentation (phân đoạn mạng) là một kỹ thuật bảo mật nâng cao, nhưng nó có thể mang lại lợi ích đáng kể cho việc bảo vệ các thiết bị IoT như những thiết bị chạy Tasmota.

- **Khái niệm:** Phân đoạn mạng liên quan đến việc chia mạng vật lý của bạn thành nhiều mạng logic riêng biệt. Điều này có thể được thực hiện bằng nhiều cách, chẳng hạn như sử dụng mạng khách (guest network) hoặc VLANs (Virtual Local Area Networks).
- **Lợi ích cho bảo mật IoT:**
 - **Cô lập thiết bị:** Bằng cách đặt các thiết bị IoT trên một mạng riêng biệt, bạn có thể giới hạn khả năng chúng truy cập vào các thiết bị quan trọng khác trên mạng chính của bạn (ví dụ: máy tính cá nhân, máy chủ lưu trữ). Nếu một thiết bị IoT bị xâm nhập, kẻ tấn công sẽ khó có thể di chuyển ngang sang các phần khác của mạng.
 - **Kiểm soát lưu lượng truy cập:** Bạn có thể thiết lập các quy tắc tường lửa để kiểm soát lưu lượng truy cập giữa các phân đoạn mạng khác nhau, chỉ cho phép các kết nối cần thiết.

- **Các phương pháp phân đoạn mạng:**
 - **Mạng khách (Guest Network):** Nhiều router hiện đại cung cấp tính năng mạng khách. Đây là một cách dễ dàng để tạo một mạng riêng biệt cho các thiết bị IoT. Tuy nhiên, mạng khách thường có những hạn chế về khả năng giao tiếp với mạng chính.
 - **VLANs (Virtual Local Area Networks):** VLANs là một giải pháp mạnh mẽ hơn cho phép bạn tạo nhiều mạng logic trên cùng một cơ sở hạ tầng mạng vật lý. Việc thiết lập VLANs thường đòi hỏi router và switch hỗ trợ VLAN và cấu hình phức tạp hơn.
- **Cân nhắc:** Việc triển khai phân đoạn mạng có thể phức tạp và có thể không cần thiết cho tất cả người dùng. Tuy nhiên, nếu bạn có nhiều thiết bị IoT hoặc lo ngại về bảo mật, đây là một biện pháp đáng cân nhắc. Bạn có thể bắt đầu với việc sử dụng mạng khách nếu router của bạn hỗ trợ.

Việc áp dụng những phương pháp bảo mật này sẽ giúp bạn bảo vệ các thiết bị Tasmota và mạng của mình khỏi các mối đe dọa tiềm ẩn, đảm bảo một hệ thống nhà thông minh an toàn và đáng tin cậy.

Module 16: Khắc phục các sự cố

Mục tiêu

Trong quá trình làm việc với Tasmota và các dự án IoT, bạn có thể gặp phải một số vấn đề không mong muốn. Module này sẽ trang bị cho bạn các kỹ năng và tài nguyên cần thiết để chẩn đoán, khắc phục sự cố và tiếp tục hành trình khám phá thế giới IoT.

16.1 Các vấn đề phổ biến và cách chẩn đoán

Dưới đây là một số vấn đề phổ biến mà bạn có thể gặp phải khi làm việc với Tasmota và cách tiếp cận để chẩn đoán chúng:

- **Vấn đề về Wi-Fi:**
 - **Thiết bị không kết nối được với Wi-Fi:**
 - **Kiểm tra cấu hình:** Đảm bảo bạn đã nhập đúng tên mạng (SSID) và mật khẩu Wi-Fi trong cấu hình Tasmota.
 - **Kiểm tra cường độ tín hiệu:** Di chuyển thiết bị gần router hơn để kiểm tra xem tín hiệu Wi-Fi có đủ mạnh không.
 - **Khởi động lại router và thiết bị:** Thử khởi động lại cả router Wi-Fi và thiết bị Tasmota.
 - **Kiểm tra giới hạn kết nối:** Một số router có giới hạn về số lượng thiết bị có thể kết nối.
 - **Sử dụng Wi-Fi Analyzer:** Sử dụng ứng dụng Wi-Fi analyzer trên điện thoại hoặc máy tính để kiểm tra kênh Wi-Fi đang được sử dụng và xem xét chuyển sang kênh ít bị nhiễu hơn.
 - **Kiểm tra cấu hình tường lửa router:** Đôi khi tường lửa trên router có thể chặn kết nối.

- **Kết nối Wi-Fi không ổn định:**
 - **Kiểm tra nhiễu sóng:** Các thiết bị điện tử khác có thể gây nhiễu sóng Wi-Fi.
 - **Cập nhật firmware router:** Đảm bảo router của bạn đang chạy phiên bản firmware mới nhất.
- **Vấn đề về MQTT:**
 - **Thiết bị không kết nối được với MQTT Broker:**
 - **Kiểm tra cấu hình:** Đảm bảo bạn đã nhập đúng địa chỉ IP/tên miền, cổng, tên người dùng và mật khẩu của MQTT broker trong cấu hình Tasmota.
 - **Kiểm tra trạng thái MQTT trên Console:** Xem bảng điều khiển Tasmota để biết thông báo lỗi liên quan đến MQTT.
 - **Kiểm tra MQTT Broker:** Đảm bảo MQTT broker đang chạy và có thể truy cập được từ mạng của bạn. Sử dụng một MQTT client khác (ví dụ: MQTT Explorer) để kiểm tra kết nối đến broker.
 - **Kiểm tra tường lửa:** Đảm bảo không có tường lửa nào chặn kết nối đến cổng MQTT (thường là 1883).
 - **Dữ liệu MQTT không được gửi hoặc nhận:**
 - **Kiểm tra topic:** Đảm bảo topic publish và subscribe được cấu hình chính xác trên cả Tasmota và hệ thống nhà thông minh của bạn.
 - **Kiểm tra định dạng payload:** Đảm bảo định dạng dữ liệu MQTT (thường là JSON) đúng cú pháp.
 - **Theo dõi topic bằng MQTT client:** Sử dụng MQTT Explorer hoặc một công cụ tương tự để theo dõi các topic liên quan và xem dữ liệu có được gửi và nhận như mong đợi không.
- **Vấn đề về phần cứng:**
 - **Thiết bị không hoạt động sau khi flash firmware:**
 - **Kiểm tra nguồn điện:** Đảm bảo thiết bị được cấp nguồn đầy đủ và đúng cách.

- **Kiểm tra kết nối:** Đảm bảo các kết nối phần cứng (nếu có) được thực hiện chính xác và chắc chắn.
 - **Thử flash lại firmware:** Có thể quá trình flash đã bị lỗi. Thử flash lại firmware.
 - **Kiểm tra serial output (nếu có thể):** Kết nối thiết bị với máy tính qua serial adapter để xem các thông báo khởi động và lỗi.
- **Cảm biến không báo cáo dữ liệu:**
 - **Kiểm tra kết nối:** Đảm bảo cảm biến được kết nối đúng chân với ESP.
 - **Kiểm tra cấu hình module:** Đảm bảo bạn đã chọn đúng loại cảm biến và chân GPIO trong cấu hình Tasmota.
 - **Kiểm tra nguồn điện cho cảm biến:** Một số cảm biến cần nguồn điện riêng.
 - **Tham khảo datasheet của cảm biến:** Đảm bảo bạn đang sử dụng cảm biến đúng cách và nó đang hoạt động bình thường.
- **Rơle không hoạt động:**
 - **Kiểm tra kết nối:** Đảm bảo rơle được kết nối đúng cách với ESP và nguồn điện.
 - **Kiểm tra cấu hình module:** Đảm bảo chân GPIO điều khiển rơle đã được cấu hình đúng.
 - **Kiểm tra trạng thái GPIO trên Console:** Sử dụng lệnh `GPIO` trên console để kiểm tra trạng thái của chân điều khiển rơle.
 - **Kiểm tra nguồn điện cho rơle:** Một số rơle cần nguồn điện riêng để hoạt động.

16.2 Sử dụng hiệu quả công cụ Console và Syslog của Tasmota

Bảng điều khiển (Console) và Syslog là những công cụ mạnh mẽ để chẩn đoán và khắc phục sự cố trong Tasmota.

- **Bảng điều khiển (Console):**

- **Truy cập:** Bạn có thể truy cập bảng điều khiển Tasmota thông qua giao diện web (trong menu **Console**) hoặc bằng cách kết nối thiết bị với máy tính qua serial adapter (sử dụng phần mềm như Arduino Serial Monitor hoặc PuTTY).
- **Thông tin khởi động:** Khi thiết bị khởi động, bảng điều khiển sẽ hiển thị các thông tin quan trọng như phiên bản Tasmota, cấu hình Wi-Fi, trạng thái MQTT, và các thông báo lỗi.
- **Theo dõi hoạt động:** Trong quá trình hoạt động, bảng điều khiển sẽ hiển thị các sự kiện, trạng thái cảm biến, và các thông báo khác.
- **Gửi lệnh:** Bạn có thể nhập các lệnh Tasmota trực tiếp vào bảng điều khiển để kiểm tra và cấu hình thiết bị. Một số lệnh hữu ích bao gồm:
 - **status:** Hiển thị thông tin trạng thái chung của thiết bị.
 - **status 0:** Hiển thị thông tin cấu hình Wi-Fi.
 - **status 2:** Hiển thị thông tin cấu hình MQTT.
 - **status 10:** Hiển thị thông tin cấu hình module.
 - **wifi:** Hiển thị thông tin chi tiết về kết nối Wi-Fi.
 - **mqtt:** Hiển thị thông tin chi tiết về kết nối MQTT.
 - **gpio:** Hiển thị trạng thái của các chân GPIO.
 - **<TênLệnh> ?:** Hiển thị trợ giúp về một lệnh cụ thể (ví dụ: **wifi ?**).
- **Quan sát lỗi:** Các thông báo lỗi thường được hiển thị rõ ràng trên bảng điều khiển, giúp bạn xác định nguyên nhân gây ra sự cố.

- **Syslog:**

- **Cấu hình:** Syslog cho phép bạn gửi các thông điệp nhật ký (logs) của Tasmota đến một máy chủ syslog riêng biệt trên mạng của bạn. Điều này hữu ích cho việc theo dõi và phân tích các vấn đề xảy ra không liên tục hoặc khó nắm bắt. Bạn có thể cấu hình máy chủ syslog trong **Configuration > Configure Logging**.
- **Xem nhật ký:** Sử dụng phần mềm máy chủ syslog trên máy tính hoặc thiết bị của bạn để xem các nhật ký được gửi từ Tasmota. Các nhật ký này có thể cung cấp thông tin chi tiết hơn về hoạt động của thiết bị theo thời gian.

16.3 Tận dụng tài liệu chính thức của Tasmota

Tài liệu chính thức của Tasmota Wiki (<https://tasmota.github.io/docs/>) là một nguồn tài nguyên vô giá cho mọi người dùng Tasmota.

- **Thông tin toàn diện:** Wiki chứa đựng thông tin chi tiết về mọi khía cạnh của Tasmota, bao gồm:
 - Hướng dẫn cài đặt và cấu hình.
 - Mô tả chi tiết về tất cả các tính năng và lệnh.
 - Hướng dẫn kết nối và cấu hình nhiều loại phần cứng khác nhau.
 - Các giải pháp cho các vấn đề thường gặp.
 - Thông tin về cách tùy chỉnh firmware.
- **Tìm kiếm hiệu quả:** Sử dụng chức năng tìm kiếm trên Wiki để nhanh chóng tìm thấy thông tin bạn cần.
- **Khám phá theo chủ đề:** Wiki được tổ chức theo các chủ đề rõ ràng, giúp bạn dễ dàng duyệt qua các phần khác nhau.
- **Đóng góp từ cộng đồng:** Wiki được duy trì và cập nhật bởi cộng đồng người dùng Tasmota, đảm bảo thông tin luôn mới và chính xác.

Lời khuyên: Khi gặp bất kỳ vấn đề nào, hãy luôn tham khảo tài liệu chính thức trước tiên. Rất có thể câu trả lời hoặc giải pháp bạn đang tìm kiếm đã được ghi lại ở đó.

16.4 Hướng đi tiếp theo: Mở rộng kỹ năng IoT của bạn

Chúc mừng bạn đã hoàn thành khóa học về Tasmota! Tuy nhiên, đây chỉ là bước khởi đầu trong hành trình khám phá thế giới rộng lớn của Internet of Things. Dưới đây là một số gợi ý về những hướng đi tiếp theo để bạn có thể mở rộng kỹ năng của mình:

- **Khám phá các nền tảng nhà thông minh khác:** Tasmota thường được sử dụng kết hợp với các nền tảng nhà thông minh như Home Assistant, openHAB, Node-RED. Hãy thử tìm hiểu và làm việc với các nền tảng này để xây dựng một hệ thống nhà thông minh toàn diện hơn.

- **Học thêm về các giao thức IoT khác:** Ngoài MQTT, còn có nhiều giao thức IoT khác như Zigbee, Z-Wave, ESP-NOW. Tìm hiểu về chúng có thể mở ra những khả năng mới cho các dự án của bạn.
- **Nâng cao kỹ năng lập trình:** Nếu bạn muốn tùy chỉnh Tasmota sâu hơn hoặc phát triển các ứng dụng IoT của riêng mình, việc học các ngôn ngữ lập trình như C++, Python hoặc JavaScript sẽ rất hữu ích.
- **Tham gia vào các dự án mã nguồn mở:** Đóng góp vào các dự án mã nguồn mở như Tasmota là một cách tuyệt vời để học hỏi, chia sẻ kiến thức và xây dựng cộng đồng.
- **Thực hiện các dự án phức tạp hơn:** Đừng ngại thử thách bản thân với các dự án IoT phức tạp hơn, kết hợp nhiều thiết bị và công nghệ khác nhau.
- **Theo dõi các xu hướng mới:** Thế giới IoT đang phát triển rất nhanh chóng. Hãy theo dõi các tin tức, blog và diễn đàn để cập nhật những xu hướng và công nghệ mới nhất.
- **Chia sẻ kiến thức của bạn:** Khi bạn đã có kinh nghiệm, hãy chia sẻ kiến thức của mình với những người mới bắt đầu. Bạn có thể viết blog, tạo video hướng dẫn hoặc tham gia vào các diễn đàn và nhóm cộng đồng.

Đây là module cuối cùng của bộ tài liệu. Hy vọng bộ tài liệu này đã cung cấp cho bạn những công cụ mạnh mẽ và động lực để bạn có thể tiếp tục khám phá thế giới IoT.

Hành trình IoT là một hành trình học hỏi không ngừng. Hãy luôn tò mò, khám phá và đừng ngại thử nghiệm những điều mới mẻ. Chúc bạn gặt hái được nhiều thành công trên con đường này!